

Squirrel

Generated by Doxygen 1.9.8

1 Data Structure Index	1
1.1 Data Structures	1
2 File Index	3
2.1 File List	3
3 Data Structure Documentation	5
3.1 BufState Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Field Documentation	5
3.1.2.1 buf	5
3.1.2.2 ptr	6
3.1.2.3 size	6
4 File Documentation	7
4.1 squirrel3_squirrel_sqapiByGemini.cpp File Reference	7
4.1.1 Macro Definition Documentation	14
4.1.1.1 _GETSAFE_OBJ	14
4.1.1.2 sq_aux_paramscheck	14
4.1.2 Function Documentation	14
4.1.2.1 _getmemberbyhandle()	14
4.1.2.2 buf_lexfeed()	16
4.1.2.3 sq_addrf()	17
4.1.2.4 sq_arrayappend()	17
4.1.2.5 sq_arrayinsert()	18
4.1.2.6 sq_arraypop()	18
4.1.2.7 sq_arrayremove()	19
4.1.2.8 sq_arrayresize()	20
4.1.2.9 sq_arrayreverse()	21
4.1.2.10 sq_aux_gettypedarg()	22
4.1.2.11 sq_aux_invalidtype()	22
4.1.2.12 sq_bindenv()	23
4.1.2.13 sq_call()	25
4.1.2.14 sq_clear()	25
4.1.2.15 sq_clone()	26
4.1.2.16 sq_close()	27
4.1.2.17 sq_cmp()	27
4.1.2.18 sq_collectgarbage()	28
4.1.2.19 sq_compile()	28
4.1.2.20 sq_compilebuffer()	29
4.1.2.21 sq_createinstance()	30
4.1.2.22 sq_deleteslot()	31
4.1.2.23 sq_enableddebuginfo()	32

4.1.2.24 sq_free()	32
4.1.2.25 sq_get()	33
4.1.2.26 sq_getattributes()	33
4.1.2.27 sq_getbase()	34
4.1.2.28 sq_getbool()	35
4.1.2.29 sq_getbyhandle()	35
4.1.2.30 sq_getcallee()	36
4.1.2.31 sq_getclass()	37
4.1.2.32 sq_getclosureinfo()	37
4.1.2.33 sq_getclosurename()	38
4.1.2.34 sq_getclosuresroot()	39
4.1.2.35 sq_getdefaultdelegate()	40
4.1.2.36 sq_getdelegate()	41
4.1.2.37 sq_geterrorfunc()	42
4.1.2.38 sq_getfloat()	42
4.1.2.39 sq_getforeignptr()	43
4.1.2.40 sq_getfreevariable()	43
4.1.2.41 sq_gethash()	44
4.1.2.42 sq_getinstanceup()	45
4.1.2.43 sq_getinteger()	46
4.1.2.44 sq_getlasterror()	46
4.1.2.45 sq_getlocal()	47
4.1.2.46 sq_getmemberhandle()	47
4.1.2.47 sq_getobjtypetag()	48
4.1.2.48 sq_getprintfunc()	49
4.1.2.49 sq_getrefcount()	49
4.1.2.50 sq_getreleasehook()	50
4.1.2.51 sq_getscratchpad()	51
4.1.2.52 sq_getsharedforeignptr()	51
4.1.2.53 sq_getsharedreleasehook()	52
4.1.2.54 sq_getsize()	53
4.1.2.55 sq_getstackobj()	54
4.1.2.56 sq_getstring()	54
4.1.2.57 sq_getstringandsize()	55
4.1.2.58 sq_getthread()	56
4.1.2.59 sq_gettop()	56
4.1.2.60 sq_gettype()	57
4.1.2.61 sq_gettypetag()	57
4.1.2.62 sq_getuserdata()	58
4.1.2.63 sq_getuserpointer()	59
4.1.2.64 sq_getversion()	59
4.1.2.65 sq_getvmrefcount()	60

4.1.2.66 sq_getvmreleasehook()	60
4.1.2.67 sq_getvmstate()	61
4.1.2.68 sq_getweakrefval()	61
4.1.2.69 sq_instanceof()	62
4.1.2.70 sq_malloc()	63
4.1.2.71 sq_move()	63
4.1.2.72 sq_newarray()	64
4.1.2.73 sq_newclass()	64
4.1.2.74 sq_newclosure()	65
4.1.2.75 sq_newmember()	66
4.1.2.76 sq_newslot()	67
4.1.2.77 sq_newtable()	68
4.1.2.78 sq_newtableex()	68
4.1.2.79 sq_newthread()	69
4.1.2.80 sq_newuserdata()	69
4.1.2.81 sq_next()	70
4.1.2.82 sq_notifyallexceptions()	71
4.1.2.83 sq_objtobool()	71
4.1.2.84 sq_objtofloat()	71
4.1.2.85 sq_objtointeger()	72
4.1.2.86 sq_objtostring()	72
4.1.2.87 sq_objtouserpointer()	73
4.1.2.88 sq_open()	73
4.1.2.89 sq_pop()	74
4.1.2.90 sq_poptop()	75
4.1.2.91 sq_push()	75
4.1.2.92 sq_pushbool()	75
4.1.2.93 sq_pushconsttable()	76
4.1.2.94 sq_pushfloat()	76
4.1.2.95 sq_pushinteger()	76
4.1.2.96 sq_pushnull()	77
4.1.2.97 sq_pushobject()	77
4.1.2.98 sq_pushregistrytable()	78
4.1.2.99 sq_pushroottable()	78
4.1.2.100 sq_pushstring()	79
4.1.2.101 sq_pushthread()	79
4.1.2.102 sq_pushuserpointer()	79
4.1.2.103 sq_rawdeleteslot()	80
4.1.2.104 sq_rawget()	81
4.1.2.105 sq_rawnewmember()	82
4.1.2.106 sq_rawset()	83
4.1.2.107 sq_readclosure()	84

4.1.2.108 sq_realloc()	85
4.1.2.109 sq_release()	85
4.1.2.110 sq_remove()	86
4.1.2.111 sq_reservestack()	86
4.1.2.112 sq_reseterror()	87
4.1.2.113 sq_resetobject()	88
4.1.2.114 sq_resume()	88
4.1.2.115 sq_resurrectunreachable()	89
4.1.2.116 sq_set()	90
4.1.2.117 sq_setattributes()	90
4.1.2.118 sq_setbyhandle()	91
4.1.2.119 sq_setclassudsize()	92
4.1.2.120 sq_setclosureroot()	93
4.1.2.121 sq_setcompilererrorhandler()	94
4.1.2.122 sq_setconsttable()	94
4.1.2.123 sq_setdebughook()	95
4.1.2.124 sq_setdelegate()	96
4.1.2.125 sq_seterrorhandler()	97
4.1.2.126 sq_setforeignptr()	97
4.1.2.127 sq_setfreevariable()	98
4.1.2.128 sq_setinstanceup()	99
4.1.2.129 sq_setnativeclosurename()	99
4.1.2.130 sq_setnativedebughook()	100
4.1.2.131 sq_setparamscheck()	101
4.1.2.132 sq_setprintfunc()	102
4.1.2.133 sq_setreleasehook()	102
4.1.2.134 sq_setroottable()	103
4.1.2.135 sq_setsharedforeignptr()	104
4.1.2.136 sq_setsharedreleasehook()	104
4.1.2.137 sq_settop()	104
4.1.2.138 sq_settypetag()	105
4.1.2.139 sq_setvmreleasehook()	106
4.1.2.140 sq_suspendvm()	107
4.1.2.141 sq_tailcall()	107
4.1.2.142 sq_throwerror()	108
4.1.2.143 sq_throwobject()	109
4.1.2.144 sq_tobool()	110
4.1.2.145 sq_tostring()	110
4.1.2.146 sq_typeof()	111
4.1.2.147 sq_wakeupvm()	112
4.1.2.148 sq_weakref()	113
4.1.2.149 sq_writeclosure()	113

4.2 squirrel3_squirrel_sqapiByGemini.cpp	114
--	-----

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

BufState

`sq_compilebuffer` 関数がメモリバッファからソースコードを読み込む際に使用する状態を保持する構造体。

5

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

squirrel3_squirrel_sqapiByGemini.cpp		7
--	-------	---

Chapter 3

Data Structure Documentation

3.1 BufState Struct Reference

`sq_compilebuffer` 関数がメモリバッファからソースコードを読み込む際に使用する状態を保持する構造体。

Data Fields

- `const SQChar * buf`
ソースコードが格納されたバッファへのポインタ。
- `SQInteger ptr`
バッファ内の現在の読み込み位置。
- `SQInteger size`
バッファの総サイズ。

3.1.1 Detailed Description

`sq_compilebuffer` 関数がメモリバッファからソースコードを読み込む際に使用する状態を保持する構造体。

この構造体は、`sq_compilebuffer` 関数が `sq_compile` と連携する際に、メモリ上のソースコードバッファをファイルのように見せかけるために使われます。`sq_compile` は `SQLLEXREADFUNC` 型の読み込み関数を要求するため、`buf_lexfeed` 関数がこの構造体を `SQUserPointer` として受け取り、バッファからの文字の読み進め状態を管理します。

Definition at line 2614 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

3.1.2 Field Documentation

3.1.2.1 buf

```
const SQChar* BufState::buf
```

ソースコードが格納されたバッファへのポインタ。

Definition at line 2615 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

Referenced by [buf_lexfeed\(\)](#), and [sq_compilebuffer\(\)](#).

3.1.2.2 ptr

```
SQInteger BufState::ptr
```

バッファ内の現在の読み込み位置。

Definition at line 2616 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

Referenced by [buf_lexfeed\(\)](#), and [sq_compilebuffer\(\)](#).

3.1.2.3 size

```
SQInteger BufState::size
```

バッファの総サイズ。

Definition at line 2617 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

Referenced by [buf_lexfeed\(\)](#), and [sq_compilebuffer\(\)](#).

The documentation for this struct was generated from the following file:

- [squirrel3_squirrel_sqapiByGemini.cpp](#)

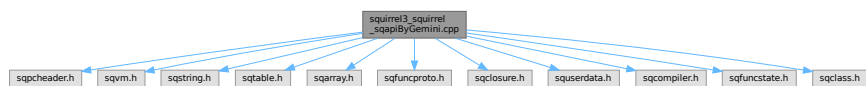
Chapter 4

File Documentation

4.1 squirrel3_squirrel_sqapiByGemini.cpp File Reference

```
#include "sqpcheader.h"
#include "sqvm.h"
#include "sqstring.h"
#include "sqtable.h"
#include "sqarray.h"
#include "sqfuncproto.h"
#include "sqclosure.h"
#include "squserdata.h"
#include "sqcompiler.h"
#include "sqfuncstate.h"
#include "sqclass.h"
```

Include dependency graph for squirrel3_squirrel_sqapiByGemini.cpp:



Data Structures

- struct [BufState](#)

sq_compilebuffer 関数がメモリバッファからソースコードを読み込む際に使用する状態を保持する構造体。

Macros

- #define [_GETSAFE_OBJ](#)(v, idx, type, o) { if(![sq_aux_gettypedarg](#)(v,idx,type,&o)) return SQ_ERROR; }
型をチェックし、安全にオブジェクトポインタを取得するための内部マクロ。
- #define [sq_aux_paramscheck](#)(v, count)
API 関数呼び出し時にスタック上に十分なパラメータが存在するかをチェックする内部マクロ。

Functions

- static bool [sq_aux_gettypedarg](#) (HSQUIRRELVm v, SQInteger idx, SQObjectType type, SQObjectPtr **o)
指定されたスタックインデックスの引数が期待される型であるかを確認し、そのオブジェクトポインタを取得します。
- SQInteger [sq_aux_invalidtype](#) (HSQUIRRELVm v, SQObjectType type)
予期しない型が指定された場合にエラーをスローします。
- HSQUIRRELVm [sq_open](#) (SQInteger initialstacksize)
新しい *Squirrel VM* インスタンスを作成し、開きます。
- HSQUIRRELVm [sq_newthread](#) (HSQUIRRELVm friendvm, SQInteger initialstacksize)
既存の *VM* と共有状態を共有する新しいスレッド (軽量 *VM*) を作成します。
- SQInteger [sq_getvmstate](#) (HSQUIRRELVm v)
VM の現在の状態を取得します。
- void [sq_seterrorhandler](#) (HSQUIRRELVm v)
VM のグローバルエラーハンドラを設定します。
- void [sq_setnatividebughook](#) (HSQUIRRELVm v, SQDEBUGHOOK hook)
ネイティブコード (C/C++) で実装されたデバッグフックを設定します。
- void [sq_setdebughook](#) (HSQUIRRELVm v)
Squirrel スクリプトで実装されたデバッグフックを設定します。
- void [sq_close](#) (HSQUIRRELVm v)
VM を閉じ、関連するすべてのリソースを解放します。
- SQInteger [sq_getversion](#) ()
Squirrel のバージョン番号を取得します。
- SQRESULT [sq_compile](#) (HSQUIRRELVm v, SQLEXREADFUNC read, SQUserPointer p, const SQChar *sourcename, SQBool raiseerror)
Squirrel ソースコードを読み込み、コンパイルして実行可能なクロージャを生成します。
- void [sq_enableddebuginfo](#) (HSQUIRRELVm v, SQBool enable)
デバッグ情報の生成を有効または無効にします。
- void [sq_notifyallexceptions](#) (HSQUIRRELVm v, SQBool enable)
すべての例外を通知するかどうかを設定します。
- void [sq_addrf](#) (HSQUIRRELVm v, HSQOBJECT *po)
Squirrel オブジェクトの参照カウントをインクリメントします。
- SQUnsignedInteger [sq_getrefcount](#) (HSQUIRRELVm v, HSQOBJECT *po)
Squirrel オブジェクトの現在の参照カウントを取得します。
- SQBool [sq_release](#) (HSQUIRRELVm v, HSQOBJECT *po)
Squirrel オブジェクトの参照カウントをデクリメントします。
- SQUnsignedInteger [sq_getvmrefcount](#) (HSQUIRRELVm v, const HSQOBJECT *po)
Squirrel VM 内部のオブジェクトの参照カウントを取得します。
- const SQChar * [sq_objtostring](#) (const HSQOBJECT *o)
オブジェクトが文字列型の場合、その *C* 文字列ポインタを取得します。
- SQInteger [sq_objtointeger](#) (const HSQOBJECT *o)
オブジェクトが数値型の場合、その整数値を取得します。
- SQFloat [sq_objtfloat](#) (const HSQOBJECT *o)
オブジェクトが数値型の場合、その浮動小数点数値を取得します。
- SQBool [sq_objtobool](#) (const HSQOBJECT *o)
オブジェクトがブール型の場合、そのブール値を取得します。
- SQUserPointer [sq_objtouserpointer](#) (const HSQOBJECT *o)
オブジェクトがユーザーポインタ型の場合、その値を取得します。
- void [sq_pushnull](#) (HSQUIRRELVm v)
スタックに *null* 値をプッシュします。
- void [sq_pushstring](#) (HSQUIRRELVm v, const SQChar *s, SQInteger len)

- スタックに文字列をプッシュします。

 - void [sq_pushinteger](#) (HSQUIRRELM v, SQInteger n)

スタックに整数値をプッシュします。
- void [sq_pushbool](#) (HSQUIRRELM v, SQBool b)

スタックにブール値をプッシュします。
- void [sq_pushfloat](#) (HSQUIRRELM v, SQFloat n)

スタックに浮動小数点数値をプッシュします。
- void [sq_pushuserpointer](#) (HSQUIRRELM v, SQUserPointer p)

スタックにユーザーポインタをプッシュします。
- void [sq_pushthread](#) (HSQUIRRELM v, HSQUIRRELM thread)

スタックにスレッドオブジェクトをプッシュします。
- SQUserPointer [sq_newuserdata](#) (HSQUIRRELM v, SQUnsignedInteger size)

新しいユーザーデータを作成し、スタックにプッシュします。
- void [sq_newtable](#) (HSQUIRRELM v)

新しい空のテーブルを作成し、スタックにプッシュします。
- void [sq_newtableex](#) (HSQUIRRELM v, SQInteger initialcapacity)

指定された初期容量で新しいテーブルを作成し、スタックにプッシュします。
- void [sq_newarray](#) (HSQUIRRELM v, SQInteger size)

指定されたサイズの新しい配列を作成し、スタックにプッシュします。
- SQRESULT [sq_newclass](#) (HSQUIRRELM v, SQBool hasbase)

新しいクラスを作成し、スタックにプッシュします。
- SQBool [sq_instanceof](#) (HSQUIRRELM v)

インスタンスが特定のクラスのインスタンス（またはその派生クラスのインスタンス）であるかを確認します。
- SQRESULT [sq_arrayappend](#) (HSQUIRRELM v, SQInteger idx)

配列の末尾に要素を追加します。
- SQRESULT [sq_arraypop](#) (HSQUIRRELM v, SQInteger idx, SQBool pushval)

配列の末尾の要素を削除（ポップ）します。
- SQRESULT [sq_arrayresize](#) (HSQUIRRELM v, SQInteger idx, SQInteger newsize)

配列のサイズを変更します。
- SQRESULT [sq_arrayreverse](#) (HSQUIRRELM v, SQInteger idx)

配列の要素の順序を反転させます。
- SQRESULT [sq_arrayremove](#) (HSQUIRRELM v, SQInteger idx, SQInteger itemidx)

配列から指定したインデックスの要素を削除します。
- SQRESULT [sq_arrayinsert](#) (HSQUIRRELM v, SQInteger idx, SQInteger destpos)

配列の指定した位置に要素を挿入します。
- void [sq_newclosure](#) (HSQUIRRELM v, SQFUNCTION func, SQUnsignedInteger nfreevars)

C/C++関数から新しいネイティブクロージャを作成し、スタックにプッシュします。
- SQRESULT [sq_getclosureinfo](#) (HSQUIRRELM v, SQInteger idx, SQInteger *nparams, SQInteger *nfreevars)

クロージャに関する情報（パラメータ数、自由変数の数）を取得します。
- SQRESULT [sq_setnativeclosurename](#) (HSQUIRRELM v, SQInteger idx, const SQChar *name)

ネイティブクロージャに名前を設定します。
- SQRESULT [sq_setparamscheck](#) (HSQUIRRELM v, SQInteger nparamscheck, const SQChar *typemask)

ネイティブクロージャのパラメータ数チェックと型チェックを設定します。
- SQRESULT [sq_bindenv](#) (HSQUIRRELM v, SQInteger idx)

クロージャに環境オブジェクト（*this* として参照されるオブジェクト）をバインドします。
- SQRESULT [sq_getclosurename](#) (HSQUIRRELM v, SQInteger idx)

クロージャの名前を取得し、スタックにプッシュします。
- SQRESULT [sq_setclosureroot](#) (HSQUIRRELM v, SQInteger idx)

スクリプトクロージャのルートテーブルを設定します。
- SQRESULT [sq_getclosureroot](#) (HSQUIRRELM v, SQInteger idx)

- スクリプトクロージャの現在のルートテーブルを取得し、スタックにプッシュします。
- SQRESULT [sq_clear](#) (HSQUIRRELV v, SQInteger idx)
テーブルまたは配列のすべての要素を削除します。
 - void [sq_pushroottable](#) (HSQUIRRELV v)
VMの現在のルートテーブルをスタックにプッシュします。
 - void [sq_pushregistrytable](#) (HSQUIRRELV v)
レジストリテーブルをスタックにプッシュします。
 - void [sq_pushconsttable](#) (HSQUIRRELV v)
定数テーブルをスタックにプッシュします。
 - SQRESULT [sq_setroottable](#) (HSQUIRRELV v)
VMの現在のルートテーブルを設定します。
 - SQRESULT [sq_setconsttable](#) (HSQUIRRELV v)
VMの定数テーブルを設定します。
 - void [sq_setforeignptr](#) (HSQUIRRELV v, SQUserPointer p)
VMに外部のポインタ (*foreign pointer*) を設定します。
 - SQUserPointer [sq_getforeignptr](#) (HSQUIRRELV v)
VMに設定されている外部ポインタを取得します。
 - void [sq_setsharedforeignptr](#) (HSQUIRRELV v, SQUserPointer p)
VMの共有状態に外部のポインタを設定します。
 - SQUserPointer [sq_getsharedforeignptr](#) (HSQUIRRELV v)
VMの共有状態に設定されている外部ポインタを取得します。
 - void [sq_setvmreleasehook](#) (HSQUIRRELV v, SQRELEASEHOOK hook)
VM固有のリリースフックを設定します。
 - SQRELEASEHOOK [sq_getvmreleasehook](#) (HSQUIRRELV v)
VM固有のリリースフックを取得します。
 - void [sq_setsharedreleasehook](#) (HSQUIRRELV v, SQRELEASEHOOK hook)
共有状態のリリースフックを設定します。
 - SQRELEASEHOOK [sq_getsharedreleasehook](#) (HSQUIRRELV v)
共有状態のリリースフックを取得します。
 - void [sq_push](#) (HSQUIRRELV v, SQInteger idx)
指定されたインデックスのスタック上のオブジェクトをスタックトップにコピー (プッシュ) します。
 - SQObjectType [sq_gettype](#) (HSQUIRRELV v, SQInteger idx)
指定されたインデックスのスタック上のオブジェクトの型を取得します。
 - SQRESULT [sq_typeof](#) (HSQUIRRELV v, SQInteger idx)
指定されたインデックスのオブジェクトの型名を取得し、文字列としてスタックにプッシュします。
 - SQRESULT [sq_tostring](#) (HSQUIRRELV v, SQInteger idx)
指定されたインデックスのオブジェクトを文字列に変換し、スタックにプッシュします。
 - void [sq_tobool](#) (HSQUIRRELV v, SQInteger idx, SQBool *b)
指定されたインデックスのオブジェクトのブール値としての真偽を判定します。
 - SQRESULT [sq_getinteger](#) (HSQUIRRELV v, SQInteger idx, SQInteger *i)
指定されたインデックスのオブジェクトを整数に変換して取得します。
 - SQRESULT [sq_getfloat](#) (HSQUIRRELV v, SQInteger idx, SQFloat *f)
指定されたインデックスのオブジェクトを浮動小数点数に変換して取得します。
 - SQRESULT [sq_getbool](#) (HSQUIRRELV v, SQInteger idx, SQBool *b)
指定されたインデックスのオブジェクトがブール値であるか確認し、その値を取得します。
 - SQRESULT [sq_getstringandsize](#) (HSQUIRRELV v, SQInteger idx, const SQChar **c, SQInteger *size)
指定されたインデックスの文字列オブジェクトから、C文字列ポインタと長さを取得します。
 - SQRESULT [sq_getstring](#) (HSQUIRRELV v, SQInteger idx, const SQChar **c)
指定されたインデックスの文字列オブジェクトから、C文字列ポインタを取得します。
 - SQRESULT [sq_getthread](#) (HSQUIRRELV v, SQInteger idx, HSQUIRRELV *thread)
指定されたインデックスのスレッドオブジェクトから、VMハンドルを取得します。

- SQRESULT [sq_clone](#) (HSQUIRRELV v, SQInteger idx)
指定されたインデックスのオブジェクトのクローン（ディープコピー）を作成し、スタックにプッシュします。
- SQInteger [sq_getsize](#) (HSQUIRRELV v, SQInteger idx)
指定されたインデックスのオブジェクトのサイズを取得します。
- SQHash [sq_gethash](#) (HSQUIRRELV v, SQInteger idx)
指定されたインデックスのオブジェクトのハッシュ値を取得します。
- SQRESULT [sq_getuserdata](#) (HSQUIRRELV v, SQInteger idx, SQUserPointer *p, SQUserPointer *typetag)
指定されたインデックスのユーザーデータオブジェクトから、データポインタと型タグを取得します。
- SQRESULT [sq_settypetag](#) (HSQUIRRELV v, SQInteger idx, SQUserPointer typetag)
ユーザーデータまたはクラスに型タグを設定します。
- SQRESULT [sq_getobjtypetag](#) (const HSQOBJECT *o, SQUserPointer *typetag)
HSQOBJECT から型タグを取得します。
- SQRESULT [sq_gettypetag](#) (HSQUIRRELV v, SQInteger idx, SQUserPointer *typetag)
指定されたインデックスのオブジェクトから型タグを取得します。
- SQRESULT [sq_getuserpointer](#) (HSQUIRRELV v, SQInteger idx, SQUserPointer *p)
指定されたインデックスのユーザーポインタオブジェクトから、ポインタ値を取得します。
- SQRESULT [sq_setinstanceup](#) (HSQUIRRELV v, SQInteger idx, SQUserPointer p)
クラスインスタンスにユーザーポインタを設定します。
- SQRESULT [sq_setclassudsize](#) (HSQUIRRELV v, SQInteger idx, SQInteger udsiz)
クラスに追加のユーザーデータ領域のサイズを設定します。
- SQRESULT [sq_getinstanceup](#) (HSQUIRRELV v, SQInteger idx, SQUserPointer *p, SQUserPointer typetag, SQBool throwerror)
クラスインスタンスからユーザーポインタ（またはユーザーデータ領域）を取得します。
- SQInteger [sq_gettop](#) (HSQUIRRELV v)
スタック上の要素数を取得します。
- void [sq_settop](#) (HSQUIRRELV v, SQInteger newtop)
スタックトップを指定した位置に設定します。
- void [sq_pop](#) (HSQUIRRELV v, SQInteger nelemstopop)
スタックトップから指定された数の要素をポップします。
- void [sq_poptop](#) (HSQUIRRELV v)
スタックトップの要素を 1 つポップします。
- void [sq_remove](#) (HSQUIRRELV v, SQInteger idx)
指定されたインデックスのスタック要素を削除します。
- SQInteger [sq_cmp](#) (HSQUIRRELV v)
スタックトップの 2 つのオブジェクトを比較します。
- SQRESULT [sq_newslot](#) (HSQUIRRELV v, SQInteger idx, SQBool bstatic)
テーブルまたはクラスに新しいスロット（メンバー）を作成します。
- SQRESULT [sq_deleteslot](#) (HSQUIRRELV v, SQInteger idx, SQBool pushval)
テーブルからスロットを削除します。
- SQRESULT [sq_set](#) (HSQUIRRELV v, SQInteger idx)
オブジェクトのスロットに値を設定します（メタメソッドを考慮します）。
- SQRESULT [sq_rawset](#) (HSQUIRRELV v, SQInteger idx)
オブジェクトのスロットに値を直接設定します（メタメソッドを無視します）。
- SQRESULT [sq_newmember](#) (HSQUIRRELV v, SQInteger idx, SQBool bstatic)
クラスに新しいメンバー（フィールドまたはメソッド）を属性付きで追加します。メタメソッドを考慮します。
- SQRESULT [sq_rawnewmember](#) (HSQUIRRELV v, SQInteger idx, SQBool bstatic)
クラスに新しいメンバーを直接追加します。メタメソッドを無視します。
- SQRESULT [sq_setdelegate](#) (HSQUIRRELV v, SQInteger idx)
テーブルまたはユーザーデータのデリゲートを設定します。
- SQRESULT [sq_rawdeleteslot](#) (HSQUIRRELV v, SQInteger idx, SQBool pushval)
テーブルからスロットを直接削除します（メタメソッドを無視します）。

- SQRESULT [sq_getdelegate](#) (HSQUIRRELV v, SQInteger idx)
テーブルまたはユーザーデータのデリゲートを取得し、スタックにプッシュします。
- SQRESULT [sq_get](#) (HSQUIRRELV v, SQInteger idx)
オブジェクトからキーに対応する値を取得します (メタメソッドを考慮します)。
- SQRESULT [sq_rawget](#) (HSQUIRRELV v, SQInteger idx)
オブジェクトからキーに対応する値を直接取得します (メタメソッドを無視します)。
- SQRESULT [sq_getstackobj](#) (HSQUIRRELV v, SQInteger idx, HSQOBJECT *po)
指定されたスタックインデックスのオブジェクトへのハンドル (*HSQOBJECT*) を取得します。
- const SQChar * [sq_getlocal](#) (HSQUIRRELV v, SQUnsignedInteger level, SQUnsignedInteger idx)
実行中の関数のローカル変数に関する情報を取得します。
- void [sq_pushobject](#) (HSQUIRRELV v, HSQOBJECT obj)
HSQOBJECT ハンドルが指すオブジェクトをスタックにプッシュします。
- void [sq_resetobject](#) (HSQOBJECT *po)
HSQOBJECT ハンドルをリセットして *null* 状態にします。
- SQRESULT [sq_throwerror](#) (HSQUIRRELV v, const SQChar *err)
VM に文字列のエラーを設定し、エラー状態にします。
- SQRESULT [sq_throwobject](#) (HSQUIRRELV v)
VM にオブジェクトのエラーを設定し、エラー状態にします。
- void [sq_reseterror](#) (HSQUIRRELV v)
VM の最後のエラー状態をリセットします。
- void [sq_getlasterror](#) (HSQUIRRELV v)
VM に最後に設定されたエラーオブジェクトを取得し、スタックにプッシュします。
- SQRESULT [sq_reservestack](#) (HSQUIRRELV v, SQInteger nsize)
VM のスタックサイズが少なくとも指定されたサイズを確保するようにします。
- SQRESULT [sq_resume](#) (HSQUIRRELV v, SQBool retval, SQBool raiseerror)
ジェネレータの実行を再開します。
- SQRESULT [sq_call](#) (HSQUIRRELV v, SQInteger params, SQBool retval, SQBool raiseerror)
スタック上のクロージャを呼び出します。
- SQRESULT [sq_tailcall](#) (HSQUIRRELV v, SQInteger nparams)
C 関数から *Squirrel* の関数を末尾呼び出しします。
- SQRESULT [sq_suspendvm](#) (HSQUIRRELV v)
VM の実行を中断します。
- SQRESULT [sq_wakeupvm](#) (HSQUIRRELV v, SQBool wakeupret, SQBool retval, SQBool raiseerror, SQBool throwerror)
中断された VM の実行を再開します。
- void [sq_setreleasehook](#) (HSQUIRRELV v, SQInteger idx, SQRELEASEHOOK hook)
ユーザーデータ、インスタンス、またはクラスにリリースフックを設定します。
- SQRELEASEHOOK [sq_getreleasehook](#) (HSQUIRRELV v, SQInteger idx)
ユーザーデータ、インスタンス、またはクラスからリリースフックを取得します。
- void [sq_setcompilererrorhandler](#) (HSQUIRRELV v, SQCOMPILERERROR f)
コンパイラのエラーハンドラを設定します。
- SQRESULT [sq_writeclosure](#) (HSQUIRRELV v, SQWRITEFUNC w, SQUserPointer up)
スクリプトクロージャをバイトコードとしてシリアライズし、指定されたライター関数に出力します。
- SQRESULT [sq_readclosure](#) (HSQUIRRELV v, SQREADFUNC r, SQUserPointer up)
シリアライズされたバイトコードを読み込み、クロージャを復元してスタックにプッシュします。
- SQChar * [sq_getscratchpad](#) (HSQUIRRELV v, SQInteger minsize)
VM の共有状態から一時的なメモリ領域 (スクラッチパッド) を取得します。
- SQRESULT [sq_resurrectunreachable](#) (HSQUIRRELV v)
ガベージコレクタによって到達不可能とマークされたオブジェクトを復活させます。
- SQInteger [sq_collectgarbage](#) (HSQUIRRELV v)
インクリメンタル・ガベージコレクションのサイクルを 1 回実行します。

- SQRESULT [sq_getcallee](#) (HSQUIRRELVm v)
現在の関数の呼び出し元 (*callee*) のクロージャを取得し、スタックにプッシュします。
- const SQChar * [sq_getfreevariable](#) (HSQUIRRELVm v, SQInteger idx, SQUnsignedInteger nval)
指定されたインデックスのクロージャから自由変数を取得します。
- SQRESULT [sq_setfreevariable](#) (HSQUIRRELVm v, SQInteger idx, SQUnsignedInteger nval)
指定されたインデックスのクロージャの自由変数を設定します。
- SQRESULT [sq_setattributes](#) (HSQUIRRELVm v, SQInteger idx)
クラスまたはそのメンバーの属性を設定します。
- SQRESULT [sq_getattributes](#) (HSQUIRRELVm v, SQInteger idx)
クラスまたはそのメンバーの属性を取得します。
- SQRESULT [sq_getmemberhandle](#) (HSQUIRRELVm v, SQInteger idx, HSQMEMBERHANDLE *handle)
クラスメンバーへの高速なアクセスを可能にするハンドルを取得します。
- SQRESULT [_getmemberbyhandle](#) (HSQUIRRELVm v, SQObjectPtr &self, const HSQMEMBERHANDLE *handle, SQObjectPtr *&val)
メンバーハンドルを使用して、クラスまたはインスタンスのメンバー値へのポインタを取得します。(内部使用向け)
- SQRESULT [sq_getbyhandle](#) (HSQUIRRELVm v, SQInteger idx, const HSQMEMBERHANDLE *handle)
メンバーハンドルを使用して、クラスまたはインスタンスのメンバーの値を取得し、スタックにプッシュします。
- SQRESULT [sq_setbyhandle](#) (HSQUIRRELVm v, SQInteger idx, const HSQMEMBERHANDLE *handle)
メンバーハンドルを使用して、クラスまたはインスタンスのメンバーに値を設定します。
- SQRESULT [sq_getbase](#) (HSQUIRRELVm v, SQInteger idx)
クラスの基底クラスを取得し、スタックにプッシュします。
- SQRESULT [sq_getclass](#) (HSQUIRRELVm v, SQInteger idx)
インスタンスからそのクラスオブジェクトを取得し、スタックにプッシュします。
- SQRESULT [sq_createinstance](#) (HSQUIRRELVm v, SQInteger idx)
指定されたインデックスのクラスから新しいインスタンスを生成し、スタックにプッシュします。
- void [sq_weakref](#) (HSQUIRRELVm v, SQInteger idx)
指定されたインデックスのオブジェクトへの弱い参照 (*weak reference*) を作成し、スタックにプッシュします。
- SQRESULT [sq_getweakrefval](#) (HSQUIRRELVm v, SQInteger idx)
弱い参照から元のオブジェクトを取得し、スタックにプッシュします。
- SQRESULT [sq_getdefaultdelegate](#) (HSQUIRRELVm v, SQObjectType t)
指定されたオブジェクト型のデフォルトデリゲートを取得し、スタックにプッシュします。
- SQRESULT [sq_next](#) (HSQUIRRELVm v, SQInteger idx)
コンテナ (テーブル、配列など) のイテレーションを 1 ステップ進めます。
- SQInteger [buf_lexfeed](#) (SQUserPointer file)
sq_compilebuffer 用の内部レキサーフィード関数。
- SQRESULT [sq_compilebuffer](#) (HSQUIRRELVm v, const SQChar *s, SQInteger size, const SQChar *source-name, SQBool raiseerror)
メモリバッファ内の *Squirrel* ソースコードをコンパイルします。
- void [sq_move](#) (HSQUIRRELVm dest, HSQUIRRELVm src, SQInteger idx)
ある VM のスタックから別の VM のスタックへオブジェクトを移動 (コピー) します。
- void [sq_setprintfunc](#) (HSQUIRRELVm v, SQPRINTFUNCTION printfunc, SQPRINTFUNCTION errfunc)
VM の標準出力および標準エラー出力用のプリント関数を設定します。
- SQPRINTFUNCTION [sq_getprintfunc](#) (HSQUIRRELVm v)
現在設定されている標準出力用のプリント関数を取得します。
- SQPRINTFUNCTION [sq_geterrorfunc](#) (HSQUIRRELVm v)
現在設定されている標準エラー出力用のプリント関数を取得します。
- void * [sq_malloc](#) (SQUnsignedInteger size)
Squirrel のカスタムアロケータを使用してメモリを確保します。

- void * [sq_realloc](#) (void *p, SQUnsignedInteger oldsize, SQUnsignedInteger newsize)
Squirrel のカスタムアロケータを使用してメモリを再確保します。
- void [sq_free](#) (void *p, SQUnsignedInteger size)
Squirrel のカスタムアロケータを使用してメモリを解放します。

4.1.1 Macro Definition Documentation

4.1.1.1 _GETSAFE_OBJ

```
#define _GETSAFE_OBJ(
    v,
    idx,
    type,
    o ) { if(!sq_aux_gettypedarg(v,idx,type,&o)) return SQ_ERROR; }
```

型をチェックし、安全にオブジェクトポインタを取得するための内部マクロ。

このマクロは、指定されたスタックインデックス `idx` のオブジェクトが期待される `type` であることを検証し、そのポインタを `o` に格納します。型が一致しない場合、`sq_aux_gettypedarg` は内部で VM エラーを設定し、このマクロは呼び出し元の関数から `SQ_ERROR` を返して即座にリターンします。これにより、多くの API 関数における引数の型チェックとエラー処理の記述を一行に集約できます。

Definition at line 46 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

4.1.1.2 sq_aux_paramscheck

```
#define sq_aux_paramscheck(
    v,
    count )
```

Value:

```
{ \
    if(sq_gettop(v) < count){ v->Raise_Error(_SC("not enough params in the stack")); return SQ_ERROR; }\
}
```

API 関数呼び出し時にスタック上に十分なパラメータが存在するかをチェックする内部マクロ。

このマクロは、現在のスタックの要素数 (`sq_gettop(v)`) が、要求される数 `count` よりも少ないかどうかを検証します。もし要素数が不足している場合、「not enough params in the stack」というエラーを VM に設定し、呼び出し元の関数から `SQ_ERROR` を返します。これにより、関数の冒頭で引数の数を安全に検証できます。

Definition at line 54 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00055 { \
00056     if(sq_gettop(v) < count){ v->Raise_Error(_SC("not enough params in the stack")); return SQ_ERROR;
    }\
00057 }
```

4.1.2 Function Documentation

4.1.2.1 _getmemberbyhandle()

```
SQRESULT _getmemberbyhandle (
    HSQUIRRELVM v,
    SQObjectPtr & self,
    const HSQMEMBERHANDLE * handle,
    SQObjectPtr *& val )
```

メンバーハンドルを使用して、クラスまたはインスタンスのメンバー値へのポインタを取得します。(内部使用向け)

Parameters

<i>v</i>	対象のSquirrel VM。
<i>self</i>	クラスまたはインスタンスオブジェクト。
<i>handle</i>	sq_getmemberhandle で取得したハンドル。
<i>val</i>	結果の値へのポインタを格納するポインタ。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

sq_getbyhandle および sq_setbyhandle から呼び出される内部ヘルパー関数です。ハンドル情報に基づき、クラスやインスタンスの内部配列から直接メンバーのメモリアドレスを取得します。

Definition at line 2406 of file squirrel3_squirrel_sqapiByGemini.cpp.

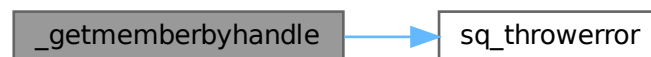
```

02407 {
02408     switch(sq_type(self)) {
02409         case OT_INSTANCE: {
02410             SQInstance *i = _instance(self);
02411             if(handle->_static) {
02412                 SQClass *c = i->_class;
02413                 val = &c->_methods[handle->_index].val;
02414             }
02415             else {
02416                 val = &i->_values[handle->_index];
02417             }
02418         }
02419         break;
02420     case OT_CLASS: {
02421         SQClass *c = _class(self);
02422         if(handle->_static) {
02423             val = &c->_methods[handle->_index].val;
02424         }
02425         else {
02426             val = &c->_defaultvalues[handle->_index].val;
02427         }
02428     }
02429     break;
02430     default:
02431         return sq_throwerror(v, _SC("wrong type(expected class or instance)"));
02432     }
02433     return SQ_OK;
02434 }
02435 }
```

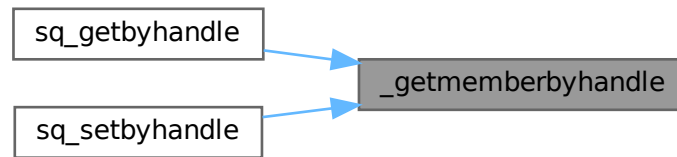
References [sq_throwerror\(\)](#).

Referenced by [sq_getbyhandle\(\)](#), and [sq_setbyhandle\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.2 buf_lexfeed()

```
SQInteger buf_lexfeed (
    SQUserPointer file )
```

sq_compilebuffer 用の内部レキサーフィード関数。

Parameters

<i>file</i>	BufState 構造体へのユーザーポインタ。
-------------	-------------------------

Returns

バッファから読み込んだ 1 文字。バッファの終端に達した場合は 0。

sq_compile に渡される SQLEXREADFUNC の実装です。file ポインタを [BufState](#) にキャストし、現在のポインタ位置の文字を返してポインタをインクリメントします。バッファの終端に達したら EOF として 0 を返します。

Definition at line 2626 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02627 {
02628     BufState *buf=(BufState*) file;
02629     if (buf->size<(buf->ptr+1))
02630         return 0;
02631     return buf->buf[buf->ptr++];
02632 }
```

References [BufState::buf](#), [BufState::ptr](#), and [BufState::size](#).

Referenced by [sq_compilebuffer\(\)](#).

Here is the caller graph for this function:



4.1.2.3 sq_addrf()

```
void sq_addrf (
    HSQUIRRELVm v,
    HSQOBJECT * po )
```

Squirrel オブジェクトの参照カウントをインクリメントします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>po</i>	参照カウントを増やすオブジェクトへのポインタ。

参照カウントを持つオブジェクト（テーブル、配列、文字列、クロージャなど）に対してのみ有効です。C 側でSquirrel オブジェクトへの参照をスタック外で保持する場合、この関数で参照カウントを増やし、不要になったら `sq_release` で減らす必要があります。これにより、オブジェクトが意図せずガベージコレクションされるのを防ぎます。

Definition at line 285 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00286 {
00287     if (!ISREFCOUNTED(sq_type(*po))) return;
00288     #ifdef NO_GARBAGE_COLLECTOR
00289         __AddRef(po->_type, po->_unVal);
00290     #else
00291         __ss(v)->_refs_table.AddRef(*po);
00292     #endif
00293 }
```

4.1.2.4 sq_arrayappend()

```
SQRESULT sq_arrayappend (
    HSQUIRRELVm v,
    SQInteger idx )
```

配列の末尾に要素を追加します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	配列が格納されているスタックインデックス。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

指定された `idx` にある配列の末尾に、スタックトップの要素を追加します。この関数はSquirrel の `array.append()` メソッドに相当します。追加操作の後、スタックトップの要素はポップされます。

Definition at line 602 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00603 {
00604     sq_aux_paramscheck(v, 2);
00605     SQObjectPtr *arr;
00606     _GETSAFE_OBJ(v, idx, OT_ARRAY, arr);
00607     _array(*arr)->Append(v->GetUp(-1));
00608     v->Pop();
00609     return SQ_OK;
00610 }
```

References [_GETSAFE_OBJ](#), and [sq_aux_paramscheck](#).

4.1.2.5 sq_arrayinsert()

```
SQRESULT sq_arrayinsert (
    HSQUIRRELVM v,
    SQInteger idx,
    SQInteger destpos )
```

配列の指定した位置に要素を挿入します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	配列が格納されているスタックインデックス。
<i>destpos</i>	挿入先のインデックス。

Returns

成功した場合はSQ_OK、インデックスが範囲外の場合はSQ_ERROR。

指定された *idx* にある配列の *destpos* で指定された位置に、スタックトップの要素を挿入します。この関数はSquirrelの `array.insert()` メソッドに相当します。挿入後、スタックトップの要素はポップされます。

Definition at line 705 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00706 {
00707     sq_aux_paramscheck(v, 1);
00708     SQObjectPtr *arr;
00709     _GETSAFE_OBJ(v, idx, OT_ARRAY, arr);
00710     SQRESULT ret = _array(*arr)->Insert(destpos, v->GetUp(-1)) ? SQ_OK : sq_throwerror(v, _SC("index
    out of range"));
00711     v->Pop();
00712     return ret;
00713 }
```

References [_GETSAFE_OBJ](#), [sq_aux_paramscheck](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.6 sq_arraypop()

```
SQRESULT sq_arraypop (
    HSQUIRRELVM v,
    SQInteger idx,
    SQBool pushval )
```

配列の末尾の要素を削除（ポップ）します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	配列が格納されているスタックインデックス。
<i>pushval</i>	true の場合、削除された要素をスタックにプッシュします。

Returns

成功した場合はSQ_OK、配列が空の場合はSQ_ERROR。

指定された *idx* にある配列の末尾の要素を削除します。この関数はSquirrel の `array.pop()` メソッドに相当します。`pushval` を true に設定すると、削除された値がスタックに残るため、値を取得して利用することができます。配列が空のときに呼び出すとエラーになります。

Definition at line 620 of file `squirrel3_squirrel_sqapiByGemini.cpp`.

```

00621 {
00622     sq_aux_paramscheck(v, 1);
00623     SQObjectPtr *arr;
00624     _GETSAFE_OBJ(v, idx, OT_ARRAY, arr);
00625     if(_array(*arr)->Size() > 0) {
00626         if(pushval != 0){ v->Push(_array(*arr)->Top()); }
00627         _array(*arr)->Pop();
00628         return SQ_OK;
00629     }
00630     return sq_throwerror(v, _SC("empty array"));
00631 }
```

References `_GETSAFE_OBJ`, `sq_aux_paramscheck`, and `sq_throwerror()`.

Here is the call graph for this function:



4.1.2.7 sq_arrayremove()

```

SQRESULT sq_arrayremove (
    HSQUIRRELVm v,
    SQInteger idx,
    SQInteger itemidx )
```

配列から指定したインデックスの要素を削除します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	配列が格納されているスタックインデックス。
<i>itemidx</i>	削除する要素のインデックス。

Returns

成功した場合はSQ_OK、インデックスが範囲外の場合はSQ_ERROR。

指定された `idx` にある配列から、`itemidx` で指定されたインデックスの要素を削除します。この関数は Squirrel の `array.remove()` メソッドに相当します。指定された要素が削除され、それ以降の要素は前方にシフトされます。削除された値は返されず、スタックの状態も変化しません。

Definition at line 689 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00690 {
00691     sq_aux_paramscheck(v, 1);
00692     SQObjectPtr *arr;
00693     _GETSAFE_OBJ(v, idx, OT_ARRAY, arr);
00694     return _array(*arr)->Remove(itemidx) ? SQ_OK : sq_throwerror(v, _SC("index out of range"));
00695 }
```

References [_GETSAFE_OBJ](#), [sq_aux_paramscheck](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.8 sq_arrayresize()

```
SQRESULT sq_arrayresize (
    HSQUIRRELVm v,
    SQInteger idx,
    SQInteger newsize )
```

配列のサイズを変更します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	配列が格納されているスタックインデックス。
<i>newsiz</i>	新しい配列のサイズ。

Returns

成功した場合はSQ_OK、`newsiz` が負の場合はSQ_ERROR。

指定された `idx` にある配列のサイズを `newsiz` に変更します。この関数は Squirrel の `array.resize()` メソッドに相当します。サイズを大きくした場合、新しく追加された要素は `null` で初期化されます。サイズを小さくした場合、末尾の要素が破棄されます。

Definition at line 641 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

00642 {
00643     sq_aux_paramscheck(v,1);
00644     SQObjectPtr *arr;
00645     _GETSAFE_OBJ(v, idx, OT_ARRAY,arr);
00646     if(newsize >= 0) {
00647         _array(*arr)->Resize(newsize);
00648         return SQ_OK;
00649     }
00650     return sq_throwerror(v,_SC("negative size"));
00651 }

```

References [_GETSAFE_OBJ](#), [sq_aux_paramscheck](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.9 sq_arrayreverse()

```

SQRESULT sq_arrayreverse (
    HSQUIRRELVm v,
    SQInteger idx )

```

配列の要素の順序を反転させます。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	配列が格納されているスタックインデックス。

Returns

常にSQ_OK。

指定された *idx* にある配列の要素の順序をインプレース（元の配列内で）で反転させます。この関数は Squirrel の `array.reverse()` メソッドに相当します。スタックの状態は変化しません。

Definition at line 661 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

00662 {
00663     sq_aux_paramscheck(v, 1);
00664     SQObjectPtr *o;
00665     _GETSAFE_OBJ(v, idx, OT_ARRAY,o);
00666     SQArray *arr = _array(*o);
00667     if(arr->Size() > 0) {
00668         SQObjectPtr t;
00669         SQInteger size = arr->Size();
00670         SQInteger n = size » 1; size -= 1;
00671         for(SQInteger i = 0; i < n; i++) {
00672             t = arr->_values[i];
00673             arr->_values[i] = arr->_values[size-i];
00674             arr->_values[size-i] = t;
00675         }

```

```

00676         return SQ_OK;
00677     }
00678     return SQ_OK;
00679 }

```

References [_GETSAFE_OBJ](#), and [sq_aux_paramscheck](#).

4.1.2.10 sq_aux_gettypedarg()

```

static bool sq_aux_gettypedarg (
    HSQUIRRELVM v,
    SQInteger idx,
    SQObjectType type,
    SQObjectPtr ** o ) [static]

```

指定されたスタックインデックスの引数が期待される型であるかを確認し、そのオブジェクトポインタを取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	スタックインデックス。
<i>type</i>	期待されるオブジェクトの型。
<i>o</i>	オブジェクトポインタを格納するポインタ。

Returns

型が一致すれば true、そうでなければエラーを発生させて false を返します。

この関数は内部ヘルパーであり、API 関数の引数型チェックを簡素化するために使用されます。型が一致しない場合、VM に「wrong argument type」という詳細な型情報を含むエラーを報告し、false を返します。成功した場合は、引数 *o* にオブジェクトへのポインタを設定します。

Definition at line 29 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

00030 {
00031     *o = &stack_get(v,idx);
00032     if(sq_type(**o) != type){
00033         SQObjectPtr oval = v->PrintObjVal(**o);
00034         v->Raise_Error(_SC("wrong argument type, expected '%s' got
'%s.50s'"),IdType2Name(type),_stringval(oval));
00035         return false;
00036     }
00037     return true;
00038 }

```

4.1.2.11 sq_aux_invalidtype()

```

SQInteger sq_aux_invalidtype (
    HSQUIRRELVM v,
    SQObjectType type )

```

予期しない型が指定された場合にエラーをスローします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>type</i>	予期しなかったオブジェクトの型。

Returns

エラーコード (SQ_ERROR)。

この関数は、特定の操作でサポートされていない型のオブジェクトが渡されたときに、「unexpected type [型名]」という形式の適切なエラーメッセージを生成してVMに報告するために使用されます。エラーメッセージの生成にはVMのスクラッチパッドを利用します。

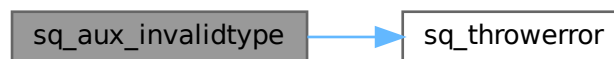
Definition at line 68 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00069 {
00070     SQUnsignedInteger buf_size = 100 * sizeof(SQChar);
00071     scsprintf(_ss(v)->GetScratchPad(buf_size), buf_size, _SC("unexpected type %s"),
              IdType2Name(type));
00072     return sq_throwerror(v, _ss(v)->GetScratchPad(-1));
00073 }
```

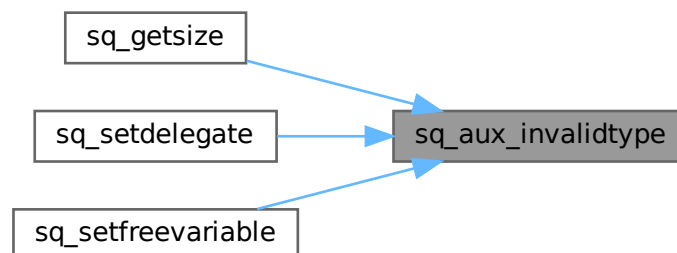
References [sq_throwerror\(\)](#).

Referenced by [sq_getsize\(\)](#), [sq_setdelegate\(\)](#), and [sq_setfreevariable\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:

**4.1.2.12 sq_bindenv()**

```
SQRESULT sq_bindenv (
    HSQUIRRELVm v,
    SQInteger idx )
```

クロージャに環境オブジェクト (this として参照されるオブジェクト) をバインドします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	クロージャが格納されているスタックインデックス。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

指定された *idx* にあるクロージャのクローンを作成し、それにスタックトップのオブジェクトを環境(*this*)としてバインドします。そして、新しく生成されたクロージャをスタックのトップにプッシュします。スタックトップにあった元の環境オブジェクトはポップされます。この操作の結果、スタックの高さは変わりません。元の *idx* にあったクロージャは変更されずに残ります。

Definition at line 818 of file `squirrel3_squirrel_sqapiByGemini.cpp`.

```

00819 {
00820     SQObjectPtr &o = stack_get(v, idx);
00821     if(!sq_isnativeclosure(o) &&
00822         !sq_isclosure(o))
00823         return sq_throwerror(v, _SC("the target is not a closure"));
00824     SQObjectPtr &env = stack_get(v, -1);
00825     if(!sq_istable(env) &&
00826         !sq_isarray(env) &&
00827         !sq_isclass(env) &&
00828         !sq_isinstance(env))
00829         return sq_throwerror(v, _SC("invalid environment"));
00830     SQWeakRef *w = _refcounted(env)->GetWeakRef(sq_type(env));
00831     SQObjectPtr ret;
00832     if(sq_isclosure(o)) {
00833         SQClosure *c = _closure(o)->Clone();
00834         __ObjRelease(c->_env);
00835         c->_env = w;
00836         __ObjAddRef(c->_env);
00837         if(_closure(o)->_base) {
00838             c->_base = _closure(o)->_base;
00839             __ObjAddRef(c->_base);
00840         }
00841         ret = c;
00842     }
00843     else { //then must be a native closure
00844         SQNativeClosure *c = _nativeclosure(o)->Clone();
00845         __ObjRelease(c->_env);
00846         c->_env = w;
00847         __ObjAddRef(c->_env);
00848         ret = c;
00849     }
00850     v->Pop();
00851     v->Push(ret);
00852     return SQ_OK;
00853 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.13 sq_call()

```
SQRESULT sq_call (
    HSQUIRRELVm v,
    SQInteger params,
    SQBool retval,
    SQBool raiseerror )
```

スタック上のクロージャを呼び出します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>params</i>	呼び出しに渡すパラメータの数。
<i>retval</i>	true の場合、関数からの戻り値をスタックにプッシュします。
<i>raiseerror</i>	実行時エラーが発生した場合にVM にエラーを発生させるかどうか。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

呼び出す前に、スタックにクロージャ、this オブジェクト、そして params 個の引数を順にプッシュする必要があります (スタックのトップから argN, ..., arg1, this, closure)。retval が true の場合、関数からの戻り値がスタックにプッシュされます (呼び出しに使われた引数やクロージャがポップされた後に)。raiseerror が true の場合、実行時エラーはVM のエラーとしてスローされます。

Definition at line 2013 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02014 {
02015     SQObjectPtr res;
02016     if (!v->Call(v->GetUp(-(params+1)), params, v->_top-params, res, raiseerror?true:false)) {
02017         v->Pop(params); //pop args
02018         return SQ_ERROR;
02019     }
02020     if (!v->_suspended)
02021         v->Pop(params); //pop args
02022     if (retval)
02023         v->Push(res); // push result
02024     return SQ_OK;
02025 }
```

4.1.2.14 sq_clear()

```
SQRESULT sq_clear (
    HSQUIRRELVm v,
    SQInteger idx )
```

テーブルまたは配列のすべての要素を削除します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	テーブルまたは配列が格納されているスタックインデックス。

Returns

成功した場合はSQ_OK、対象がテーブルまたは配列でない場合はSQ_ERROR。

テーブルの場合はすべてのキーと値のペアが削除されます。配列の場合はサイズが0にリサイズされます。コンテナ自体のオブジェクトは削除されず、空の状態になります。

Definition at line 920 of file squirrel3_squirrel_sqapiByGemini.cpp.

```
00921 {
00922     SQObject &o=stack_get(v,idx);
00923     switch(sq_type(o)) {
00924         case OT_TABLE: _table(o)->Clear(); break;
00925         case OT_ARRAY: _array(o)->Resize(0); break;
00926         default:
00927             return sq_throwerror(v, _SC("clear only works on table and array"));
00928         break;
00929     }
00930 }
00931 return SQ_OK;
00932 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.15 sq_clone()

```
SQRESULT sq_clone (
    HSQUIRRELM v,
    SQInteger idx )
```

指定されたインデックスのオブジェクトのクローン（ディープコピー）を作成し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	クローン元のオブジェクトのスタックインデックス。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

テーブル、配列、クラスインスタンスなどのコンテナオブジェクトがクローン可能です。クローンされたオブジェクトは、元のオブジェクトとは独立したコピーになります。テーブルや配列をクローンした場合、その要素も再帰的にクローンされます。クローン不可能な型のオブジェクトに対して呼び出すとエラーになります。元のオブジェクトはスタック上に残ります。

Definition at line 1274 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01275 {
01276     SQObjectPtr &o = stack_get(v, idx);
01277     v->PushNull();
01278     if(!v->Clone(o, stack_get(v, -1))){
01279         v->Pop();
01280         return SQ_ERROR;
01281     }
01282     return SQ_OK;
01283 }
```

4.1.2.16 sq_close()

```
void sq_close (
    HSQUIRRELVm v )
```

VM を閉じ、関連するすべてのリソースを解放します。

Parameters

<code>v</code>	閉じる対象のSquirrel VM。
----------------	--------------------

この関数は、`sq_open` で作成されたルートVM に対してのみ呼び出すべきです。関連するすべてのスレッドと共有状態 (SQSharedState) が破棄され、確保されていた全メモリが解放されます。プログラムの終了時に必ず呼び出す必要があります。

Definition at line 207 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00208 {
00209     SQSharedState *ss = _ss(v);
00210     _thread(ss->_root_vm)->Finalize();
00211     sq_delete(ss, SQSharedState);
00212 }
```

4.1.2.17 sq_cmp()

```
SQInteger sq_cmp (
    HSQUIRRELVm v )
```

スタックトップの2つのオブジェクトを比較します。

Parameters

<code>v</code>	対象のSquirrel VM。
----------------	-----------------

Returns

比較結果。obj1 > obj2 なら 1、obj1 < obj2 なら -1、obj1 == obj2 なら 0。

スタックトップの2つのオブジェクトを比較します。スタックトップ (-1) が右辺 (obj2)、その次 (-2) が左辺 (obj1) として比較されます。この関数は `_cmp` メタメソッドを呼び出す可能性があります。比較後、2つのオブジェクトはスタックから ** ポップされません **。

Definition at line 1541 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01542 {
01543     SQInteger res;
01544     v->ObjCmp(stack_get(v, -1), stack_get(v, -2), res);
01545     return res;
01546 }
```

4.1.2.18 sq_collectgarbage()

```
SQInteger sq_collectgarbage (
    HSQUIRRELVM v )
```

インクリメンタル・ガベージコレクションのサイクルを1回実行します。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

Returns

回収されたオブジェクトの数。GC ビルドでない場合は-1。

ガベージコレクションのサイクルを1回実行し、到達不可能なオブジェクトを収集します。Squirrel のGC はインクリメンタルであるため、この関数を定期的に呼び出すことで、メモリを段階的に解放できます。戻り値は、そのサイクルで収集されたオブジェクトの数です。GC が無効なビルドでは-1 を返します。

Definition at line 2221 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02222 {
02223 #ifndef NO_GARBAGE_COLLECTOR
02224     return _ss(v)->CollectGarbage(v);
02225 #else
02226     return -1;
02227 #endif
02228 }
```

4.1.2.19 sq_compile()

```
SQRESULT sq_compile (
    HSQUIRRELVM v,
    SQLEXREADFUNC read,
    SQUserPointer p,
    const SQChar * sourcename,
    SQBool raiseerror )
```

Squirrel ソースコードを読み込み、コンパイルして実行可能なクロージャを生成します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>read</i>	ソースコードを読み込むためのコールバック関数。
<i>p</i>	<i>read</i> 関数に渡されるユーザーポインタ。
<i>sourcename</i>	ソースコードの名称（デバッグ情報に使用）。
<i>raiseerror</i>	コンパイルエラーが発生した場合にVM にエラーを発生させるかどうか。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

コンパイルが成功すると、結果として得られるルートクロージャがスタックにプッシュされます。*raiseerror* がtrue の場合、コンパイルエラーはVM のエラーとしてスローされます。*false* の場合、この関数が SQ_ERROR

を返すのみです。NO_COMPILER が定義されているビルドでは、この関数は常にエラーを返し、実行時コンパイルはできません。

Definition at line 237 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00238 {
00239     SQObjectPtr o;
00240     #ifndef NO_COMPILER
00241         if(Compile(v, read, p, sourcename, o, raiseerror?true:false, _ss(v)->_debuginfo)) {
00242             v->Push(SQClosure::Create(_ss(v), _funcproto(o),
00243                                     _table(v->_roottable)->GetWeakRef(OT_TABLE)));
00243             return SQ_OK;
00244         }
00245         return SQ_ERROR;
00246     #else
00247         return sq_throwerror(v,_SC("this is a no compiler build"));
00248     #endif
00249 }
```

References [sq_throwerror\(\)](#).

Referenced by [sq_compilebuffer\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.20 sq_compilebuffer()

```
SQRESULT sq_compilebuffer (
    HSQUIRRELVm v,
    const SQChar * s,
    SQInteger size,
    const SQChar * sourcename,
    SQBool raiseerror )
```

メモリバッファ内のSquirrel ソースコードをコンパイルします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>s</i>	ソースコードが含まれるバッファへのポインタ。
<i>size</i>	バッファのサイズ (バイト単位)。
<i>sourcename</i>	ソースコードの名称 (デバッグ情報に使用)。
<i>raiseerror</i>	コンパイルエラーが発生した場合にVM にエラーを発生させるかどうか。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

sq_compile のラッパーであり、ファイルI/O の代わりにメモリバッファから直接読み込みます。内部では [BufState](#) 構造体と [buf_lexfeed](#) 関数を使い、メモリバッファを [sq_compile](#) が要求する読み込みストリームとして適合させます。

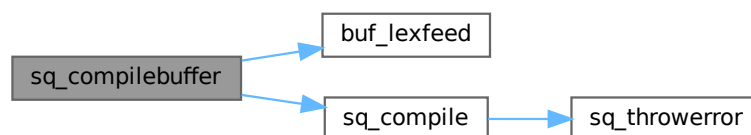
Definition at line 2644 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

02644 {
02645     BufState buf;
02646     buf.buf = s;
02647     buf.size = size;
02648     buf.ptr = 0;
02649     return sq_compile(v, buf_lexfeed, &buf, sourcename, raiseerror);
02650 }
```

References [BufState::buf](#), [buf_lexfeed\(\)](#), [BufState::ptr](#), [BufState::size](#), and [sq_compile\(\)](#).

Here is the call graph for this function:



4.1.2.21 sq_createinstance()

```

SQRESULT sq_createinstance (
    HSQUIRRELVM v,
    SQInteger idx )
```

指定されたインデックスのクラスから新しいインスタンスを生成し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	クラスのスタックインデックス。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

この関数はインスタンスを生成するだけで、コンストラクタは呼び出しません。コンストラクタを呼び出すには `sq_call` を使用する必要があります。

Definition at line 2517 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02518 {
02519     SQObjectPtr *o = NULL;
02520     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
02521     v->Push(_class(*o)->CreateInstance());
02522     return SQ_OK;
02523 }
```

References [_GETSAFE_OBJ](#).

4.1.2.22 sq_deleteslot()

```
SQRESULT sq_deleteslot (
    HSQUIRRELVM v,
    SQInteger idx,
    SQBool pushval )
```

テーブルからスロットを削除します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	テーブルのスタックインデックス。
<i>pushval</i>	true の場合、削除されたスロットの値をスタックにプッシュします。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

この関数を呼び出す前に、削除するスロットのキーをスタックトップにプッシュしておく必要があります。この操作は `_deleteslot` メタメソッドをトリガーする可能性があります。pushval が true の場合、削除された値がスタックにプッシュされます（キーがあった場所に）。キーが見つからなかった場合、エラーが返されます。

Definition at line 1577 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01578 {
01579     sq_aux_paramscheck(v, 2);
01580     SQObjectPtr *self;
01581     _GETSAFE_OBJ(v, idx, OT_TABLE,self);
01582     SQObjectPtr &key = v->GetUp(-1);
01583     if(sq_type(key) == OT_NULL) return sq_throwerror(v, _SC("null is not a valid key"));
01584     SQObjectPtr res;
01585     if(!v->DeleteSlot(*self, key, res)){
01586         v->Pop();
01587         return SQ_ERROR;
01588     }
01589     if(pushval) v->GetUp(-1) = res;
01590     else v->Pop();
01591     return SQ_OK;
01592 }
```

References [_GETSAFE_OBJ](#), [sq_aux_paramscheck](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.23 sq_enableddebuginfo()

```
void sq_enableddebuginfo (
    HSQUIRRELVM v,
    SQBool enable )
```

デバッグ情報の生成を有効または無効にします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>enable</i>	true で有効、false で無効。

デバッグ情報を有効にすると、コンパイラは行番号やローカル変数名などの情報をバイトコードに埋め込みます。これにより、デバッグが容易になりますが、バイトコードのサイズは増加します。この設定は共有状態に影響し、`sq_compile` を呼び出す前に設定する必要があります。

Definition at line 259 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00260 {
00261     _ss(v)->_debuginfo = enable?true:false;
00262 }
```

4.1.2.24 sq_free()

```
void sq_free (
    void * p,
    SQUnsignedInteger size )
```

Squirrel のカスタムアロケータを使用してメモリを解放します。

Parameters

<i>p</i>	解放するメモリブロックへのポインタ。
<i>size</i>	解放するメモリのサイズ (バイト単位)。

Squirrel の内部実装と同じメモリアロケータ (デフォルトでは標準ライブラリの `free`) をC 側から利用するための関数です。

Definition at line 2729 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).


```

02730 {
02731     SQ_FREE(p,size);
02732 }

```

4.1.2.25 sq_get()

```

SQRESULT sq_get (
    HSQUIRRELVm v,
    SQInteger idx )

```

オブジェクトからキーに対応する値を取得します（メタメソッドを考慮します）。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

オブジェクトからキーに対応する値を取得します。この操作は `_get` メタメソッドをトリガーする可能性があり、デリゲートチェーンも検索します。呼び出す前に、取得したい値のキーをスタックトップにプッシュしておく必要があります。成功した場合、キーはポップされ、対応する値がスタックトップにプッシュされます。キーが見つからない場合、エラーが返されます。

Definition at line 1795 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

01796 {
01797     SQObjectPtr &self=stack_get(v,idx);
01798     SQObjectPtr &obj = v->GetUp(-1);
01799     if(v->Get(self,obj,obj,false,DONT_FALL_BACK))
01800         return SQ_OK;
01801     v->Pop();
01802     return SQ_ERROR;
01803 }

```

4.1.2.26 sq_getattributes()

```

SQRESULT sq_getattributes (
    HSQUIRRELVm v,
    SQInteger idx )

```

クラスまたはそのメンバーの属性を取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	クラスのスタックインデックス。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

この関数を呼び出す前に、属性を取得したいメンバーのキーをスタックトップにプッシュしておく必要があります。キーが `null` の場合はクラス自体の属性テーブルが取得されます。成功すると、キーはポップされ、属性テーブルがスタックにプッシュされます。

Definition at line 2353 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02354 {
02355     SQObjectPtr *o = NULL;
02356     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
02357     SQObjectPtr &key = stack_get(v,-1);
02358     SQObjectPtr attrs;
02359     if(sq_type(key) == OT_NULL) {
02360         attrs = _class(*o)->_attributes;
02361         v->Pop();
02362         v->Push(attrs);
02363         return SQ_OK;
02364     }
02365     else if(_class(*o)->GetAttributes(key,attrs)) {
02366         v->Pop();
02367         v->Push(attrs);
02368         return SQ_OK;
02369     }
02370     return sq_throwerror(v,_SC("wrong index"));
02371 }
```

References [_GETSAFE_OBJ](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.27 sq_getbase()

```
SQRESULT sq_getbase (
    HSQUIRRELVm v,
    SQInteger idx )
```

クラスの基底クラスを取得し、スタックにプッシュします。

Parameters

<code>v</code>	対象のSquirrel VM。
<code>idx</code>	クラスのスタックインデックス。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

指定されたクラスの基底クラスを取得し、スタックにプッシュします。継承関係をC側からたどるために使用します。クラスに基底クラスがない場合は `null` をプッシュします。

Definition at line 2484 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

02485 {
02486     SQObjectPtr *o = NULL;
02487     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
02488     if(_class(*o)->_base)
02489         v->Push(SQObjectPtr(_class(*o)->_base));
02490     else
02491         v->PushNull();
02492     return SQ_OK;
02493 }

```

References [_GETSAFE_OBJ](#).

4.1.2.28 sq_getbool()

```

SQRESULT sq_getbool (
    HSQUIRRELVm v,
    SQInteger idx,
    SQBool * b )

```

指定されたインデックスのオブジェクトがブール値であるか確認し、その値を取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。
<i>b</i>	結果のブール値を格納するポインタ。

Returns

対象がブール値であればSQ_OK、そうでなければSQ_ERROR。

sq_tobool とは異なり、この関数は対象オブジェクトの型が厳密にブール型であることを要求します。他の型（null, 整数など）が指定された場合はエラーを返します。

Definition at line 1207 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

01208 {
01209     SQObjectPtr &o = stack_get(v, idx);
01210     if(sq_isbool(o)) {
01211         *b = _integer(o);
01212         return SQ_OK;
01213     }
01214     return SQ_ERROR;
01215 }

```

4.1.2.29 sq_getbyhandle()

```

SQRESULT sq_getbyhandle (
    HSQUIRRELVm v,
    SQInteger idx,
    const HSQMEMBERHANDLE * handle )

```

メンバーハンドルを使用して、クラスまたはインスタンスのメンバーの値を取得し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	クラスまたはインスタンスのスタックインデックス。
<i>handle</i>	<i>Sq_getmemberhandle</i> で取得したハンドル。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

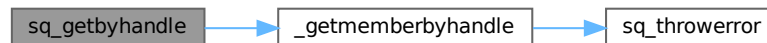
sq_getmemberhandle で事前に取得したハンドルを使い、高速にメンバーの値を取得します。ハッシュ検索を行わないため、ループ内で同じメンバーに繰り返しアクセスする場合に非常に効率的です。

Definition at line 2445 of file squirrel3_squirrel_sqapiByGemini.cpp.

```
02446 {
02447     SQObjectPtr &self = stack_get(v, idx);
02448     SQObjectPtr *val = NULL;
02449     if(SQ_FAILED(_getmemberbyhandle(v, self, handle, val))) {
02450         return SQ_ERROR;
02451     }
02452     v->Push(_realval(*val));
02453     return SQ_OK;
02454 }
```

References [_getmemberbyhandle\(\)](#).

Here is the call graph for this function:

**4.1.2.30 sq_getcallee()**

```
SQRESULT sq_getcallee (
    HSQUIRRELVm v )
```

現在の関数の呼び出し元 (callee) のクロージャを取得し、スタックにプッシュします。

Parameters

v	対象のSquirrel VM。
---	-----------------

Returns

成功した場合はSQ_OK、コールスタックが浅すぎて呼び出し元が存在しない場合はSQ_ERROR。

現在実行中の関数の呼び出し元 (callee) のクロージャオブジェクトを取得し、スタックにプッシュします。これにより、関数内から自身を呼び出した関数や環境にアクセスできます。コールスタックの深さが1 (トップレベルの呼び出し) の場合、呼び出し元は存在しないためエラーが返されます。

Definition at line 2236 of file squirrel3_squirrel_sqapiByGemini.cpp.

```
02237 {
02238     if(v->_callsstacksize > 1)
02239     {
02240         v->Push(v->_callsstack[v->_callsstacksize - 2]._closure);
02241         return SQ_OK;
02242     }
02243     return sq_throwerror(v, _SC("no closure in the calls stack"));
02244 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.31 sq_getclass()

```
SQRESULT sq_getclass (
    HSQUIRRELVm v,
    SQInteger idx )
```

インスタンスからそのクラスオブジェクトを取得し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	インスタンスのスタックインデックス。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

指定されたインスタンスがどのクラスから生成されたかを取得し、そのクラスオブジェクトをスタックにプッシュします。インスタンスの型情報を実行時に調べるために使用します。

Definition at line 2502 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02503 {
02504     SQObjectPtr *o = NULL;
02505     _GETSAFE_OBJ(v, idx, OT_INSTANCE, o);
02506     v->Push(SQObjectPtr(_instance(*o)->_class));
02507     return SQ_OK;
02508 }
```

References [_GETSAFE_OBJ](#).

4.1.2.32 sq_getclosureinfo()

```
SQRESULT sq_getclosureinfo (
    HSQUIRRELVm v,
    SQInteger idx,
    SQInteger * nparams,
    SQInteger * nfreevars )
```

クロージャに関する情報（パラメータ数、自由変数の数）を取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	クロージャが格納されているスタックインデックス。
<i>nparams</i>	パラメータの数を格納するポインタ。
<i>nfreevars</i>	自由変数の数を格納するポインタ。

Returns

成功した場合はSQ_OK、対象がクロージャでない場合はSQ_ERROR。

指定された *idx* にあるクロージャの情報を取得します。Squirrel のスクリプトクロージャ (OT_CLOSURE) とネイティブクロージャ (OT_NATIVECLOSURE) の両方に対応しています。スクリプトクロージャの場合、パラメータ数は関数定義から取得されます。ネイティブクロージャの場合、`sq_setparamscheck` で設定された値が返されます。

Definition at line 742 of file `squirrel3_squirrel_sqapiByGemini.cpp`.

```

00743 {
00744     SQObject o = stack_get(v, idx);
00745     if(sq_type(o) == OT_CLOSURE) {
00746         SQClosure *c = _closure(o);
00747         SQFunctionProto *proto = c->_function;
00748         *nparams = proto->_nparameters;
00749         *nfreevars = proto->_noutervalues;
00750         return SQ_OK;
00751     }
00752     else if(sq_type(o) == OT_NATIVECLOSURE)
00753     {
00754         SQNativeClosure *c = _nativeclosure(o);
00755         *nparams = c->_nparamscheck;
00756         *nfreevars = (SQInteger)c->_noutervalues;
00757         return SQ_OK;
00758     }
00759     return sq_throwerror(v, _SC("the object is not a closure"));
00760 }
```

References `sq_throwerror()`.

Here is the call graph for this function:



4.1.2.33 sq_getclosurename()

```

SQRESULT sq_getclosurename (
    HSQUIRRELVm v,
    SQInteger idx )
```

クロージャの名前を取得し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	クロージャが格納されているスタックインデックス。

Returns

成功した場合はSQ_OK、対象がクロージャでない場合はSQ_ERROR。

指定された *idx* にあるクロージャの名前を文字列として取得し、スタックにプッシュします。スクリプトクロージャの場合は関数名を、ネイティブクロージャの場合は `sq_setnativeclosurename` で設定された名前を取得します。

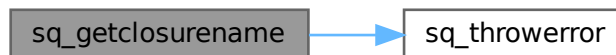
Definition at line 862 of file `squirrel3_squirrel_sqapiByGemini.cpp`.

```

00863 {
00864     SQObjectPtr &o = stack_get(v, idx);
00865     if(!sq_isnativeclosure(o) &&
00866         !sq_isclosure(o))
00867         return sq_throwerror(v, _SC("the target is not a closure"));
00868     if(sq_isnativeclosure(o))
00869     {
00870         v->Push(_nativeclosure(o)->_name);
00871     }
00872     else { //closure
00873         v->Push(_closure(o)->_function->_name);
00874     }
00875     return SQ_OK;
00876 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.34 sq_getclosureroot()

```

SQRESULT sq_getclosureroot (
    HSQUIRRELVm v,
    SQInteger idx )
```

スクリプトクロージャの現在のルートテーブルを取得し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	クロージャが格納されているスタックインデックス。

Returns

成功した場合はSQ_OK、対象がクロージャでない場合はSQ_ERROR。

クロージャに設定されているルートテーブルをスタックにプッシュします。ルートテーブルが設定されていない場合は、クロージャ生成時にキャプチャされたデフォルトのルートテーブルが返されます。

Definition at line 905 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00906 {
00907     SQObjectPtr &c = stack_get(v,idx);
00908     if(!sq_isclosure(c)) return sq_throwerror(v, _SC("closure expected"));
00909     v->Push(_closure(c)->_root->_obj);
00910     return SQ_OK;
00911 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:

**4.1.2.35 sq_getdefaultdelegate()**

```
SQRESULT sq_getdefaultdelegate (
    HSQUIRRELVm v,
    SQObjectType t )
```

指定されたオブジェクト型のデフォルトデリゲートを取得し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>t</i>	デフォルトデリゲートを取得したいオブジェクトの型。

Returns

成功した場合はSQ_OK、その型にデフォルトデリゲートが存在しない場合はSQ_ERROR。

Squirrel の組み込み型 (OT_STRING, OT_INTEGER など) に設定されているデフォルトデリゲートを取得します。これにより、`string.slice()` のような組み込み型のメソッドをC 側から取得したり、置き換えたりすることが可能になります。

Definition at line 2565 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02566 {
02567     SQSharedState *ss = _ss(v);
02568     switch(t) {
02569         case OT_TABLE: v->Push(ss->_table_default_delegate); break;
02570         case OT_ARRAY: v->Push(ss->_array_default_delegate); break;
```



```

02571     case OT_STRING: v->Push(ss->_string_default_delegate); break;
02572     case OT_INTEGER: case OT_FLOAT: v->Push(ss->_number_default_delegate); break;
02573     case OT_GENERATOR: v->Push(ss->_generator_default_delegate); break;
02574     case OT_CLOSURE: case OT_NATIVECLOSURE: v->Push(ss->_closure_default_delegate); break;
02575     case OT_THREAD: v->Push(ss->_thread_default_delegate); break;
02576     case OT_CLASS: v->Push(ss->_class_default_delegate); break;
02577     case OT_INSTANCE: v->Push(ss->_instance_default_delegate); break;
02578     case OT_WEAKREF: v->Push(ss->_weakref_default_delegate); break;
02579     default: return sq_throwerror(v,_SC("the type doesn't have a default delegate"));
02580 }
02581 return SQ_OK;
02582 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.36 sq_getdelegate()

```

SQLRESULT sq_getdelegate (
    HSQUIRRELVm v,
    SQInteger idx )

```

テーブルまたはユーザーデータのデリゲートを取得し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	テーブルまたはユーザーデータのスタックインデックス。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

指定されたインデックスのテーブルまたはユーザーデータのデリゲートを取得し、スタックにプッシュします。デリゲートが設定されていない場合は、null がプッシュされます。

Definition at line 1770 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

01771 {
01772     SQObjectPtr &self=stack_get(v,idx);
01773     switch(sq_type(self)) {
01774     case OT_TABLE:
01775     case OT_USERDATA:
01776         if(!_delegable(self)->_delegate) {
01777             v->PushNull();
01778             break;
01779         }
01780         v->Push(SQObjectPtr(_delegable(self)->_delegate));
01781         break;
01782     default: return sq_throwerror(v,_SC("wrong type")); break;

```

```

01783     }
01784     return SQ_OK;
01785
01786 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.37 sq_geterrorfunc()

```

SQPRINTFUNCTION sq_geterrorfunc (
    HSQUIRRELVM v )
```

現在設定されている標準エラー出力用のプリント関数を取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

Returns

エラープリント関数のポインタ。

`sq_setprintfunc` で設定された標準エラー出力用の関数ポインタを返します。

Definition at line 2694 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

02695 {
02696     return _ss(v)->_errorfunc;
02697 }
```

4.1.2.38 sq_getfloat()

```

SQRESULT sq_getfloat (
    HSQUIRRELVM v,
    SQInteger idx,
    SQFloat * f )
```

指定されたインデックスのオブジェクトを浮動小数点数に変換して取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。
<i>f</i>	結果の浮動小数点数を格納するポインタ。

Returns

変換に成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

対象オブジェクトが数値（整数または浮動小数点数）の場合に、その値を浮動小数点数に変換してポインタ `f` に格納します。それ以外の型の場合はエラーを返します。

Definition at line 1189 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01190 {
01191     SQObjectPtr &o = stack_get(v, idx);
01192     if(sq_isnumeric(o)) {
01193         *f = tofloat(o);
01194         return SQ_OK;
01195     }
01196     return SQ_ERROR;
01197 }
```

4.1.2.39 sq_getforeignptr()

```
SQUserPointer sq_getforeignptr (
    HSQUIRRELVm v )
```

VM に設定されている外部ポインタを取得します。

Parameters

<code>v</code>	対象のSquirrel VM。
----------------	-----------------

Returns

以前に `sq_setforeignptr` で設定されたポインタ。

VM インスタンスに `sq_setforeignptr` で設定された外部ポインタを取得します。ホストアプリケーションがVM に関連付けた独自のデータを取得するために使用します。

Definition at line 1015 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01016 {
01017     return v->_foreignptr;
01018 }
```

4.1.2.40 sq_getfreevariable()

```
const SQChar * sq_getfreevariable (
    HSQUIRRELVm v,
    SQInteger idx,
    SQUnsignedInteger nval )
```

指定されたインデックスのクロージャから自由変数を取得します。

Parameters

<code>v</code>	対象のSquirrel VM。
<code>idx</code>	クロージャのスタックインデックス。
<code>nval</code>	取得する自由変数のインデックス (0 から始まる)。

Returns

自由変数の名前。見つからない場合や対象がクロージャでない場合はNULL。

クロージャがキャプチャした自由変数（外部のローカル変数）にC側からアクセスします。成功した場合、指定されたインデックス (nval) の自由変数の値を取得してスタックにプッシュし、その名前を返します。デバッグや高度なメタプログラミングに使用されます。

Definition at line 2254 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02255 {
02256     SQObjectPtr &self=stack_get(v,idx);
02257     const SQChar *name = NULL;
02258     switch(sq_type(self))
02259     {
02260     case OT_CLOSURE:{
02261         SQClosure *clo = _closure(self);
02262         SQFunctionProto *fp = clo->_function;
02263         if(((SQUnsignedInteger)fp->noutervalues) > nval) {
02264             v->Push(*(_outer(clo->_outervalues[nval])->_valptr));
02265             SQOuterVar &ov = fp->_outervalues[nval];
02266             name = _stringval(ov._name);
02267         }
02268     }
02269     break;
02270     case OT_NATIVECLOSURE:{
02271         SQNativeClosure *clo = _nativeclosure(self);
02272         if(clo->noutervalues > nval) {
02273             v->Push(clo->_outervalues[nval]);
02274             name = _SC("@NATIVE");
02275         }
02276     }
02277     break;
02278     default: break; //shutup compiler
02279     }
02280     return name;
02281 }
```

4.1.2.41 sq_gethash()

```
SQHash sq_gethash (
    HSQUIRRELVm v,
    SQInteger idx )
```

指定されたインデックスのオブジェクトのハッシュ値を取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。

Returns

オブジェクトのハッシュ値。

このハッシュ値は、オブジェクトがテーブルのキーとして使用される際に内部的に計算されるものと同じです。異なるオブジェクトでも同じハッシュ値を返す可能性があります、同じオブジェクトは常に同じハッシュ値を返します。この関数はスタックの状態を変更しません。

Definition at line 1321 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01322 {
01323     SQObjectPtr &o = stack_get(v, idx);
01324     return HashObj(o);
01325 }
```

4.1.2.42 sq_getinstanceup()

```
SQRESULT sq_getinstanceup (
    HSQUIRRELVM v,
    SQInteger idx,
    SQUserPointer * p,
    SQUserPointer typetag,
    SQBool throwerror )
```

クラスインスタンスからユーザーポインタ（またはユーザーデータ領域）を取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	インスタンスのスタックインデックス。
<i>p</i>	ユーザーポインタを格納するポインタ。
<i>typetag</i>	期待される型タグ。0の場合は型タグのチェックを行いません。
<i>throwerror</i>	型が一致しない場合にエラーをスローするかどうか。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

typetag が0でない場合、インスタンスのクラスまたはその基底クラスのいずれかが指定された型タグを持っているかを確認します。これにより、C++のdynamic_castのような安全な型チェックが可能です。この関数はsq_setinstanceupで設定されたポインタ、またはsq_setclassudsizeで確保されたユーザーデータ領域へのポインタを取得します。

Definition at line 1457 of file squirrel3_squirrel_sqapiByGemini.cpp.

```
01458 {
01459     SQObjectPtr &o = stack_get(v, idx);
01460     if (sq_type(o) != OT_INSTANCE) return throwerror ? sq_throwerror(v, _SC("the object is not a class
instance")) : SQ_ERROR;
01461     (*p) = _instance(o)->_userpointer;
01462     if (typetag != 0) {
01463         SQClass *cl = _instance(o)->_class;
01464         do {
01465             if (cl->_typetag == typetag)
01466                 return SQ_OK;
01467             cl = cl->_base;
01468         } while (cl != NULL);
01469         return throwerror ? sq_throwerror(v, _SC("invalid type tag")) : SQ_ERROR;
01470     }
01471     return SQ_OK;
01472 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.43 sq_getinteger()

```
SQRESULT sq_getinteger (
    HSQUIRRELVM v,
    SQInteger idx,
    SQInteger * i )
```

指定されたインデックスのオブジェクトを整数に変換して取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。
<i>i</i>	結果の整数を格納するポインタ。

Returns

変換に成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

対象オブジェクトが数値（整数または浮動小数点数）またはブール値の場合に、その値を整数に変換してポインタ *i* に格納します。浮動小数点数は小数点以下が切り捨てられます。ブール値は `true` が 1、`false` が 0 になります。それ以外の型の場合はエラーを返します。

Definition at line 1167 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01168 {
01169     SQObjectPtr &o = stack_get(v, idx);
01170     if(sq_isnumeric(o)) {
01171         *i = tointeger(o);
01172         return SQ_OK;
01173     }
01174     if(sq_isbool(o)) {
01175         *i = SQVM::IsFalse(o)?SQFalse:SQTrue;
01176         return SQ_OK;
01177     }
01178     return SQ_ERROR;
01179 }
```

4.1.2.44 sq_getlasterror()

```
void sq_getlasterror (
    HSQUIRRELVM v )
```

VM に最後に設定されたエラーオブジェクトを取得し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

VM に最後に設定されたエラーオブジェクト (`lasterror`) をスタックにプッシュします。`sq_call` などが `SQ_ERROR` を返した後に、具体的なエラー内容を取得するために使用します。

Definition at line 1959 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01960 {
01961     v->Push(v->_lasterror);
01962 }
```

4.1.2.45 sq_getlocal()

```
const SQChar * sq_getlocal (
    HSQUIRRELVM v,
    SQUnsignedInteger level,
    SQUnsignedInteger idx )
```

実行中の関数のローカル変数に関する情報を取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>level</i>	コールスタックのレベル (0 は現在の関数、1 はその呼び出し元など)。
<i>idx</i>	ローカル変数のインデックス。

Returns

ローカル変数の名前。見つからない場合はNULL。

指定されたコールスタックレベルのローカル変数または自由変数の情報を取得します。level は0が現在の関数、1が呼び出し元を指します。成功した場合、変数の名前を返し、その値をスタックにプッシュします。この関数はデバッガの実装など、高度な目的のために使用されます。

Definition at line 1871 of file squirrel3_squirrel_sqapiByGemini.cpp.

```
01872 {
01873     SQUnsignedInteger cstksize=v->_callsstacksize;
01874     SQUnsignedInteger lvl=(cstksize-level)-1;
01875     SQInteger stackbase=v->_stackbase;
01876     if (lvl<cstksize){
01877         for(SQUnsignedInteger i=0;i<level;i++){
01878             SQVM::CallInfo &ci=v->_callsstack[(cstksize-i)-1];
01879             stackbase-=ci._prevstkbase;
01880         }
01881         SQVM::CallInfo &ci=v->_callsstack[lvl];
01882         if(sq_type(ci._closure)!=OT_CLOSURE)
01883             return NULL;
01884         SQClosure *c=_closure(ci._closure);
01885         SQFunctionProto *func=c->_function;
01886         if(func->_noutervalues > (SQInteger)idx) {
01887             v->Push(*_outer(c->_outervalues[idx])->_valptr);
01888             return _stringval(func->_outervalues[idx]._name);
01889         }
01890         idx -= func->_noutervalues;
01891         return func->GetLocal(v,stackbase,idx, (SQInteger)(ci._ip-func->_instructions)-1);
01892     }
01893     return NULL;
01894 }
```

4.1.2.46 sq_getmemberhandle()

```
SQRESULT sq_getmemberhandle (
    HSQUIRRELVM v,
    SQInteger idx,
    HSQMEMBERHANDLE * handle )
```

クラスメンバーへの高速なアクセスを可能にするハンドルを取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	クラスのスタックインデックス。
<i>handle</i>	結果のメンバーハンドルを格納するポインタ。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

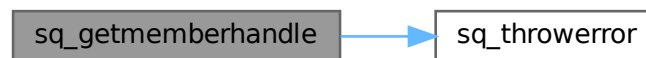
この関数を呼び出す前に、メンバーのキーをスタックトップにプッシュしておく必要があります。取得したハンドルは `sq_getbyhandle` や `sq_setbyhandle` と共に使用することで、ハッシュテーブルのルックアップをバイパスしてメンバーに直接アクセスできます。頻繁に同じメンバーにアクセスする場合のパフォーマンスを向上させます。成功するとキーはスタックからポップされます。

Definition at line 2381 of file `squirrel3_squirrel_sqapiByGemini.cpp`.

```
02382 {
02383     SQObjectPtr *o = NULL;
02384     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
02385     SQObjectPtr &key = stack_get(v,-1);
02386     SQTable *m = _class(*o)->_members;
02387     SQObjectPtr val;
02388     if(m->Get(key,val)) {
02389         handle->_static = _isfield(val) ? SQFalse : SQTrue;
02390         handle->_index = _member_idx(val);
02391         v->Pop();
02392         return SQ_OK;
02393     }
02394     return sq_throwerror(v,_SC("wrong index"));
02395 }
```

References `_GETSAFE_OBJ`, and `sq_throwerror()`.

Here is the call graph for this function:



4.1.2.47 sq_getobjtypetag()

```
SQRESULT sq_getobjtypetag (
    const HSQOBJECT * o,
    SQUserPointer * typetag )
```

HSQOBJECT から型タグを取得します。

Parameters

<i>o</i>	対象のHSQOBJECT へのポインタ。
<i>typetag</i>	型タグを格納するポインタ。

Returns

対象が型タグを持つオブジェクト（インスタンス、ユーザーデータ、クラス）であればSQ_OK、そうでなければSQ_ERROR。

`sq_gettypetag` と似ていますが、スタックインデックスの代わりにオブジェクトハンドルを直接取ります。インスタンスの場合は、そのクラスの型タグを返します。

Definition at line 1371 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01372 {
01373     switch(sq_type(*o)) {
01374         case OT_INSTANCE: *typetag = _instance(*o)->_class->_typetag; break;
01375         case OT_USERDATA: *typetag = _userdata(*o)->_typetag; break;
01376         case OT_CLASS:    *typetag = _class(*o)->_typetag; break;
01377         default: return SQ_ERROR;
01378     }
01379     return SQ_OK;
01380 }
```

Referenced by [sq_gettypetag\(\)](#).

Here is the caller graph for this function:



4.1.2.48 sq_getprintfunc()

```
SQPRINTFUNCTION sq_getprintfunc (
    HSQUIRRELVm v )
```

現在設定されている標準出力用のプリント関数を取得します。

Parameters

v	対象のSquirrel VM。
---	-----------------

Returns

プリント関数のポインタ。

`sq_setprintfunc` で設定された標準出力用の関数ポインタを返します。

Definition at line 2683 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02684 {
02685     return _ss(v)->_printfunc;
02686 }
```

4.1.2.49 sq_getrefcount()

```
SQUnsignedInteger sq_getrefcount (
    HSQUIRRELVm v,
    HSQOBJECT * po )
```

Squirrel オブジェクトの現在の参照カウントを取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>po</i>	対象のオブジェクトへのポインタ。

Returns

オブジェクトの参照カウント。参照カウントされない型の場合は 0。

この関数は主にデバッグ目的で、特定のオブジェクトがC 側からいくつ参照されているかを確認するために使用されます。NO_GARBAGE_COLLECTOR ビルドとGC ビルドで参照カウントの管理方法が異なるため、結果はビルド形態に依存します。

Definition at line 303 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00304 {
00305     if (!ISREFCOUNTED(sq_type(*po))) return 0;
00306 #ifdef NO_GARBAGE_COLLECTOR
00307     return po->_unVal.pRefCounted->_uiRef;
00308 #else
00309     return _ss(v)->_refs_table.GetRefCount(*po);
00310 #endif
00311 }
```

4.1.2.50 sq_getreleasehook()

```
SQRELEASEHOOK sq_getreleasehook (
    HSQUIRRELVM v,
    SQInteger idx )
```

ユーザーデータ、インスタンス、またはクラスからリリースフックを取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。

Returns

設定されているリリースフック関数へのポインタ。設定されていない場合はNULL。

指定されたインデックスのユーザーデータ、インスタンス、またはクラスに現在設定されているリリースフック関数へのポインタを取得します。フックが設定されていない場合は NULL を返します。

Definition at line 2120 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02121 {
02122     SQObjectPtr &ud=stack_get(v,idx);
02123     switch(sq_type(ud) ) {
02124     case OT_USERDATA: return _userdata(ud)->_hook; break;
02125     case OT_INSTANCE: return _instance(ud)->_hook; break;
02126     case OT_CLASS:    return _class(ud)->_hook; break;
02127     default: return NULL;
02128     }
02129 }
```

4.1.2.51 sq_getscratchpad()

```
SQChar * sq_getscratchpad (
    HSQUIRRELVm v,
    SQInteger minsize )
```

VM の共有状態から一時的なメモリ領域（スクラッチパッド）を取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>minsize</i>	必要とする最小サイズ（バイト単位）。

Returns

スクラッチパッドへのポインタ。

VM の共有状態から、一時的なデータ操作に使えるメモリ領域（スクラッチパッド）へのポインタを取得します。このメモリ領域は、複数のAPI 呼び出しにまたがって内容が保証されません。主に、C API 内部で一時的な文字列を構築するなどの短期間の用途に使用されます。要求された *minsize* より大きな領域が返されることもあります。

Definition at line 2194 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02195 {
02196     return _ss(v)->GetScratchPad(minsize);
02197 }
```

4.1.2.52 sq_getsharedforeignptr()

```
SQUserPointer sq_getsharedforeignptr (
    HSQUIRRELVm v )
```

VM の共有状態に設定されている外部ポインタを取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

Returns

以前に `sq_setsharedforeignptr` で設定されたポインタ。

VM の共有状態に `sq_setsharedforeignptr` で設定された外部ポインタを取得します。ホストアプリケーションが共有状態に関連付けした独自のデータを取得するために使用します。

Definition at line 1037 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01038 {
01039     return _ss(v)->_foreignptr;
01040 }
```

4.1.2.53 sq_getsharedreleasehook()

```
SQRELEASEHOOK sq_getsharedreleasehook (
    HSQUIRRELVM v )
```

共有状態のリリースフックを取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

Returns

設定されている共有リリースフック関数へのポインタ。

現在VMの共有状態に設定されているリリースフック関数へのポインタを取得します。

Definition at line 1081 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01082 {
01083     return __ss(v)->_releasehook;
01084 }
```

4.1.2.54 sq_getsize()

```
SQInteger sq_getsize (
    HSQUIRRELVM v,
    SQInteger idx )
```

指定されたインデックスのオブジェクトのサイズを取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。

Returns

オブジェクトのサイズ。

サイズの意味はオブジェクトの型によって異なります:

- OT_STRING: 文字列の長さ
- OT_TABLE: 格納されているキーと値のペアの数
- OT_ARRAY: 配列の要素数
- OT_USERDATA: `sq_newuserdata` で確保されたメモリのサイズ
- OT_INSTANCE/OT_CLASS: `sq_setclassudsize` で設定されたユーザーデータ領域のサイズ サポートされていない型の場合はエラーがスローされます。

Definition at line 1298 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01299 {
01300     SQObjectPtr &o = stack_get(v, idx);
01301     SQObjectType type = sq_type(o);
01302     switch(type) {
01303     case OT_STRING:      return _string(o)->_len;
01304     case OT_TABLE:      return _table(o)->CountUsed();
01305     case OT_ARRAY:      return _array(o)->Size();
01306     case OT_USERDATA:   return _userdata(o)->_size;
01307     case OT_INSTANCE:  return _instance(o)->_class->_udsize;
01308     case OT_CLASS:     return _class(o)->_udsize;
01309     }
```

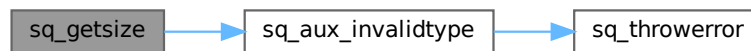
```

01309     default:
01310         return sq_aux_invalidtype(v, type);
01311     }
01312 }

```

References [sq_aux_invalidtype\(\)](#).

Here is the call graph for this function:



4.1.2.55 sq_getstackobj()

```

SQRESULT sq_getstackobj (
    HSQUIRRELVm v,
    SQInteger idx,
    HSQOBJECT * po )

```

指定されたスタックインデックスのオブジェクトへのハンドル（HSQOBJECT）を取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。
<i>po</i>	結果のHSQOBJECT を格納するポインタ。

Returns

常にSQ_OK。

このハンドルは、オブジェクトへの直接的なポインタのようなものです。sq_addrref と sq_release と組み合わせて使用することで、オブジェクトをスタック外で安全に保持するために使用できます。この関数自体は参照カウントを操作しません。

Definition at line 1857 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

01858 {
01859     *po=stack_get(v,idx);
01860     return SQ_OK;
01861 }

```

4.1.2.56 sq_getstring()

```

SQRESULT sq_getstring (
    HSQUIRRELVm v,
    SQInteger idx,
    const SQChar ** c )

```

指定されたインデックスの文字列オブジェクトから、C 文字列ポインタを取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	文字列オブジェクトのスタックインデックス。
<i>c</i>	文字列へのポインタを格納するポインタ。

Returns

対象が文字列であればSQ_OK、そうでなければSQ_ERROR。

sq_getstringandsize の簡易版で、文字列の長さは取得しません。ポインタの有効期間や注意点はsq_getstringandsize と同じです。

Definition at line 1243 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01244 {
01245     SQObjectPtr *o = NULL;
01246     _GETSAFE_OBJ(v, idx, OT_STRING,o);
01247     *c = _stringval(*o);
01248     return SQ_OK;
01249 }
```

References [_GETSAFE_OBJ](#).

4.1.2.57 sq_getstringandsize()

```
SQRESULT sq_getstringandsize (
    HSQUIRRELVM v,
    SQInteger idx,
    const SQChar ** c,
    SQInteger * size )
```

指定されたインデックスの文字列オブジェクトから、C 文字列ポインタと長さを取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	文字列オブジェクトのスタックインデックス。
<i>c</i>	文字列へのポインタを格納するポインタ。
<i>size</i>	文字列の長さを格納するポインタ。

Returns

対象が文字列であればSQ_OK、そうでなければSQ_ERROR。

取得されるポインタ (c) はSquirrel VM が管理する内部バッファを指しています。このポインタは、対象の文字列オブジェクトがGC によって回収されるまで有効です。文字列の内容を変更してはいけません。

Definition at line 1226 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01227 {
01228     SQObjectPtr *o = NULL;
01229     _GETSAFE_OBJ(v, idx, OT_STRING,o);
01230     *c = _stringval(*o);
01231     *size = _string(*o)->_len;
01232     return SQ_OK;
01233 }
```

References [_GETSAFE_OBJ](#).

4.1.2.58 sq_getthread()

```
SQRESULT sq_getthread (
    HSQUIRRELVM v,
    SQInteger idx,
    HSQUIRRELVM * thread )
```

指定されたインデックスのスレッドオブジェクトから、VM ハンドルを取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	スレッドオブジェクトのスタックインデックス。
<i>thread</i>	VM ハンドルを格納するポインタ。

Returns

対象がスレッドであればSQ_OK、そうでなければSQ_ERROR。

取得したVM ハンドル (thread) を使って、そのスレッド (コルーチン) を直接操作することができます (例: sq_wakeupvm で実行を再開する)。

Definition at line 1259 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01260 {
01261     SQObjectPtr *o = NULL;
01262     _GETSAFE_OBJ(v, idx, OT_THREAD,o);
01263     *thread = _thread(*o);
01264     return SQ_OK;
01265 }
```

References [_GETSAFE_OBJ](#).

4.1.2.59 sq_gettop()

```
SQInteger sq_gettop (
    HSQUIRRELVM v )
```

スタック上の要素数を取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

Returns

スタック上の要素数。

現在のスタックベースからスタックトップまでの要素数を返します。これは、現在のコールフレームでアクセス可能なスタック要素の総数に相当します。負のインデックス (例: -1 でトップ要素) を使用する際の基準となり、また、関数呼び出しのためにスタックにいくつかの引数が積まれているかを確認するためにも使用されます。

Definition at line 1480 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).


```

01481 {
01482     return (v->_top) - v->_stackbase;
01483 }

```

Referenced by [sq_settop\(\)](#).

Here is the caller graph for this function:



4.1.2.60 sq_gettype()

```

SQObjectType sq_gettype (
    HSQUIRRELVm v,
    SQInteger idx )

```

指定されたインデックスのスタック上のオブジェクトの型を取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。

Returns

オブジェクトの型 (SQObjectType)。

指定されたスタックインデックス *idx* にあるオブジェクトの型 (SQObjectType) を返します。スタック上のオブジェクトを操作する前に、その型を確認するために使用します。この関数はスタックの状態を変更しません。

Definition at line 1105 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

01106 {
01107     return sq_type(stack_get(v, idx));
01108 }

```

4.1.2.61 sq_gettypetag()

```

SQRESULT sq_gettypetag (
    HSQUIRRELVm v,
    SQInteger idx,
    SQUserPointer * typetag )

```

指定されたインデックスのオブジェクトから型タグを取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。
<i>typetag</i>	型タグを格納するポインタ。

Returns

対象が型タグを持つ型であればSQ_OK、そうでなければSQ_ERROR。

インスタンスの場合は、そのインスタンスが属するクラスの型タグを返します。ユーザーデータ、クラス、インスタンス以外のオブジェクトには型タグがないため、エラーが返されます。この関数はスタックの状態を変更しません。

Definition at line 1390 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01391 {
01392     SQObjectPtr &o = stack_get(v,idx);
01393     if (SQ_FAILED(sq_getobjtypetag(&o, typetag)))
01394         return SQ_ERROR; // this is not an error it should be a bool but would break backward
                                compatibility
01395     return SQ_OK;
01396 }
```

References [sq_getobjtypetag\(\)](#).

Here is the call graph for this function:



4.1.2.62 sq_getuserdata()

```
SQRESULT sq_getuserdata (
    HSQUIRRELVM v,
    SQInteger idx,
    SQUserPointer * p,
    SQUserPointer * typetag )
```

指定されたインデックスのユーザーデータオブジェクトから、データポインタと型タグを取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	ユーザーデータオブジェクトのスタックインデックス。
<i>p</i>	ユーザーデータへのポインタを格納するポインタ。
<i>typetag</i>	型タグを格納するポインタ (NULL も可)。

Returns

対象がユーザーデータであればSQ_OK、そうでなければSQ_ERROR。

`p` には `sq_newuserdata` で確保されたメモリ領域へのポインタが設定されます。`typetag` がNULL でない場合、`sq_settypetag` で設定された型タグがそこに格納されます。これにより、C 側でユーザーデータの種類を安全に識別できます。

Definition at line 1336 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01337 {
01338     SQObjectPtr *o = NULL;
01339     _GETSAFE_OBJ(v, idx, OT_USERDATA, o);
01340     (*p) = _userdataval(*o);
01341     if(typetag) *typetag = _userdata(*o)->_typetag;
01342     return SQ_OK;
01343 }
```

References [_GETSAFE_OBJ](#).

4.1.2.63 sq_getuserpointer()

```
SQRESULT sq_getuserpointer (
    HSQUIRRELM v,
    SQInteger idx,
    SQUserPointer * p )
```

指定されたインデックスのユーザーポインタオブジェクトから、ポインタ値を取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	ユーザーポインタオブジェクトのスタックインデックス。
<i>p</i>	ポインタ値を格納するポインタ。

Returns

対象がユーザーポインタであればSQ_OK、そうでなければSQ_ERROR。

`sq_pushuserpointer` でスタックに積まれたポインタの値を取得します。対象オブジェクトの型が `OT_USERPOINTER` でない場合はエラーを返します。

Definition at line 1406 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01407 {
01408     SQObjectPtr *o = NULL;
01409     _GETSAFE_OBJ(v, idx, OT_USERPOINTER, o);
01410     (*p) = _userpointer(*o);
01411     return SQ_OK;
01412 }
```

References [_GETSAFE_OBJ](#).

4.1.2.64 sq_getversion()

```
SQInteger sq_getversion ( )
```

Squirrel のバージョン番号を取得します。

Returns

Squirrel のバージョンを表す整数。(例: 301 はバージョン 3.0.1 を意味します)

バージョン番号は (major * 100) + (minor * 10) + patch の形式でエンコードされています。これにより、ホストアプリケーションは実行時に使用している Squirrel ライブラリのバージョンを確認できます。

Definition at line 220 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00221 {
00222     return SQUIRREL_VERSION_NUMBER;
00223 }
```

4.1.2.65 sq_getvmrefcount()

```
SQUnsignedInteger sq_getvmrefcount (
    HSQUIRRELVM SQ_UNUSED_ARG v,
    const HSQOBJECT * po )
```

Squirrel VM 内部のオブジェクトの参照カウントを取得します。

Parameters

<i>v</i>	対象の Squirrel VM (現在は未使用)。
<i>po</i>	対象のオブジェクトへのポインタ。

Returns

オブジェクトの内部参照カウント。参照カウントされない型の場合は 0。

`sq_getrefcount` とは異なり、この関数はガベージコレクタの参照テーブルではなく、オブジェクト自体の参照カウンタ (`_uiRef`) を直接返します。これは VM 内部での参照の数を示します。主にデバッグや高度なメモリ管理に使用されます。

Definition at line 343 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00344 {
00345     if (!ISREFCOUNTED(sq_type(*po))) return 0;
00346     return po->_unVal.pRefCounted->_uiRef;
00347 }
```

4.1.2.66 sq_getvmreleasehook()

```
SQRELEASEHOOK sq_getvmreleasehook (
    HSQUIRRELVM v )
```

VM 固有のリリースフックを取得します。

Parameters

<i>v</i>	対象の Squirrel VM。
----------	------------------

Returns

設定されているリリースフック関数へのポインタ。

現在VM インスタンスに設定されているリリースフック関数へのポインタを取得します。

Definition at line 1059 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01060 {
01061     return v->_releasehook;
01062 }
```

4.1.2.67 sq_getvmstate()

```
SQInteger sq_getvmstate (
    HSQUIRRELVM v )
```

VM の現在の状態を取得します。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

Returns

VM の状態を示す値 (SQ_VMSTATE_IDLE, SQ_VMSTATE_RUNNING, SQ_VMSTATE_SUSPENDED)。

VM の状態は以下の通りです:

- SQ_VMSTATE_IDLE: VM は現在何も実行していません。コールスタックが空の状態です。
- SQ_VMSTATE_RUNNING: VM は現在スクリプトを実行中です。コールスタックに 1 つ以上の呼び出しがあります。
- SQ_VMSTATE_SUSPENDED: VM は `sq_suspendvm` によって中断されています。 `sq_wakeupvm` で再開できます。

Definition at line 138 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00139 {
00140     if(v->_suspended)
00141         return SQ_VMSTATE_SUSPENDED;
00142     else {
00143         if(v->_callsstacksize != 0) return SQ_VMSTATE_RUNNING;
00144         else return SQ_VMSTATE_IDLE;
00145     }
00146 }
```

4.1.2.68 sq_getweakrefval()

```
SQRESULT sq_getweakrefval (
    HSQUIRRELVM v,
    SQInteger idx )
```

弱い参照から元のオブジェクトを取得し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	弱い参照オブジェクトのスタックインデックス。

Returns

成功した場合はSQ_OK、対象が弱い参照でない場合はSQ_ERROR。

弱い参照から元のオブジェクトを取得します。元のオブジェクトが既にGCによって回収されていれば null をプッシュします。キャッシュやオブジェクト間の親子関係の実装に役立ちます。

Definition at line 2548 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02549 {
02550     SQObjectPtr &o = stack_get(v, idx);
02551     if(sq_type(o) != OT_WEAKREF) {
02552         return sq_throwerror(v, _SC("the object must be a weakref"));
02553     }
02554     v->Push(_weakref(o)->_obj);
02555     return SQ_OK;
02556 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.69 sq_instanceof()

```
SQBool sq_instanceof (
    HSQUIRRELVM v )
```

インスタンスが特定のクラスのインスタンス（またはその派生クラスのインスタンス）であるかを確認します。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

Returns

インスタンスがクラスに属する場合はSQTrue、そうでない場合はSQFalse。エラーの場合はSQ_ERROR。

スタックトップ (-1) のインスタンスが、その下 (-2) にあるクラスのインスタンス（またはその派生クラスのインスタンス）であるかを判定します。この関数はSquirrel の instanceof 演算子に相当します。比較後、スタックの状態は変化しません。引数の型がインスタンスとクラスでない場合はエラーを返します。

Definition at line 586 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00587 {
00588     SQObjectPtr &inst = stack_get(v,-1);
00589     SQObjectPtr &cl = stack_get(v,-2);
00590     if(sq_type(inst) != OT_INSTANCE || sq_type(cl) != OT_CLASS)
00591         return sq_throwerror(v,_SC("invalid param type"));
00592     return _instance(inst)->InstanceOf(_class(cl)) ? SQTrue : SQFalse;
00593 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.70 sq_malloc()

```
void * sq_malloc (
    SQUnsignedInteger size )
```

Squirrel のカスタムアロケータを使用してメモリを確保します。

Parameters

<i>size</i>	確保するサイズ (バイト単位)。
-------------	------------------

Returns

確保されたメモリへのポインタ。

Squirrel の内部実装と同じメモリアロケータ (デフォルトでは標準ライブラリの malloc) を C 側から利用するための関数です。カスタムメモリアロケータで Squirrel をビルドした場合、この関数もそのカスタムアロケータを使用します。

Definition at line 2705 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02706 {
02707     return SQ_MALLOC(size);
02708 }
```

4.1.2.71 sq_move()

```
void sq_move (
    HSQUIRRELVM dest,
    HSQUIRRELVM src,
    SQInteger idx )
```

ある VM のスタックから別の VM のスタックへオブジェクトを移動 (コピー) します。

Parameters

<i>dest</i>	移動先のVM。
<i>src</i>	移動元のVM。
<i>idx</i>	移動元のスタック上のオブジェクトのインデックス。

src VM の *idx* にあるオブジェクトを取得し、*dest* VM のスタックのトップにプッシュします。これは、VM 間（例えばメインVM とスレッド間）でデータをやり取りする際に使用されます。移動元のスタックの状態は変化しません。

Definition at line 2659 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02660 {
02661     dest->Push(stack_get(src, idx));
02662 }
```

4.1.2.72 sq_newarray()

```
void sq_newarray (
    HSQUIRRELVm v,
    SQInteger size )
```

指定されたサイズの新しい配列を作成し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>size</i>	配列の初期サイズ。

指定された *size* を持つ配列オブジェクトを新しく生成し、スタックのトップにプッシュします。新しく作成された配列の要素はすべて `null` で初期化されます。

Definition at line 553 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00554 {
00555     v->Push(SQArray::Create(_ss(v), size));
00556 }
```

4.1.2.73 sq_newclass()

```
SQRESULT sq_newclass (
    HSQUIRRELVm v,
    SQBool hasbase )
```

新しいクラスを作成し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>hasbase</i>	<code>true</code> の場合、スタックトップのクラスを基底クラスとして継承します。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

新しいクラスオブジェクトを作成し、スタックにプッシュします。hasbase が true の場合、この関数を呼び出す前に基底クラスをスタックにプッシュしておく必要があります。成功すると、基底クラスはスタックからポップされ、新しく作成されたクラスオブジェクトがスタックにプッシュされます。hasbase が false の場合は、スタック操作は不要です。作成されたクラスは、後続のAPI呼び出し (sq_newslot など) でメンバーを定義できます。

Definition at line 565 of file squirrel3_squirrel_sqapiByGemini.cpp.

```
00566 {
00567     SQClass *baseclass = NULL;
00568     if (hasbase) {
00569         SQObjectPtr &base = stack_get(v, -1);
00570         if (sq_type(base) != OT_CLASS)
00571             return sq_throwerror(v, _SC("invalid base type"));
00572         baseclass = _class(base);
00573     }
00574     SQClass *newclass = SQClass::Create(_ss(v), baseclass);
00575     if (baseclass) v->Pop();
00576     v->Push(newclass);
00577     return SQ_OK;
00578 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.74 sq_newclosure()

```
void sq_newclosure (
    HSQUIRRELVm v,
    SQFUNCTION func,
    SQUnsignedInteger nfreevars )
```

C/C++関数から新しいネイティブクロージャを作成し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>func</i>	Squirrel から呼び出されるC/C++関数へのポインタ。
<i>nfreevars</i>	クロージャがキャプチャする自由変数の数。

C/C++関数 (SQFUNCTION) から新しいネイティブクロージャを作成し、スタックにプッシュします。nfreevars には、クロージャがキャプチャする自由変数の数を指定します。この関数を呼び出す前に、nfreevars 個の自由変数をスタックにプッシュしておく必要があります。これらの自由変数はクロージャ

にバインドされ、クロージャが呼び出される際にアクセスできます。バインド後、自由変数はスタックからポップされます。

Definition at line 722 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00723 {
00724     SQNativeClosure *nc = SQNativeClosure::Create(_ss(v), func, nfreevars);
00725     nc->_nparamscheck = 0;
00726     for(SQUnsignedInteger i = 0; i < nfreevars; i++) {
00727         nc->_outervalues[i] = v->Top();
00728         v->Pop();
00729     }
00730     v->Push(SQObjectPtr(nc));
00731 }
```

4.1.2.75 sq_newmember()

```
SQRESULT sq_newmember (
    HSQUIRRELVm v,
    SQInteger idx,
    SQBool bstatic )
```

クラスに新しいメンバー（フィールドまたはメソッド）を属性付きで追加します。メタメソッドを考慮します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	クラスのスタックインデックス。
<i>bstatic</i>	静的メンバーとして追加するかどうか。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

この関数を呼び出す前に、スタックに属性、値、キーの順でプッシュしておく必要があります (スタック: ..., attributes, value, key)。属性は、メンバーに関連付けられる追加のメタデータ（通常はテーブル）です。この操作は `_newmember` メタメソッドをトリガーする可能性があります。メンバーが既に存在する場合、この関数はエラーを返します。成功すると、キー、値、属性はスタックからポップされます。

Definition at line 1664 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01665 {
01666     SQObjectPtr &self = stack_get(v, idx);
01667     if(sq_type(self) != OT_CLASS) return sq_throwerror(v, _SC("new member only works with classes"));
01668     SQObjectPtr &key = v->GetUp(-3);
01669     if(sq_type(key) == OT_NULL) return sq_throwerror(v, _SC("null key"));
01670     if(!v->NewSlotA(self, key, v->GetUp(-2), v->GetUp(-1), bstatic?true:false, false)) {
01671         v->Pop(3);
01672         return SQ_ERROR;
01673     }
01674     v->Pop(3);
01675     return SQ_OK;
01676 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.76 sq_newslot()

```

SQRESULT sq_newslot (
    HSQUIRRELVM v,
    SQInteger idx,
    SQBool bstatic )
  
```

テーブルまたはクラスに新しいスロット（メンバー）を作成します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	テーブルまたはクラスのスタックインデックス。
<i>bstatic</i>	(クラスの場合のみ) 静的メンバーとして作成するかどうか。

Returns

常にSQ_OK。エラーはVM内で発生します。

この関数を呼び出す前に、スタックに値、その次にキーをプッシュしておく必要があります (スタック: ..., value, key)。成功すると、キーと値はスタックからポップされます。null はキーとして使用できません。この操作は `_newslot` メタメソッドをトリガーする可能性があります。スロットが既に存在する場合、上書きは行われず、エラーにもなりません（何も起こりません）。値を更新したい場合は `sq_set` を使用してください。

Definition at line 1556 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

01557 {
01558     sq_aux_paramscheck(v, 3);
01559     SQObjectPtr &self = stack_get(v, idx);
01560     if(sq_type(self) == OT_TABLE || sq_type(self) == OT_CLASS) {
01561         SQObjectPtr &key = v->GetUp(-2);
01562         if(sq_type(key) == OT_NULL) return sq_throwerror(v, _SC("null is not a valid key"));
01563         v->NewSlot(self, key, v->GetUp(-1), bstatic?true:false);
01564         v->Pop(2);
01565     }
01566     return SQ_OK;
01567 }
  
```

References [sq_aux_paramscheck](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.77 sq_newtable()

```
void sq_newtable (
    HSQUIRRELVM v )
```

新しい空のテーブルを作成し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

Squirrel の連想配列であるテーブルオブジェクトを新しく生成し、スタックのトップにプッシュします。初期容量はデフォルト値（通常は 0）になります。多くの要素を追加する予定がある場合は、パフォーマンス向上のため `sq_newtableex` の使用を検討してください。

Definition at line 529 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00530 {
00531     v->Push(SQTable::Create(_ss(v), 0));
00532 }
```

4.1.2.78 sq_newtableex()

```
void sq_newtableex (
    HSQUIRRELVM v,
    SQInteger initialcapacity )
```

指定された初期容量で新しいテーブルを作成し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>initialcapacity</i>	テーブルの初期容量（要素数）。

あらかじめ格納する要素数がある程度わかっている場合にこの関数を使用すると、テーブルの初期リサイズに伴うパフォーマンスの低下を避けることができます。

Definition at line 541 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00542 {
00543     v->Push(SQTable::Create(_ss(v), initialcapacity));
00544 }
```

4.1.2.79 sq_newthread()

```
HSQUIRRELVM sq_newthread (
    HSQUIRRELVM friendvm,
    SQInteger initialstacksize )
```

既存のVM と共有状態を共有する新しいスレッド（軽量VM）を作成します。

Parameters

<i>friendvm</i>	新しいスレッドが共有状態を共有する既存のVM（フレンドVM）。
<i>initialstacksize</i>	新しいスレッドの初期スタックサイズ。

Returns

成功した場合は新しいスレッドVM へのハンドル。失敗した場合はNULL。

スレッドはコルーチンの実装に使用される軽量なVM です。これは *friendvm* とメモリ空間やグローバルな状態を共有しますが、独立した実行スタックと状態を持ちます。成功すると、新しく作成されたスレッドオブジェクトが *friendvm* のスタックにプッシュされます。返されたハンドルは、独立したVM として *sq_call* などで使用できます。

Definition at line 111 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00112 {
00113     SQSharedState *ss;
00114     SQVM *v;
00115     ss=_ss(friendvm);
00116
00117     v= (SQVM *)SQ_MALLOC(sizeof(SQVM));
00118     new (v) SQVM(ss);
00119
00120     if(v->Init(friendvm, initialstacksize)) {
00121         friendvm->Push(v);
00122         return v;
00123     } else {
00124         sq_delete(v, SQVM);
00125         return NULL;
00126     }
00127 }
```

4.1.2.80 sq_newuserdata()

```
SQUserPointer sq_newuserdata (
    HSQUIRRELVM v,
    SQUnsignedInteger size )
```

新しいユーザーデータを作成し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>size</i>	確保するメモリ領域のサイズ（バイト単位）。

Returns

確保されたメモリ領域へのポインタ。

ユーザーデータは、Squirrel VM によってガベージコレクトされる管理対象のメモリブロックです。sq_setdelegate でデリゲートを設定してメタメソッドを持たせたり、sq_setreleasehook で解放時の処理を定義したりできます。C 側で管理したい任意のデータ構造を格納するために使用します。返されるポインタは確保された領域の先頭を指します。

Definition at line 516 of file squirrel3_squirrel_sqapiByGemini.cpp.

```
00517 {
00518     SQUserData *ud = SQUserData::Create(_ss(v), size + SQ_ALIGNMENT);
00519     v->Push(ud);
00520     return (SQUserPointer)sq_aligning(ud + 1);
00521 }
```

4.1.2.81 sq_next()

```
SQRESULT sq_next (
    HSQUIRRELVm v,
    SQInteger idx )
```

コンテナ（テーブル、配列など）のイテレーションを1ステップ進めます。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	イテレートするコンテナオブジェクトのスタックインデックス。

Returns

イテレーションが続く場合はSQ_OK、終了した場合はSQ_ERROR。

C 側で foreach ループを実装するために使用します。呼び出す前に、イテレータ（初回は null）をスタックトップにプッシュします。成功すると、イテレータはポップされ、新しいキーと値がスタックにプッシュされます。イテレーションが終了すると SQ_ERROR を返し、スタックは変更されません。

Definition at line 2591 of file squirrel3_squirrel_sqapiByGemini.cpp.

```
02592 {
02593     SQObjectPtr o=stack_get(v,idx), &refpos = stack_get(v,-1), realkey, val;
02594     if(sq_type(o) == OT_GENERATOR) {
02595         return sq_throwerror(v, _SC("cannot iterate a generator"));
02596     }
02597     int faketojump;
02598     if(!v->FOREACH_OP(o, realkey, val, refpos, 0, 666, faketojump))
02599         return SQ_ERROR;
02600     if(faketojump != 666) {
02601         v->Push(realkey);
02602         v->Push(val);
02603         return SQ_OK;
02604     }
02605     return SQ_ERROR;
02606 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.82 sq_notifyallexceptions()

```
void sq_notifyallexceptions (
    HSQUIRRELVm v,
    SQBool enable )
```

すべての例外を通知するかどうかを設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>enable</i>	true で有効、false で無効。

この機能が有効な場合、try...catchで捕捉された例外でもデバッグフックがトリガーされます。デフォルトでは、捕捉されなかった (unhandled) 例外のみがデバッグフックに通知されます。この設定は共有状態に影響します。

Definition at line 272 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00273 {
00274     _ss(v)->_notifyallexceptions = enable?true:false;
00275 }
```

4.1.2.83 sq_objtobool()

```
SQBool sq_objtobool (
    const HSQOBJECT * o )
```

オブジェクトがブール型の場合、そのブール値を取得します。

Parameters

<i>o</i>	対象のオブジェクトへのポインタ。
----------	------------------

Returns

オブジェクトがブール値であればその値 (SQTrue/SQFalse)、そうでなければSQFalse。

sq_objtostringと同様に、HSQOBJECT 構造体から直接値を取得します。対象がブール型でない場合は常にSQFalseを返します。

Definition at line 401 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00402 {
00403     if(sq_isbool(*o)) {
00404         return _integer(*o);
00405     }
00406     return SQFalse;
00407 }
```

4.1.2.84 sq_objtfloat()

```
SQFloat sq_objtfloat (
    const HSQOBJECT * o )
```

オブジェクトが数値型の場合、その浮動小数点数値を取得します。

Parameters

<i>o</i>	対象のオブジェクトへのポインタ。
----------	------------------

Returns

オブジェクトが数値であればその浮動小数点数値、そうでなければ 0.0。

`sq_objtostring` と同様に、`HSQOBJECT` 構造体から直接値を取得します。整数も浮動小数点数に変換されます。

Definition at line 386 of file `squirrel3_squirrel_sqapiByGemini.cpp`.

```
00387 {
00388     if(sq_isnumeric(*o)) {
00389         return tofloat(*o);
00390     }
00391     return 0;
00392 }
```

4.1.2.85 sq_objtointeger()

```
SQInteger sq_objtointeger (
    const HSQOBJECT * o )
```

オブジェクトが数値型の場合、その整数値を取得します。

Parameters

<i>o</i>	対象のオブジェクトへのポインタ。
----------	------------------

Returns

オブジェクトが数値であればその整数値、そうでなければ 0。

`sq_objtostring` と同様に、`HSQOBJECT` 構造体から直接値を取得します。浮動小数点数は切り捨てられます。

Definition at line 371 of file `squirrel3_squirrel_sqapiByGemini.cpp`.

```
00372 {
00373     if(sq_isnumeric(*o)) {
00374         return tointeger(*o);
00375     }
00376     return 0;
00377 }
```

4.1.2.86 sq_objtostring()

```
const SQChar * sq_objtostring (
    const HSQOBJECT * o )
```

オブジェクトが文字列型の場合、そのC文字列ポインタを取得します。

Parameters

<i>o</i>	対象のオブジェクトへのポインタ。
----------	------------------

Returns

オブジェクトが文字列であればその内容のC 文字列ポインタ、そうでなければNULL。

この関数はスタック上のオブジェクトではなく、HSQOBJECT 構造体から直接値を取得します。高速なアクセスが必要な場合や、スタック操作を避けたい場合に使用します。

Definition at line 356 of file squirrel3_squirrel_sqapiByGemini.cpp.

```
00357 {
00358     if(sq_type(*o) == OT_STRING) {
00359         return _stringval(*o);
00360     }
00361     return NULL;
00362 }
```

4.1.2.87 sq_objtouserpointer()

```
SQUserPointer sq_objtouserpointer (
    const HSQOBJECT * o )
```

オブジェクトがユーザーポインタ型の場合、その値を取得します。

Parameters

<i>o</i>	対象のオブジェクトへのポインタ。
----------	------------------

Returns

オブジェクトがユーザーポインタであればそのポインタ値、そうでなければNULL。

sq_objtostring と同様に、HSQOBJECT 構造体から直接値を取得します。

Definition at line 415 of file squirrel3_squirrel_sqapiByGemini.cpp.

```
00416 {
00417     if(sq_isuserpointer(*o)) {
00418         return _userpointer(*o);
00419     }
00420     return 0;
00421 }
```

4.1.2.88 sq_open()

```
HSQUIRRELVN sq_open (
    SQInteger initialstacksize )
```

新しいSquirrel VM インスタンスを作成し、開きます。

Parameters

<i>initialstacksize</i>	VM の初期スタックサイズ。
-------------------------	----------------

Returns

成功した場合は新しいVM へのハンドル。失敗した場合はNULL。

この関数は、Squirrel スクリプトを実行するための主要なコンテナである新しいVM を作成します。全ての Squirrel API を利用する上で、最初に呼び出すべき関数です。initialstacksize はVM の初期コールスタックのサイズを指定します。成功すると、ルートVM のハンドルが返されます。このハンドルは sq_close で解放する必要があります。

Definition at line 84 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00085 {
00086     SQSharedState *ss;
00087     SQVM *v;
00088     sq_new(ss, SQSharedState);
00089     ss->Init();
00090     v = (SQVM *)SQ_MALLOC(sizeof(SQVM));
00091     new (v) SQVM(ss);
00092     ss->_root_vm = v;
00093     if(v->Init(NULL, initialstacksize)) {
00094         return v;
00095     } else {
00096         sq_delete(v, SQVM);
00097         return NULL;
00098     }
00099     return v;
00100 }
```

4.1.2.89 sq_pop()

```
void sq_pop (
    HSQUIRRELVm v,
    SQInteger nelemstopop )
```

スタックトップから指定された数の要素をポップします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>nelemstopop</i>	ポップする要素の数。

スタックトップから nelemstopop 個の要素を削除し、スタックトップを更新します。関数からの戻り値や不要になった一時的な値をスタックから取り除くために使用します。

Definition at line 1506 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01507 {
01508     assert(v->_top >= nelemstopop);
01509     v->Pop(nelemstopop);
01510 }
```

Referenced by [sq_settop\(\)](#).

Here is the caller graph for this function:



4.1.2.90 sq_poptop()

```
void sq_poptop (
    HSQUIRRELVM v )
```

スタックトップの要素を1つポップします。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

`sq_pop(v, 1)` と等価です。スタックトップの単一の要素を削除します。

Definition at line 1517 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01518 {
01519     assert(v->_top >= 1);
01520     v->Pop();
01521 }
```

4.1.2.91 sq_push()

```
void sq_push (
    HSQUIRRELVM v,
    SQInteger idx )
```

指定されたインデックスのスタック上のオブジェクトをスタックトップにコピー（プッシュ）します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	コピー元のスタックインデックス。

指定された `idx` にあるオブジェクトのコピーをスタックのトップにプッシュします。元のオブジェクトは `idx` の位置にそのまま残ります。スタックトップは1つ増加します。

Definition at line 1093 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01094 {
01095     v->Push(stack_get(v, idx));
01096 }
```

4.1.2.92 sq_pushbool()

```
void sq_pushbool (
    HSQUIRRELVM v,
    SQBool b )
```

スタックにブール値をプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>b</i>	プッシュするブール値 (true または false)。

ブール型のSquirrel オブジェクトを生成し、スタックのトップにプッシュします。

Definition at line 467 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00468 {
00469     v->Push(b?true:false);
00470 }
```

4.1.2.93 sq_pushconsttable()

```
void sq_pushconsttable (
    HSQUIRRELVM v )
```

定数テーブルをスタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

定数テーブルには、Squirrel スクリプト内で `const` キーワードで宣言された定数が格納されます。通常、このテーブルは読み取り専用として扱います。

Definition at line 959 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00960 {
00961     v->Push(_ss(v)->_consts);
00962 }
```

4.1.2.94 sq_pushfloat()

```
void sq_pushfloat (
    HSQUIRRELVM v,
    SQFloat n )
```

スタックに浮動小数点数値をプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>n</i>	プッシュする浮動小数点数。

浮動小数点数型のSquirrel オブジェクトを生成し、スタックのトップにプッシュします。

Definition at line 478 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00479 {
00480     v->Push(n);
00481 }
```

4.1.2.95 sq_pushinteger()

```
void sq_pushinteger (
    HSQUIRRELVM v,
    SQInteger n )
```

スタックに整数値をプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>n</i>	プッシュする整数。

整数型のSquirrel オブジェクトを生成し、スタックのトップにプッシュします。

Definition at line 456 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00457 {
00458     v->Push(n);
00459 }
```

4.1.2.96 sq_pushnull()

```
void sq_pushnull (
    HSQUIRRELVm v )
```

スタックに null 値をプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

VM のスタックのトップに null 型のオブジェクトを一つ追加します。これによりスタックのトップポイントが一つ増加します。Squirrel 言語における null 値を C API 側から生成する際に使用します。

Definition at line 429 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00430 {
00431     v->PushNull();
00432 }
```

Referenced by [sq_settop\(\)](#).

Here is the caller graph for this function:



4.1.2.97 sq_pushobject()

```
void sq_pushobject (
    HSQUIRRELVm v,
    HSQOBJECT obj )
```

HSQOBJECT ハンドルが指すオブジェクトをスタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>obj</i>	プッシュするオブジェクトのハンドル。

`sq_getstackobj` で取得し、`sq_addrref` で参照を保持しているオブジェクトを、後でスタックに戻す際などに使用します。

Definition at line 1902 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01903 {
01904     v->Push(SQObjectPtr(obj));
01905 }
```

4.1.2.98 sq_pushregistrytable()

```
void sq_pushregistrytable (
    HSQUIRRELVM v )
```

レジストリテーブルをスタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

レジストリテーブルは、VM の生存期間中にC コードがSquirrel の値を永続的に格納するために使用できるグローバルなテーブルです。ガベージコレクタから保護したいオブジェクトを格納するのに便利です。全てのVM で共有されます。

Definition at line 949 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00950 {
00951     v->Push(_ss(v)->_registry);
00952 }
```

4.1.2.99 sq_pushroottable()

```
void sq_pushroottable (
    HSQUIRRELVM v )
```

VM の現在のルートテーブルをスタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

ルートテーブルは、グローバル変数の検索スコープの基点となるテーブルです。この関数で取得したテーブルに新しいスロットを追加することで、グローバル変数をC 側から定義できます。

Definition at line 939 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00940 {
00941     v->Push(v->_roottable);
00942 }
```

4.1.2.100 sq_pushstring()

```
void sq_pushstring (
    HSQUIRRELVM v,
    const SQChar * s,
    SQInteger len )
```

スタックに文字列をプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>s</i>	プッシュするC 文字列。
<i>len</i>	文字列の長さ。-1 の場合、文字列は null 終端されていると見なされます。

指定されたC 文字列から新しいSquirrel 文字列オブジェクトを生成し、それをスタックのトップにプッシュします。VM は文字列のコピーを内部で管理するため、この関数呼び出し後に元の *s* のバッファを解放しても問題ありません。*s* がNULL の場合は、代わりに null オブジェクトがプッシュされます。

Definition at line 443 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00444 {
00445     if (s)
00446         v->Push(SQObjectPtr(SQString::Create(_ss(v), s, len)));
00447     else v->PushNull();
00448 }
```

4.1.2.101 sq_pushthread()

```
void sq_pushthread (
    HSQUIRRELVM v,
    HSQUIRRELVM thread )
```

スタックにスレッドオブジェクトをプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>thread</i>	プッシュするスレッドVM のハンドル。

指定されたスレッド（コルーチン）オブジェクトをスタックのトップにプッシュします。これは、例えば他のコルーチンに現在のコルーチンを渡すような場合に使用されます。

Definition at line 502 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00503 {
00504     v->Push(thread);
00505 }
```

4.1.2.102 sq_pushuserpointer()

```
void sq_pushuserpointer (
    HSQUIRRELVM v,
    SQUserPointer p )
```

スタックにユーザーポインタをプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>p</i>	プッシュするユーザーポインタ。

ユーザーポインタは、Squirrel からC/C++の任意のデータを参照するための軽量な方法です。VMはこのポインタの内容を解釈せず、単に値を保持するだけです。ガベージコレクションの対象にはなりません。

Definition at line 490 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00491 {
00492     v->Push(p);
00493 }
```

4.1.2.103 sq_rawdeleteslot()

```
SQRESULT sq_rawdeleteslot (
    HSQUIRRELVm v,
    SQInteger idx,
    SQBool pushval )
```

テーブルからスロットを直接削除します（メタメソッドを無視します）。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	テーブルのスタックインデックス。
<i>pushval</i>	true の場合、削除されたスロットの値をスタックにプッシュします。

Returns

常にSQ_OK。

この関数を呼び出す前に、削除するスロットのキーをスタックトップにプッシュしておく必要があります。この操作は `_deleteslot` メタメソッドを ****無視**** します。pushval が true の場合、削除された値（存在しない場合は null）がスタックにプッシュされます。キーが存在しない場合でもエラーにはなりません。

Definition at line 1746 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01747 {
01748     sq_aux_paramscheck(v, 2);
01749     SQObjectPtr *self;
01750     _GETSAFE_OBJ(v, idx, OT_TABLE, self);
01751     SQObjectPtr &key = v->GetUp(-1);
01752     SQObjectPtr t;
01753     if(_table(*self)->Get(key,t)) {
01754         _table(*self)->Remove(key);
01755     }
01756     if(pushval != 0)
01757         v->GetUp(-1) = t;
01758     else
01759         v->Pop();
01760     return SQ_OK;
01761 }
```

References [_GETSAFE_OBJ](#), and [sq_aux_paramscheck](#).

4.1.2.104 sq_rawget()

```
SQRESULT sq_rawget (
    HSQUIRRELVm v,
    SQInteger idx )
```

オブジェクトからキーに対応する値を直接取得します（メタメソッドを無視します）。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

オブジェクトからキーに対応する値を直接取得します。この操作は_get メタメソッドやデリゲートチェーンを**無視**します。パフォーマンスが重要な場合やメタメソッドの動作を意図的に避けたい場合に使用します。呼び出す前に、取得したい値のキーをスタックトップにプッシュしておく必要があります。成功した場合、キーはポップされ、値がプッシュされます。キーが見つからない場合、エラーが返されます。

Definition at line 1812 of file squirrel3_squirrel_sqapiByGemini.cpp.

```
01813 {
01814     SQObjectPtr &self=stack_get(v,idx);
01815     SQObjectPtr &obj = v->GetUp(-1);
01816     switch(sq_type(self)) {
01817     case OT_TABLE:
01818         if(_table(self)->Get(obj,obj))
01819             return SQ_OK;
01820         break;
01821     case OT_CLASS:
01822         if(_class(self)->Get(obj,obj))
01823             return SQ_OK;
01824         break;
01825     case OT_INSTANCE:
01826         if(_instance(self)->Get(obj,obj))
01827             return SQ_OK;
01828         break;
01829     case OT_ARRAY:{
01830         if(sq_isnumeric(obj)){
01831             if(_array(self)->Get(tointeger(obj),obj)) {
01832                 return SQ_OK;
01833             }
01834         }
01835         else {
01836             v->Pop();
01837             return sq_throwerror(v,_SC("invalid index type for an array"));
01838         }
01839     }
01840     break;
01841     default:
01842         v->Pop();
01843         return sq_throwerror(v,_SC("rawget works only on array/table/instance and class"));
01844     }
01845     v->Pop();
01846     return sq_throwerror(v,_SC("the index doesn't exist"));
01847 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.105 sq_rawnewmember()

```

SQRESULT sq_rawnewmember (
    HSQUIRRELVm v,
    SQInteger idx,
    SQBool bstatic )
  
```

クラスに新しいメンバーを直接追加します。メタメソッドを無視します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	クラスのスタックインデックス。
<i>bstatic</i>	静的メンバーとして追加するかどうか。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

クラスに新しいメンバー（フィールドまたはメソッド）を属性付きで直接追加します。この操作は `_newmember` メタメソッドを **** 無視 **** します。メンバーが既に存在する場合、この関数はエラーを返します。クラスの定義をC側から行う際に、パフォーマンスを重視する場合やメタメソッドの介入を避けたい場合に使用します。呼び出す前に、スタックに属性、値、キーの順でプッシュしておく必要があります。成功するとこれら3つの値はポップされます。

Definition at line 1686 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

1687 {
1688     SQObjectPtr &self = stack_get(v, idx);
1689     if(sq_type(self) != OT_CLASS) return sq_throwerror(v, _SC("new member only works with classes"));
1690     SQObjectPtr &key = v->GetUp(-3);
1691     if(sq_type(key) == OT_NULL) return sq_throwerror(v, _SC("null key"));
1692     if(!v->NewSlotA(self, key, v->GetUp(-2), v->GetUp(-1), bstatic?true:false, true)) {
1693         v->Pop(3);
1694         return SQ_ERROR;
1695     }
1696     v->Pop(3);
1697     return SQ_OK;
1698 }
  
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.106 sq_rawset()

```
SQRESULT sq_rawset (
    HSQUIRRELVm v,
    SQInteger idx )
```

オブジェクトのスロットに値を直接設定します（メタメソッドを無視します）。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

この関数を呼び出す前に、スタックに値、その次にキーをプッシュしておく必要があります (スタック: ..., value, key)。_set メタメソッドやデリゲートは無視され、オブジェクト自体のスロットに直接書き込みます。スロットが存在しない場合は新しく作成されます。対象オブジェクトが配列、テーブル、クラス、インスタンスでない場合はエラーになります。成功すると、キーと値はスタックからポップされます。

Definition at line 1618 of file squirrel3_squirrel_sqapiByGemini.cpp.

```

1619 {
1620     SQObjectPtr &self = stack_get(v, idx);
1621     SQObjectPtr &key = v->GetUp(-2);
1622     if(sq_type(key) == OT_NULL) {
1623         v->Pop(2);
1624         return sq_throwerror(v, _SC("null key"));
1625     }
1626     switch(sq_type(self)) {
1627     case OT_TABLE:
1628         _table(self)->NewSlot(key, v->GetUp(-1));
1629         v->Pop(2);
1630         return SQ_OK;
1631     break;
1632     case OT_CLASS:
1633         _class(self)->NewSlot(_ss(v), key, v->GetUp(-1), false);
1634         v->Pop(2);
1635         return SQ_OK;
1636     break;
1637     case OT_INSTANCE:
1638         if(_instance(self)->Set(key, v->GetUp(-1))) {
1639             v->Pop(2);
1640             return SQ_OK;
1641         }
1642     break;
1643     case OT_ARRAY:
1644         if(v->Set(self, key, v->GetUp(-1), false)) {
1645             v->Pop(2);
  
```

```

01646         return SQ_OK;
01647     }
01648     break;
01649     default:
01650         v->Pop(2);
01651         return sq_throwerror(v, _SC("rawset works only on array/table/class and instance"));
01652     }
01653     v->Raise_IdxError(v->GetUp(-2)); return SQ_ERROR;
01654 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.107 sq_readclosure()

```

SQLRESULT sq_readclosure (
    HSQUIRRELVM v,
    SQREADFUNC r,
    SQUserPointer up )

```

シリアライズされたバイトコードを読み込み、クロージャを復元してスタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>r</i>	バイトコードを読み込むためのコールバック関数。
<i>up</i>	<i>r</i> 関数に渡されるユーザーポインタ。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

`sq_writeclosure` でシリアライズされたバイトコードを、指定された読み込み関数 *r* を介して読み込み、クロージャオブジェクトを復元してスタックにプッシュします。これにより、スクリプトの再コンパイルを省略し、アプリケーションの起動時間を短縮できます。ストリームの先頭にあるバイトコードタグを検証し、不正なストリームの場合はエラーを返します。

Definition at line 2172 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

02173 {
02174     SQObjectPtr closure;
02175
02176     unsigned short tag;
02177     if(r(up,&tag,2) != 2)
02178         return sq_throwerror(v,_SC("io error"));
02179     if(tag != SQ_BYTECODE_STREAM_TAG)
02180         return sq_throwerror(v,_SC("invalid stream"));
02181     if(!SQClosure::Load(v,up,r,closure))

```

```

02182         return SQ_ERROR;
02183     v->Push(closure);
02184     return SQ_OK;
02185 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.108 sq_realloc()

```

void * sq_realloc (
    void * p,
    SQUnsignedInteger oldsize,
    SQUnsignedInteger newsize )

```

Squirrel のカスタムアロケータを使用してメモリを再確保します。

Parameters

<i>p</i>	再確保するメモリブロックへのポインタ。
<i>oldsize</i>	古いサイズ (バイト単位)。
<i>newsize</i>	新しいサイズ (バイト単位)。

Returns

再確保されたメモリへのポインタ。

Squirrel の内部実装と同じメモリアロケータ (デフォルトでは標準ライブラリの `realloc`) を C 側から利用するための関数です。

Definition at line 2718 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

02719 {
02720     return SQ_REALLOC(p,oldsize,newsize);
02721 }

```

4.1.2.109 sq_release()

```

SQBool sq_release (
    HSQUIRRELVm v,
    HSQOBJECT * po )

```

Squirrel オブジェクトの参照カウントをデクリメントします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>po</i>	参照カウントを減らすオブジェクトへのポインタ。

Returns

参照カウントが 0 になった場合に `true`、それ以外は `false`。

`sq_addrf` で取得した参照は、必ずこの関数で解放する必要があります。参照カウントが 0 になると、オブジェクトはガベージコレクタによって解放される候補となります。参照カウントされない型に対しては何もしません。

Definition at line 322 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00323 {
00324     if(!ISREFCOUNTED(sq_type(*po))) return SQTrue;
00325 #ifdef NO_GARBAGE_COLLECTOR
00326     bool ret = (po->_unVal.pRefCounted->uiRef <= 1) ? SQTrue : SQFalse;
00327     __Release(po->_type, po->_unVal);
00328     return ret; //the ret val doesn't work(and cannot be fixed)
00329 #else
00330     return _ss(v)->_refs_table.Release(*po);
00331 #endif
00332 }
```

4.1.2.110 `sq_remove()`

```
void sq_remove (
    HSQUIRRELM v,
    SQInteger idx )
```

指定されたインデックスのスタック要素を削除します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	削除する要素のスタックインデックス。

指定された `idx` の要素を削除し、その上にあるすべての要素を一つ下にシフトします。スタックの途中の要素を削除したい場合に使用します。

Definition at line 1530 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01531 {
01532     v->Remove(idx);
01533 }
```

4.1.2.111 `sq_reservestack()`

```
SQRESULT sq_reservestack (
    HSQUIRRELM v,
    SQInteger nsize )
```

VM のスタックサイズが少なくとも指定されたサイズを確保するようにします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>nsiz</i> e	追加で必要となるスタックのスロット数。

Returns

成功した場合はSQ_OK、失敗した（メモリ確保に失敗したか、メタメソッド実行中だった）場合はSQ_ERROR。

一度に多くのオブジェクトをスタックにプッシュする前にこの関数を呼ぶことで、スタック再確保のオーバーヘッドを削減できます。メタメソッドの実行中はスタックのリサイズができないため、エラーが返されます。

Definition at line 1971 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

01972 {
01973     if (((SQUnsignedInteger)v->_top + nsiz)e > v->_stack.size()) {
01974         if (v->_nmetamethodscall) {
01975             return sq_throwerror(v, _SC("cannot resize stack while in a metamethod"));
01976         }
01977         v->_stack.resize(v->_stack.size() + ((v->_top + nsiz)e - v->_stack.size()));
01978     }
01979     return SQ_OK;
01980 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.112 sq_reseterror()

```

void sq_reseterror (
    HSQUIRRELVm v )
```

VM の最後のエラー状態をリセットします。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

VM の `lasterror` を `null` にリセットします。C 側でエラーを捕捉し、処理した後にVM を正常な状態に戻すために呼び出します。

Definition at line 1949 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

01950 {
01951     v->_lasterror.Null();
01952 }

```

4.1.2.113 sq_resetobject()

```

void sq_resetobject (
    HSQOBJECT * po )

```

HSQOBJECT ハンドルをリセットして null 状態にします。

Parameters

<i>po</i>	リセットするHSQOBJECT へのポインタ。
-----------	-------------------------

この関数は、ハンドルが指していたオブジェクトの参照カウントを解放 (sq_release) するわけではなく ** ことに注意してください。単にハンドル構造体自体をクリアするだけです。

Definition at line 1912 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

01913 {
01914     po->_unVal.pUserPointer=NULL;po->_type=OT_NULL;
01915 }

```

4.1.2.114 sq_resume()

```

SQRESULT sq_resume (
    HSQUIRRELVM v,
    SQBool retval,
    SQBool raiseerror )

```

ジェネレータの実行を再開します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>retval</i>	true の場合、ジェネレータからの戻り値をスタックにプッシュします。
<i>raiseerror</i>	実行時エラーが発生した場合にVM にエラーを発生させるかどうか。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

この関数を呼び出す前に、再開するジェネレータオブジェクトをスタックにプッシュしておく必要があります。ジェネレータに値を渡す (yield 式の戻り値として) 場合は、この関数を呼び出す前にその値をプッシュし、ジェネレータはスタックの-2 の位置にある必要があります。retval が true の場合、ジェネレータが yield した値または return した値がスタックにプッシュされます。

Definition at line 1990 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

01991 {
01992     if (sq_type(v->GetUp(-1)) == OT_GENERATOR)
01993     {
01994         v->PushNull(); //retval
01995         if (!v->Execute(v->GetUp(-2), 0, v->_top, v->GetUp(-1), raiseerror,
            SQVM::ET_RESUME_GENERATOR))

```



```

01996         {v->Raise_Error(v->_lasterror); return SQ_ERROR;}
01997         if(!retval)
01998             v->Pop();
01999         return SQ_OK;
02000     }
02001     return sq_throwerror(v,_SC("only generators can be resumed"));
02002 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.115 sq_resurrectunreachable()

```

SQLRESULT sq_resurrectunreachable (
    HSQUIRRELVm v )

```

ガベージコレクタによって到達不可能とマークされたオブジェクトを復活させます。

Parameters

v	対象のSquirrel VM。
---	-----------------

Returns

成功した場合はSQ_OK、GC ビルドでない場合はSQ_ERROR。

ガベージコレクタの「マーク」フェーズ後に、到達不可能と判断されたオブジェクトの中から、リリースフックを持つものを「復活」させます。これは、循環参照によって到達不可能になったが、クリーンアップが必要なオブジェクトのリリースフックを安全に呼び出すためのGC 内部メカニズムの一部です。通常、アプリケーション開発者がこの関数を直接呼び出す必要はありません。

Definition at line 2205 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

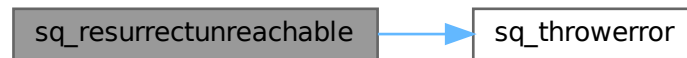
```

02206 {
02207     #ifndef NO_GARBAGE_COLLECTOR
02208         _ss(v)->ResurrectUnreachable(v);
02209         return SQ_OK;
02210     #else
02211         return sq_throwerror(v,_SC("sq_resurrectunreachable requires a garbage collector build"));
02212     #endif
02213 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.116 sq_set()

```

SQRESULT sq_set (
    HSQUIRRELVm v,
    SQInteger idx )
  
```

オブジェクトのスロットに値を設定します（メタメソッドを考慮します）。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

この関数を呼び出す前に、スタックに値、その次にキーをプッシュしておく必要があります (スタック: ..., value, key)。スロットが存在しない場合は新しく作成されます。この関数は `_set` メタメソッドをトリガーする可能性があります。また、デリゲートチェーンをたどってスロットを探し、更新します。成功すると、キーと値はスタックからポップされます。

Definition at line 1601 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

1602 {
1603     SQObjectPtr &self = stack_get(v, idx);
1604     if (v->Set(self, v->GetUp(-2), v->GetUp(-1), DONT_FALL_BACK)) {
1605         v->Pop(2);
1606         return SQ_OK;
1607     }
1608     return SQ_ERROR;
1609 }
  
```

4.1.2.117 sq_setattributes()

```

SQRESULT sq_setattributes (
    HSQUIRRELVm v,
    SQInteger idx )
  
```

クラスまたはそのメンバーの属性を設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	クラスのスタックインデックス。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

クラスまたはクラスメンバーにメタデータを関連付けるための「属性」を設定します。この関数を呼び出す前に、スタックに属性の値、その次にキーをプッシュしておく必要があります。キーが `null` の場合はクラス自体の属性を操作し、キーがメンバー名の場合はそのメンバーの属性を操作します。属性は通常、テーブルであり、アノテーションや追加情報として利用できます。成功すると、古い属性値がスタックにプッシュされます。

Definition at line 2324 of file `squirrel3_squirrel_sqapiByGemini.cpp`.

```

02325 {
02326     SQObjectPtr *o = NULL;
02327     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
02328     SQObjectPtr &key = stack_get(v,-2);
02329     SQObjectPtr &val = stack_get(v,-1);
02330     SQObjectPtr attrs;
02331     if(sq_type(key) == OT_NULL) {
02332         attrs = _class(*o)->_attributes;
02333         _class(*o)->_attributes = val;
02334         v->Pop(2);
02335         v->Push(attrs);
02336         return SQ_OK;
02337     }else if(_class(*o)->GetAttributes(key,attrs)) {
02338         _class(*o)->SetAttributes(key,val);
02339         v->Pop(2);
02340         v->Push(attrs);
02341         return SQ_OK;
02342     }
02343     return sq_throwerror(v,_SC("wrong index"));
02344 }
```

References `_GETSAFE_OBJ`, and `sq_throwerror()`.

Here is the call graph for this function:



4.1.2.118 sq_setbyhandle()

```

SQLRESULT sq_setbyhandle (
    HSQUIRRELVm v,
    SQInteger idx,
    const HSQMEMBERHANDLE * handle )
```

メンバーハンドルを使用して、クラスまたはインスタンスのメンバーに値を設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	クラスまたはインスタンスのスタックインデックス。
<i>handle</i>	sq_getmemberhandle で取得したハンドル。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

sq_getmemberhandle で事前に取得したハンドルを使い、高速にメンバーの値を設定します。この関数を呼び出す前に、設定する値をスタックトップにプッシュしておく必要があります。設定後、その値はスタックからポップされます。

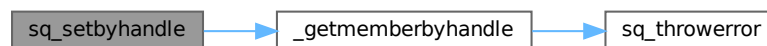
Definition at line 2464 of file squirrel3_squirrel_sqapiByGemini.cpp.

```

02465 {
02466     SQObjectPtr &self = stack_get(v, idx);
02467     SQObjectPtr &newval = stack_get(v, -1);
02468     SQObjectPtr *val = NULL;
02469     if (SQ_FAILED(_getmemberbyhandle(v, self, handle, val))) {
02470         return SQ_ERROR;
02471     }
02472     *val = newval;
02473     v->Pop();
02474     return SQ_OK;
02475 }
```

References [_getmemberbyhandle\(\)](#).

Here is the call graph for this function:



4.1.2.119 sq_setclassudsize()

```

SQLRESULT sq_setclassudsize (
    HSQUIRRELVM v,
    SQInteger idx,
    SQInteger udsiz )
```

クラスに追加のユーザーデータ領域のサイズを設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	クラスのスタックインデックス。
<i>udsize</i>	設定するユーザーデータ領域のサイズ (バイト単位)。

Returns

成功した場合はSQ_OK、対象がクラスでないか、クラスがロックされている場合はSQ_ERROR。

この関数でサイズを設定すると、そのクラスから生成される各インスタンスは、指定されたサイズのメモリブロックを内部に持つようになります。このメモリ領域は、C/C++側のデータをインスタンス自体に埋め込むために使用できます。この関数はクラスがロックされる（例：最初のインスタンスが生成される）前に呼び出す必要があります。

Definition at line 1438 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01439 {
01440     SQObjectPtr &o = stack_get(v,idx);
01441     if(sq_type(o) != OT_CLASS) return sq_throwerror(v,_SC("the object is not a class"));
01442     if(_class(o)->_locked) return sq_throwerror(v,_SC("the class is locked"));
01443     _class(o)->_udsize = udsiz;
01444     return SQ_OK;
01445 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:

**4.1.2.120 sq_setclosureroot()**

```
SQRESULT sq_setclosureroot (
    HSQUIRRELVm v,
    SQInteger idx )
```

スクリプトクロージャのルートテーブルを設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	クロージャが格納されているスタックインデックス。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

指定された *idx* にあるスクリプトクロージャのルートテーブルを、スタックトップのテーブルに設定します。ルートテーブルは自由変数が見つからなかった場合のフォールバックとして検索されます。これにより、クロージャが実行される際のグローバル変数のスコープを動的に変更できます。設定後、スタックトップのテーブルはポップされます。

Definition at line 885 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

00886 {
00887     SQObjectPtr &c = stack_get(v, idx);
00888     SQObject o = stack_get(v, -1);
00889     if(!sq_isclosure(c)) return sq_throwerror(v, _SC("closure expected"));
00890     if(sq_istable(o)) {
00891         _closure(c)->SetRoot(_table(o)->GetWeakRef(OT_TABLE));
00892         v->Pop();
00893         return SQ_OK;
00894     }
00895     return sq_throwerror(v, _SC("invalid type"));
00896 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.121 sq_setcompilererrorhandler()

```

void sq_setcompilererrorhandler (
    HSQUIRRELVm v,
    SQCOMPILERERROR f )

```

コンパイラのエラーハンドラを設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>f</i>	新しいコンパイラエラーハンドラ関数へのポインタ。

`sq_compile` や `sq_compilebuffer` の実行中にコンパイルエラーが発生した際に呼び出される、グローバルなエラーハンドラ関数を設定します。デフォルトでは、コンパイラエラーはVMのエラーとしてスローされますが、この関数でカスタムハンドラを設定することで、エラーの報告方法（例：ファイルへのログ出力、IDE への通知など）をカスタマイズできます。

Definition at line 2137 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

02138 {
02139     _ss(v)->_compilererrorhandler = f;
02140 }

```

4.1.2.122 sq_setconsttable()

```

SQRRESULT sq_setconsttable (
    HSQUIRRELVm v )

```

VM の定数テーブルを設定します。

Parameters

<code>v</code>	対象のSquirrel VM。
----------------	-----------------

Returns

成功した場合はSQ_OK、スタックトップがテーブルでない場合はSQ_ERROR。

この関数を呼び出す前に、新しい定数テーブルをスタックトップにプッシュしておく必要があります。この設定は共有状態に影響するため、関連する全てのVMに影響します。設定後、スタックトップのオブジェクトはポップされます。

Definition at line 987 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00988 {
00989     SQObject o = stack_get(v, -1);
00990     if(sq_istable(o)) {
00991         _ss(v)->_consts = o;
00992         v->Pop();
00993         return SQ_OK;
00994     }
00995     return sq_throwerror(v, _SC("invalid type, expected table"));
00996 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.123 sq_setdebughook()

```
void sq_setdebughook (
    HSQUIRRELVm v )
```

Squirrel スクリプトで実装されたデバッグフックを設定します。

Parameters

<code>v</code>	対象のSquirrel VM。
----------------	-----------------

スタックのトップにあるクロージャをデバッグフックとして設定します。sq_setnatividebughook と同様に、VM の実行中に特定のイベントで呼び出されます。スタックトップのオブジェクトがNULL の場合、デバッグフックは無効になります。この関数を呼び出すと、ネイティブのデバッグフックは無効化されます。設定後、スタックトップのオブジェクトはポップされます。

Definition at line 189 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

00190 {
00191     SQObject o = stack_get(v,-1);
00192     if(sq_isclosure(o) || sq_isnativeclosure(o) || sq_isnull(o)) {
00193         v->_debughook_closure = o;
00194         v->_debughook_native = NULL;
00195         v->_debughook = !sq_isnull(o);
00196         v->Pop();
00197     }
00198 }

```

4.1.2.124 sq_setdelegate()

```

SQRESULT sq_setdelegate (
    HSQUIRRELVM v,
    SQInteger idx )

```

テーブルまたはユーザーデータのデリゲートを設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	テーブルまたはユーザーデータのスタックインデックス。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

デリゲートは、元のオブジェクトにスロットが見つからない場合に検索されるフォールバックオブジェクトです。これにより、継承に似た動作を実現できます。呼び出す前に、デリゲートとして設定するテーブル（または null でデリゲートを解除）をスタックトップにプッシュしておく必要があります。循環参照になるようなデリゲート設定はエラーになります。設定後、デリゲートオブジェクトはスタックからポップされます。

Definition at line 1707 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

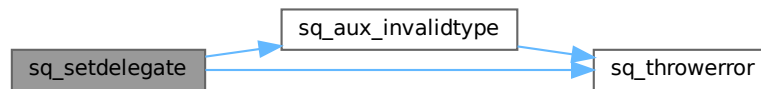
```

01708 {
01709     SQObjectPtr &self = stack_get(v, idx);
01710     SQObjectPtr &mt = v->GetUp(-1);
01711     SQObjectType type = sq_type(self);
01712     switch(type) {
01713     case OT_TABLE:
01714         if(sq_type(mt) == OT_TABLE) {
01715             if(!_table(self)->SetDelegate(_table(mt))) {
01716                 return sq_throwerror(v, _SC("delegate cycle"));
01717             }
01718             v->Pop();
01719         }
01720         else if(sq_type(mt)==OT_NULL) {
01721             _table(self)->SetDelegate(NULL); v->Pop(); }
01722         else return sq_aux_invalidtype(v,type);
01723         break;
01724     case OT_USERDATA:
01725         if(sq_type(mt)==OT_TABLE) {
01726             _userdata(self)->SetDelegate(_table(mt)); v->Pop(); }
01727         else if(sq_type(mt)==OT_NULL) {
01728             _userdata(self)->SetDelegate(NULL); v->Pop(); }
01729         else return sq_aux_invalidtype(v, type);
01730         break;
01731     default:
01732         return sq_aux_invalidtype(v, type);
01733         break;
01734     }
01735     return SQ_OK;
01736 }

```

References [sq_aux_invalidtype\(\)](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.125 sq_seterrorhandler()

```
void sq_seterrorhandler (
    HSQUIRRELVm v )
```

VM のグローバルエラーハンドラを設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

スタックのトップにあるクロージャまたはネイティブクロージャをエラーハンドラとして設定します。コンパイルエラーや実行時エラーが発生し、それが捕捉されなかった場合にこのハンドラが呼び出されます。ハンドラにはエラーオブジェクトが引数として渡されます。NULL を設定すると、エラーハンドラは削除されます。設定後、スタックトップのオブジェクトはポップされます。

Definition at line 156 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00157 {
00158     SQObject o = stack_get(v, -1);
00159     if(sq_isclosure(o) || sq_isnativeclosure(o) || sq_isnull(o)) {
00160         v->_errorhandler = o;
00161         v->Pop();
00162     }
00163 }
```

4.1.2.126 sq_setforeignptr()

```
void sq_setforeignptr (
    HSQUIRRELVm v,
    SQUserPointer p )
```

VM に外部のポインタ (foreign pointer) を設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>p</i>	設定するユーザーポインタ。

このポインタはVM に紐付けられ、ホストアプリケーションが任意のデータをVM インスタンスに関連付けるために使用できます。Squirrel 自体はこのポインタを使用しません。各VM インスタンスは独立した外部ポインタを持つことができます。

Definition at line 1004 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01005 {
01006     v->_foreignptr = p;
01007 }
```

4.1.2.127 sq_setfreevariable()

```
SQRESULT sq_setfreevariable (
    HSQUIRRELVm v,
    SQInteger idx,
    SQUnsignedInteger nval )
```

指定されたインデックスのクロージャの自由変数を設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	クロージャのスタックインデックス。
<i>nval</i>	設定する自由変数のインデックス (0 から始まる)。

Returns

成功した場合はSQ_OK、失敗した（インデックスが範囲外など）場合はSQ_ERROR。

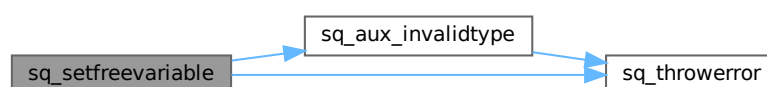
クロージャがキャプチャした自由変数の値を、スタックトップの値で更新します。この関数を呼び出す前に、設定する値をスタックトップにプッシュしておく必要があります。設定後、その値はスタックからポップされます。

Definition at line 2291 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02292 {
02293     SQObjectPtr &self=stack_get(v,idx);
02294     switch(sq_type(self))
02295     {
02296     case OT_CLOSURE:{
02297         SQFunctionProto *fp = _closure(self)->_function;
02298         if(((SQUnsignedInteger)fp->noutervalues) > nval){
02299             *(_outer(_closure(self)->outervalues[nval])->_valptr) = stack_get(v,-1);
02300         }
02301         else return sq_throwerror(v,_SC("invalid free var index"));
02302     }
02303     break;
02304     case OT_NATIVECLOSURE:
02305         if(_nativeclosure(self)->noutervalues > nval){
02306             _nativeclosure(self)->outervalues[nval] = stack_get(v,-1);
02307         }
02308         else return sq_throwerror(v,_SC("invalid free var index"));
02309     break;
02310     default:
02311         return sq_aux_invalidtype(v, sq_type(self));
02312     }
02313     v->Pop();
02314     return SQ_OK;
02315 }
```

References [sq_aux_invalidtype\(\)](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.128 sq_setinstanceup()

```
SQRESULT sq_setinstanceup (
    HSQUIRRELVM v,
    SQInteger idx,
    SQUserPointer p )
```

クラスインスタンスにユーザーポインタを設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	インスタンスのスタックインデックス。
<i>p</i>	設定するユーザーポインタ。

Returns

対象がインスタンスであればSQ_OK、そうでなければSQ_ERROR。

これは、C/C++側のデータをSquirrel のインスタンスに直接関連付けるための便利な方法です。このポインタはSquirrel からは直接アクセスできず、C API を介してのみ操作可能です。このポインタはインスタンスの `_userpointer` メンバーに格納されます。

Definition at line 1422 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01423 {
01424     SQObjectPtr &o = stack_get(v,idx);
01425     if(sq_type(o) != OT_INSTANCE) return sq_throwerror(v,_SC("the object is not a class instance"));
01426     _instance(o)->_userpointer = p;
01427     return SQ_OK;
01428 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.129 sq_setnativeclosurename()

```
SQRESULT sq_setnativeclosurename (
    HSQUIRRELVM v,
    SQInteger idx,
    const SQChar * name )
```

ネイティブクロージャに名前を設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	ネイティブクロージャが格納されているスタックインデックス。
<i>name</i>	設定する名前。

Returns

成功した場合はSQ_OK、対象がネイティブクロージャでない場合はSQ_ERROR。

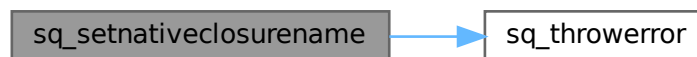
この名前はデバッグ情報（コールスタック）やエラーメッセージに表示されるため、デバッグの際に非常に役立ちます。設定された名前は新しいSquirrel 文字列としてVM に格納されます。

Definition at line 770 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00771 {
00772     SQObject o = stack_get(v, idx);
00773     if(sq_isnativeclosure(o)) {
00774         SQNativeClosure *nc = _nativeclosure(o);
00775         nc->_name = SQString::Create(_ss(v), name);
00776         return SQ_OK;
00777     }
00778     return sq_throwerror(v, _SC("the object is not a nativeclosure"));
00779 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.130 sq_setnativedebughook()

```
void sq_setnativedebughook (
    HSQUIRRELVM v,
    SQDEBUGHOOK hook )
```

ネイティブコード（C/C++）で実装されたデバッグフックを設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>hook</i>	設定するデバッグフック関数へのポインタ。

デバッグフックは、VM の実行中に特定のイベント（行の変更、関数の呼び出し/リターンなど）が発生したときに呼び出されます。これにより、デバッガなどのツールを実装できます。hook にNULL を設定すると、

ネイティブデバッグフックは無効になります。この関数を呼び出すと、Squirrel スクリプトで実装されたデバッグフックは無効化されます。

Definition at line 173 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00174 {
00175     v->_debughook_native = hook;
00176     v->_debughook_closure.Null();
00177     v->_debughook = hook?true:false;
00178 }
```

4.1.2.131 sq_setparamscheck()

```
SQRESULT sq_setparamscheck (
    HSQUIRRELVM v,
    SQInteger nparamscheck,
    const SQChar * typemask )
```

ネイティブクロージャのパラメータ数チェックと型チェックを設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>nparamscheck</i>	期待されるパラメータの数。可変長引数の場合は <code>SQ_MATCHTYPEMASKSTRING</code> を使用します。
<i>typemask</i>	パラメータの型を定義するマスク文字列 (例: ".is n")。NULL の場合は型チェックを行いません。

Returns

成功した場合は `SQ_OK`、失敗した場合は `SQ_ERROR`。

スタックトップにあるネイティブクロージャのパラメータ数と型の検証ルールを設定します。`nparamscheck` に正の値を設定すると引数の数が固定され、負の値を設定すると少なくとも `abs(nparamscheck)` 個の引数が必要であることを示します。`typemask` を使用すると、各パラメータの型を自動的に検証でき、C 関数側での型チェックの記述を削減できます。

Definition at line 789 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00790 {
00791     SQObject o = stack_get(v, -1);
00792     if(!sq_isnativeclosure(o))
00793         return sq_throwerror(v, _SC("native closure expected"));
00794     SQNativeClosure *nc = _nativeclosure(o);
00795     nc->nparamscheck = nparamscheck;
00796     if(typemask) {
00797         SQIntVec res;
00798         if(!CompileTypemask(res, typemask))
00799             return sq_throwerror(v, _SC("invalid typemask"));
00800         nc->_typecheck.copy(res);
00801     }
00802     else {
00803         nc->_typecheck.resize(0);
00804     }
00805     if(nparamscheck == SQ_MATCHTYPEMASKSTRING) {
00806         nc->nparamscheck = nc->_typecheck.size();
00807     }
00808     return SQ_OK;
00809 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.132 sq_setprintfunc()

```
void sq_setprintfunc (
    HSQUIRRELVM v,
    SQPRINTFUNCTION printfunc,
    SQPRINTFUNCTION errfunc )
```

VM の標準出力および標準エラー出力用のプリント関数を設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>printfunc</i>	標準出力用の関数ポインタ。
<i>errfunc</i>	標準エラー出力用の関数ポインタ。

Squirrel の `print()` 関数が呼び出されたときに `printfunc` が使用され、未処理の例外が発生したときに `errfunc` が使用されます。これにより、Squirrel からの出力をホストアプリケーションのログシステムなどにリダイレクトできます。この設定は共有状態に保存され、関連する全てのVMに影響します。

Definition at line 2671 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02672 {
02673     _ss(v)->_printfunc = printfunc;
02674     _ss(v)->_errorfunc = errfunc;
02675 }
```

4.1.2.133 sq_setreleasehook()

```
void sq_setreleasehook (
    HSQUIRRELVM v,
    SQInteger idx,
    SQRELEASEHOOK hook )
```

ユーザーデータ、インスタンス、またはクラスにリリースフックを設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。
<i>hook</i>	オブジェクトがガベージコレクタによって破棄されるときに呼び出されるフック関数。

ユーザーデータ、インスタンス、またはクラスがGC によって破棄される際に呼び出されるコールバック関数（リリースフック）を設定します。このフックは、オブジェクトに関連付けられたC/C++側のリソース（ファイルハンドル、メモリ、ロックなど）を安全に解放するために不可欠です。フック関数には、解放されるオブジェクトのユーザーポインタとサイズが渡されます。

Definition at line 2102 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02103 {
02104     SQObjectPtr &ud=stack_get(v,idx);
02105     switch(sq_type(ud) ) {
02106     case OT_USERDATA:  _userdata(ud)->_hook = hook;      break;
02107     case OT_INSTANCE:  _instance(ud)->_hook = hook;      break;
02108     case OT_CLASS:     _class(ud)->_hook = hook;        break;
02109     default: return;
02110     }
02111 }
```

4.1.2.134 sq_setroottable()

```
SQRESULT sq_setroottable (
    HSQUIRRELVM v )
```

VM の現在のルートテーブルを設定します。

Parameters

v	対象のSquirrel VM。
----------	-----------------

Returns

成功した場合はSQ_OK、スタックトップがテーブルまたは null でない場合はSQ_ERROR。

この関数を呼び出す前に、新しいルートテーブルをスタックトップにプッシュしておく必要があります。これにより、VM 全体のグローバルスコープを動的に切り替えることができます。設定後、スタックトップのオブジェクトはポップされます。

Definition at line 970 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
00971 {
00972     SQObject o = stack_get(v, -1);
00973     if(sq_istable(o) || sq_isnull(o)) {
00974         v->_roottable = o;
00975         v->Pop();
00976         return SQ_OK;
00977     }
00978     return sq_throwerror(v, _SC("invalid type"));
00979 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.135 sq_setsharedforeignptr()

```
void sq_setsharedforeignptr (
    HSQUIRRELVm v,
    SQUserPointer p )
```

VM の共有状態に外部のポインタを設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>p</i>	設定するユーザーポインタ。

このポインタは共有状態 (Shared State) に紐付けられ、同じ共有状態を持つすべてのVM インスタンス (スレッド) からアクセス可能です。アプリケーション全体で共有したいデータへのポインタを設定するのに便利です。

Definition at line 1026 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01027 {
01028     _ss(v)->_foreignptr = p;
01029 }
```

4.1.2.136 sq_setsharedreleasehook()

```
void sq_setsharedreleasehook (
    HSQUIRRELVm v,
    SQRELEASEHOOK hook )
```

共有状態のリリースフックを設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>hook</i>	共有状態が破棄されるときに呼び出されるフック関数。

VM の共有状態が破棄されるときに呼び出されるリリースフックを設定します。このフックは、ルートVM が `sq_close` で閉じられる際に一度だけ呼び出されます。 `sq_setsharedforeignptr` で設定したデータなど、すべてのVM で共有されるリソースをクリーンアップするために使用します。

Definition at line 1070 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01071 {
01072     _ss(v)->_releasehook = hook;
01073 }
```

4.1.2.137 sq_settop()

```
void sq_settop (
    HSQUIRRELVm v,
    SQInteger newtop )
```

スタックトップを指定した位置に設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>newtop</i>	新しいスタックトップのインデックス。

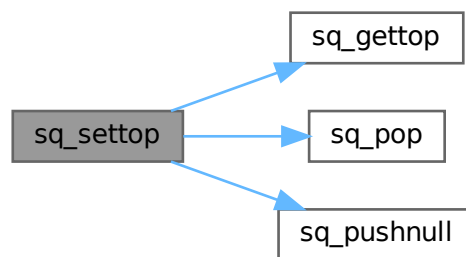
newtop が現在のトップより小さい場合、スタックは切り詰められます（要素がポップされます）。大きい場合、スタックは `null` で埋められて拡張されます。`sq_settop(v, 0)` はスタックを空にする一般的な方法です。

Definition at line 1491 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01492 {  
01493     SQInteger top = sq_gettop(v);  
01494     if(top > newtop)  
01495         sq_pop(v, top - newtop);  
01496     else  
01497         while(top++ < newtop) sq_pushnull(v);  
01498 }
```

References [sq_gettop\(\)](#), [sq_pop\(\)](#), and [sq_pushnull\(\)](#).

Here is the call graph for this function:



4.1.2.138 sq_settypetag()

```
SQRESULT sq_settypetag (  
    HSQUIRRELVm v,  
    SQInteger idx,  
    SQUserPointer typetag )
```

ユーザーデータまたはクラスに型タグを設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	ユーザーデータまたはクラスのスタックインデックス。
<i>typetag</i>	設定する型タグ（任意のポインタ値）。

Returns

成功した場合はSQ_OK、対象が不適切な型の場合はSQ_ERROR。

型タグは、C 側でオブジェクトの種類を安全に識別するために使用できます。例えば、異なるC++クラスのインスタンスを指すユーザーデータを区別するのに役立ちます。VM はこの値を解釈しません。

Definition at line 1353 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01354 {
01355     SQObjectPtr &o = stack_get(v,idx);
01356     switch(sq_type(o)) {
01357         case OT_USERDATA:    _userdata(o)->_typetag = typetag;    break;
01358         case OT_CLASS:       _class(o)->_typetag = typetag;       break;
01359         default:              return sq_throwerror(v,_SC("invalid object type"));
01360     }
01361     return SQ_OK;
01362 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.139 sq_setvmreleasehook()

```
void sq_setvmreleasehook (
    HSQUIRRELVm v,
    SQRELEASEHOOK hook )
```

VM 固有のリリースフックを設定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>hook</i>	VM が破棄されるときに呼び出されるフック関数。

VM インスタンスが破棄されるときに呼び出されるリリースフックを設定します。このフックは、`sq_close` がルートVM に対して呼び出され、このVM インスタンスが破棄される直前に実行されます。主に `sq_setforeignptr` で設定したデータなど、VM インスタンスに固有のリソースを安全にクリーンアップするために使用されます。

Definition at line 1048 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01049 {
01050     v->_releasehook = hook;
01051 }
```

4.1.2.140 sq_suspendvm()

```
SQRESULT sq_suspendvm (
    HSQUIRRELVM v )
```

VM の実行を中断します。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

VM の実行を中断し、サスペンド状態にします。この関数は、ネイティブ関数内から呼び出されることを想定しています。VM は中断した時点の状態を保持し、後で `sq_wakeupvm` を使って実行を再開できます。これは協調的マルチタスクや時間のかかる処理の分割実行などを実装するのに役立ちます。この関数は `SQ_SUSPEND` を返します。ネイティブ関数はこの戻り値をそのまま呼び出し元に返す必要があります。

Definition at line 2059 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02060 {
02061     return v->Suspend();
02062 }
```

4.1.2.141 sq_tailcall()

```
SQRESULT sq_tailcall (
    HSQUIRRELVM v,
    SQInteger nparams )
```

C 関数からSquirrel の関数を末尾呼び出しします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>nparams</i>	呼び出す関数に渡すパラメータの数。

Returns

成功した場合はSQ_TAILCALL_FLAG。失敗した場合はSQ_ERROR。

C 関数からSquirrel 関数への末尾呼び出しを実行します。これにより、C のコールスタックフレームを消費せずにSquirrel 関数に制御を移すことができます。ネイティブ関数が他のSquirrel 関数を呼び出して終了する場合に最適です。この関数が `SQ_TAILCALL_FLAG` を返した場合、呼び出し元のC 関数は直ちに `SQ_TAILCALL_FLAG` を `return` しなければなりません。ジェネレータは末尾呼び出しできません。

Definition at line 2034 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02035 {
02036     SQObjectPtr &res = v->GetUp(-(nparams + 1));
02037     if (sq_type(res) != OT_CLOSURE) {
02038         return sq_throwerror(v, _SC("only closure can be tail called"));
02039     }
02040     SQClosure *clo = _closure(res);
```

```

02041     if (clo->_function->_bgenerator)
02042     {
02043         return sq_throwerror(v, _SC("generators cannot be tail called"));
02044     }
02045
02046     SQInteger stackbase = (v->_top - nparams) - v->_stackbase;
02047     if (!v->TailCall(clo, stackbase, nparams)) {
02048         return SQ_ERROR;
02049     }
02050     return SQ_TAILCALL_FLAG;
02051 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.142 sq_throwerror()

```

SQRESULT sq_throwerror (
    HSQUIRRELVM v,
    const SQChar * err )

```

VM に文字列のエラーを設定し、エラー状態にします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>err</i>	エラーメッセージ文字列。

Returns

常にSQ_ERROR。

指定された文字列をエラーメッセージとしてVM の `lasterror` に設定し、SQ_ERROR を返します。これは C API 関数からSquirrel の例外をスローする標準的な方法です。 `sq_call` などの関数はこのエラーを受け取ってVM の実行を停止します。

Definition at line 1924 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```

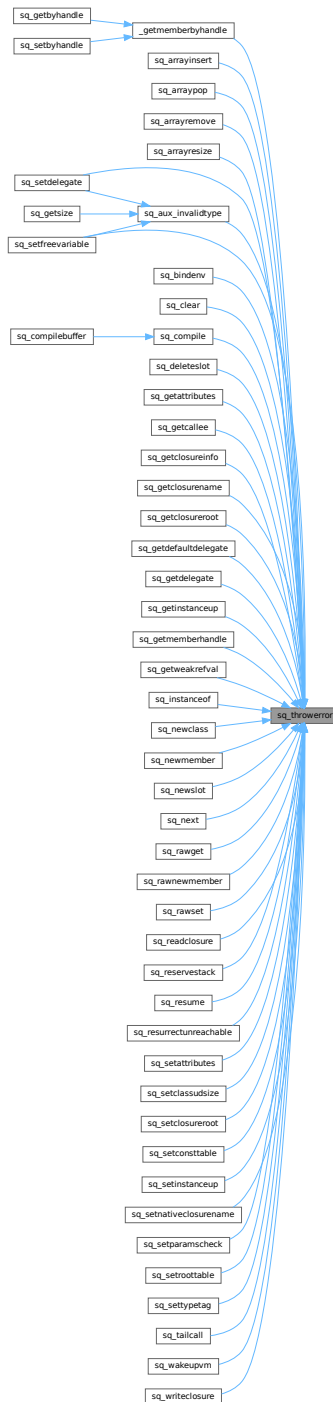
01925 {
01926     v->_lasterror=SQString::Create(_ss(v),err);
01927     return SQ_ERROR;
01928 }

```

Referenced by [_getmemberbyhandle\(\)](#), [sq_arrayinsert\(\)](#), [sq_arraypop\(\)](#), [sq_arrayremove\(\)](#), [sq_arrayresize\(\)](#), [sq_aux_invalidtype\(\)](#), [sq_bindenv\(\)](#), [sq_clear\(\)](#), [sq_compile\(\)](#), [sq_deleteslot\(\)](#), [sq_getattributes\(\)](#), [sq_getcallee\(\)](#), [sq_getclosureinfo\(\)](#), [sq_getclosurename\(\)](#), [sq_getclosureroot\(\)](#), [sq_getdefaultdelegate\(\)](#), [sq_getdelegate\(\)](#), [sq_getinstanceup\(\)](#), [sq_getmemberhandle\(\)](#), [sq_getweakrefval\(\)](#), [sq_instanceof\(\)](#), [sq_newclass\(\)](#), [sq_newmember\(\)](#),

[sq_newslot\(\)](#), [sq_next\(\)](#), [sq_rawget\(\)](#), [sq_rawnewmember\(\)](#), [sq_rawset\(\)](#), [sq_readclosure\(\)](#), [sq_reservestack\(\)](#), [sq_resume\(\)](#), [sq_resurrectunreachable\(\)](#), [sq_setattributes\(\)](#), [sq_setclassudsize\(\)](#), [sq_setclosureroot\(\)](#), [sq_setconsttable\(\)](#), [sq_setdelegate\(\)](#), [sq_setfreevariable\(\)](#), [sq_setinstanceup\(\)](#), [sq_setnativeclosurename\(\)](#), [sq_setparamscheck\(\)](#), [sq_setroottable\(\)](#), [sq_settypetag\(\)](#), [sq_tailcall\(\)](#), [sq_wakeupvm\(\)](#), and [sq_writeclosure\(\)](#).

Here is the caller graph for this function:



4.1.2.143 sq_throwobject()

```
SQRESULT sq_throwobject (
```

```
HSQUIRRELVM v )
```

VM にオブジェクトのエラーを設定し、エラー状態にします。

Parameters

<i>v</i>	対象のSquirrel VM。
----------	-----------------

Returns

常にSQ_ERROR。

スタックトップにある任意のSquirrel オブジェクトをVM の `lasterror` に設定し、SQ_ERROR を返します。これにより、文字列以外のオブジェクト（テーブルやインスタンスなど）を例外としてスローできます。オブジェクトは設定後にスタックからポップされます。

Definition at line 1936 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01937 {
01938     v->_lasterror = v->GetUp(-1);
01939     v->Pop();
01940     return SQ_ERROR;
01941 }
```

4.1.2.144 sq_tobool()

```
void sq_tobool (
    HSQUIRRELVM v,
    SQInteger idx,
    SQBool * b )
```

指定されたインデックスのオブジェクトのブール値としての真偽を判定します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。
<i>b</i>	結果のブール値を格納するポインタ。

Squirrel のルールに従い、`null` と `false` のみが偽 (SQFalse) と見なされ、それ以外 (0, 0.0, 空文字列も含む) はすべて真 (SQTrue) と見なされます。この関数はスタックの状態を変更しません。

Definition at line 1153 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01154 {
01155     SQObjectPtr &o = stack_get(v, idx);
01156     *b = SQVM::IsFalse(o)?SQFalse:SQTrue;
01157 }
```

4.1.2.145 sq_tostring()

```
SQRESULT sq_tostring (
    HSQUIRRELVM v,
    SQInteger idx )
```

指定されたインデックスのオブジェクトを文字列に変換し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

これはSquirrel の `tostring` 関数に相当するAPIです。オブジェクトに `_tostring` メタメソッドが定義されていれば、それが呼び出されます。そうでなければ、デフォルトの文字列表現が生成されます。元のオブジェクトはスタックに残り、変換後の文字列がスタックトップにプッシュされます。

Definition at line 1135 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01136 {
01137     SQObjectPtr &o = stack_get(v, idx);
01138     SQObjectPtr res;
01139     if(!v->ToString(o,res)) {
01140         return SQ_ERROR;
01141     }
01142     v->Push(res);
01143     return SQ_OK;
01144 }
```

4.1.2.146 sq_typeof()

```
SQRESULT sq_typeof (
    HSQUIRRELVm v,
    SQInteger idx )
```

指定されたインデックスのオブジェクトの型名を取得し、文字列としてスタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

指定されたインデックス `idx` のオブジェクトの型名 ("integer", "string" など) を取得し、文字列としてスタックにプッシュします。これはSquirrel 言語の `typeof` 演算子に相当するAPIです。この操作はメタメソッドをトリガーしません。

Definition at line 1117 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
01118 {
01119     SQObjectPtr &o = stack_get(v, idx);
01120     SQObjectPtr res;
01121     if(!v->TypeOf(o,res)) {
01122         return SQ_ERROR;
01123     }
01124     v->Push(res);
01125     return SQ_OK;
01126 }
```

4.1.2.147 sq_wakeupvm()

```
SQRESULT sq_wakeupvm (
    HSQUIRRELVM v,
    SQBool wakeupret,
    SQBool retval,
    SQBool raiseerror,
    SQBool throwerror )
```

中断されたVM の実行を再開します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>wakeupret</i>	true の場合、スタックトップの値を中断した箇所への戻り値として渡します。
<i>retval</i>	true の場合、再開されたVM が終了または再度中断した際の戻り値をスタックにプッシュします。
<i>raiseerror</i>	実行時エラーが発生した場合にVM にエラーを発生させるかどうか。
<i>throwerror</i>	true の場合、wakeupret で渡された値を例外としてスローしてVM を再開します。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

サスペンド状態のVM の実行を再開します。wakeupret が true の場合、スタックトップの値を中断した箇所への戻り値として渡します（例えば suspend() の戻り値になります）。retval が true の場合、再開されたVM が次に終了または再度中断した際の戻り値がスタックにプッシュされます。throwerror を true にすると、wakeupret で渡された値を例外としてスローし、VM を再開できます。

Definition at line 2074 of file squirrel3_squirrel_sqapiByGemini.cpp.

```
02075 {
02076     SQObjectPtr ret;
02077     if(!v->_suspended)
02078         return sq_throwerror(v,_SC("cannot resume a vm that is not running any code"));
02079     SQInteger target = v->_suspended_target;
02080     if(wakeupret) {
02081         if(target != -1) {
02082             v->GetAt(v->_stackbase+v->_suspended_target)=v->GetUp(-1); //retval
02083         }
02084         v->Pop();
02085     } else if(target != -1) { v->GetAt(v->_stackbase+v->_suspended_target).Null(); }
02086     SQObjectPtr dummy;
02087     if(!v->Execute(dummy,-1,-1,ret,raiseerror,throwerror?SQVM::ET_RESUME_THROW_VM :
02088         SQVM::ET_RESUME_VM)) {
02089         return SQ_ERROR;
02090     }
02091     if(retval)
02092         v->Push(ret);
02093     return SQ_OK;
02094 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.148 sq_weakref()

```
void sq_weakref (
    HSQUIRRELVm v,
    SQInteger idx )
```

指定されたインデックスのオブジェクトへの弱い参照（weak reference）を作成し、スタックにプッシュします。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>idx</i>	対象オブジェクトのスタックインデックス。

弱い参照は、オブジェクトの参照カウントを増加させません。そのため、弱い参照がオブジェクトを指している場合、他に強い参照がなければオブジェクトはGCによって回収されます。参照カウントされない型（整数、null など）に対しては、オブジェクト自身がそのままプッシュされます。

Definition at line 2531 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02532 {
02533     SQObject &o=stack_get(v,idx);
02534     if (ISREFCOUNTED(sq_type(o))) {
02535         v->Push(_refcounted(o)->GetWeakRef(sq_type(o)));
02536         return;
02537     }
02538     v->Push(o);
02539 }
```

4.1.2.149 sq_writeclosure()

```
SQRESULT sq_writeclosure (
    HSQUIRRELVm v,
    SQWRITEFUNC w,
    SQUserPointer up )
```

スクリプトクロージャをバイトコードとしてシリアライズし、指定されたライター関数に出力します。

Parameters

<i>v</i>	対象のSquirrel VM。
<i>w</i>	バイトコードを書き込むためのコールバック関数。
<i>up</i>	<i>w</i> 関数に渡されるユーザーポインタ。

Returns

成功した場合はSQ_OK、失敗した場合はSQ_ERROR。

スタックトップにあるスクリプトクロージャを、プラットフォームに依存しないバイトコード形式にシリアライズし、指定された書き込み関数 *w* を介して出力します。これにより、コンパイル済みのスクリプトを保存し、後で `sq_readclosure` を使って高速にロードできます。自由変数（free variable）を持つクロージャは、外部環境への参照を解決できないためシリアライズできません。

Definition at line 2150 of file [squirrel3_squirrel_sqapiByGemini.cpp](#).

```
02151 {
```

```

02152     SQObjectPtr *o = NULL;
02153     _GETSAFE_OBJ(v, -1, OT_CLOSURE,o);
02154     unsigned short tag = SQ_BYTECODE_STREAM_TAG;
02155     if(_closure(*o)->_function->_noutervalues)
02156         return sq_throwerror(v,_SC("a closure with free variables bound cannot be serialized"));
02157     if(w(up,&tag,2) != 2)
02158         return sq_throwerror(v,_SC("io error"));
02159     if(!_closure(*o)->Save(v,up,w))
02160         return SQ_ERROR;
02161     return SQ_OK;
02162 }

```

References [_GETSAFE_OBJ](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.2 squirrel3_squirrel_sqapiByGemini.cpp

[Go to the documentation of this file.](#)

```

00001 //cpp
00002
00003 /*
00004     Written By Gemini
00005 */
00006 #include "sqpcheader.h"
00007 #include "sgvm.h"
00008 #include "sqstring.h"
00009 #include "sqtable.h"
00010 #include "sqarray.h"
00011 #include "sqfuncproto.h"
00012 #include "sqclosure.h"
00013 #include "squserdata.h"
00014 #include "sqcompiler.h"
00015 #include "sqfuncstate.h"
00016 #include "sqclass.h"
00017
00029 static bool sq_aux_gettypedarg(HSQUIRRELV v,SQInteger idx,SQObjectType type,SQObjectPtr **o)
00030 {
00031     *o = &stack_get(v,idx);
00032     if(sq_type(**o) != type){
00033         SQObjectPtr oval = v->PrintObjVal(**o);
00034         v->Raise_Error(_SC("wrong argument type, expected '%s' got
00035 '%.50s'",IdType2Name(type),_stringval(oval)));
00036         return false;
00037     }
00038     return true;
00039 }
00046 #define _GETSAFE_OBJ(v,idx,type,o) { if(!sq_aux_gettypedarg(v,idx,type,&o)) return SQ_ERROR; }
00047
00054 #define sq_aux_paramscheck(v,count) \
00055 { \
00056     if(sq_gettop(v) < count){ v->Raise_Error(_SC("not enough params in the stack")); return SQ_ERROR; } \
00057 }
00058
00068 SQInteger sq_aux_invalidtype(HSQUIRRELV v,SQObjectType type)
00069 {
00070     SQUnsignedInteger buf_size = 100 *sizeof(SQChar);
00071     scsprintf(_ss(v)->GetScratchPad(buf_size), buf_size, _SC("unexpected type %s"),
00072         IdType2Name(type));
00073     return sq_throwerror(v, _ss(v)->GetScratchPad(-1));
00074 }

```

```

00084 HSQUIRRELVm sq_open(SQInteger initialstacksize)
00085 {
00086     SQSharedState *ss;
00087     SQVM *v;
00088     sq_new(ss, SQSharedState);
00089     ss->Init();
00090     v = (SQVM *)SQ_MALLOC(sizeof(SQVM));
00091     new (v) SQVM(ss);
00092     ss->_root_vm = v;
00093     if(v->Init(NULL, initialstacksize)) {
00094         return v;
00095     } else {
00096         sq_delete(v, SQVM);
00097         return NULL;
00098     }
00099     return v;
00100 }
00101
00111 HSQUIRRELVm sq_newthread(HSQUIRRELVm friendvm, SQInteger initialstacksize)
00112 {
00113     SQSharedState *ss;
00114     SQVM *v;
00115     ss=_ss(friendvm);
00116
00117     v= (SQVM *)SQ_MALLOC(sizeof(SQVM));
00118     new (v) SQVM(ss);
00119
00120     if(v->Init(friendvm, initialstacksize)) {
00121         friendvm->Push(v);
00122         return v;
00123     } else {
00124         sq_delete(v, SQVM);
00125         return NULL;
00126     }
00127 }
00128
00138 SQInteger sq_getvmstate(HSQUIRRELVm v)
00139 {
00140     if(v->_suspended)
00141         return SQ_VMSTATE_SUSPENDED;
00142     else {
00143         if(v->_callsstacksize != 0) return SQ_VMSTATE_RUNNING;
00144         else return SQ_VMSTATE_IDLE;
00145     }
00146 }
00147
00156 void sq_seterrorhandler(HSQUIRRELVm v)
00157 {
00158     SQObject o = stack_get(v, -1);
00159     if(sq_isclosure(o) || sq_isnativeclosure(o) || sq_isnull(o)) {
00160         v->_errorhandler = o;
00161         v->Pop();
00162     }
00163 }
00164
00173 void sq_setnatividebughook(HSQUIRRELVm v, SQDEBUGHOOK hook)
00174 {
00175     v->_debughook_native = hook;
00176     v->_debughook_closure.Null();
00177     v->_debughook = hook?true:false;
00178 }
00179
00189 void sq_setdebughook(HSQUIRRELVm v)
00190 {
00191     SQObject o = stack_get(v, -1);
00192     if(sq_isclosure(o) || sq_isnativeclosure(o) || sq_isnull(o)) {
00193         v->_debughook_closure = o;
00194         v->_debughook_native = NULL;
00195         v->_debughook = !sq_isnull(o);
00196         v->Pop();
00197     }
00198 }
00199
00207 void sq_close(HSQUIRRELVm v)
00208 {
00209     SQSharedState *ss = _ss(v);
00210     _thread(ss->_root_vm)->Finalize();
00211     sq_delete(ss, SQSharedState);
00212 }
00213
00220 SQInteger sq_getversion()
00221 {
00222     return SQUIRREL_VERSION_NUMBER;
00223 }
00224
00237 SQRESULT sq_compile(HSQUIRRELVm v, SQLEXREADFUNC read, SQUserPointer p, const SQChar *sourcename, SQBool
    raiseerror)

```

```

00238 {
00239     SQObjectPtr o;
00240 #ifndef NO_COMPILER
00241     if(Compile(v, read, p, sourcename, o, raiseerror?true:false, _ss(v)->_debuginfo)) {
00242         v->Push(SQClosure::Create(_ss(v), _funcproto(o),
00243             _table(v->_roottable)->GetWeakRef(OT_TABLE)));
00244         return SQ_OK;
00245     }
00246     return SQ_ERROR;
00247 #else
00248     return sq_throwerror(v, _SC("this is a no compiler build"));
00249 #endif
00250 }
00251 void sq_enableddebuginfo(HSQIRRELV v, SQBool enable)
00252 {
00253     _ss(v)->_debuginfo = enable?true:false;
00254 }
00255 void sq_notifyallexceptions(HSQIRRELV v, SQBool enable)
00256 {
00257     _ss(v)->_notifyallexceptions = enable?true:false;
00258 }
00259 void sq_addref(HSQIRRELV v, HSQOBJECT *po)
00260 {
00261     if(!ISREFCOUNTED(sq_type(*po))) return;
00262 #ifndef NO_GARBAGE_COLLECTOR
00263     __AddRef(po->_type, po->_unVal);
00264 #else
00265     _ss(v)->_refs_table.AddRef(*po);
00266 #endif
00267 }
00268 SQUnsignedInteger sq_getrefcount(HSQIRRELV v, HSQOBJECT *po)
00269 {
00270     if(!ISREFCOUNTED(sq_type(*po))) return 0;
00271 #ifndef NO_GARBAGE_COLLECTOR
00272     return po->_unVal.pRefCounted->uiRef;
00273 #else
00274     return _ss(v)->_refs_table.GetRefCount(*po);
00275 #endif
00276 }
00277 SQBool sq_release(HSQIRRELV v, HSQOBJECT *po)
00278 {
00279     if(!ISREFCOUNTED(sq_type(*po))) return SQTrue;
00280 #ifndef NO_GARBAGE_COLLECTOR
00281     bool ret = (po->_unVal.pRefCounted->uiRef <= 1) ? SQTrue : SQFalse;
00282     __Release(po->_type, po->_unVal);
00283     return ret; //the ret val doesn't work(and cannot be fixed)
00284 #else
00285     return _ss(v)->_refs_table.Release(*po);
00286 #endif
00287 }
00288 SQUnsignedInteger sq_getvmrefcount(HSQIRRELV SQ_UNUSED_ARG(v), const HSQOBJECT *po)
00289 {
00290     if(!ISREFCOUNTED(sq_type(*po))) return 0;
00291     return po->_unVal.pRefCounted->uiRef;
00292 }
00293 const SQChar *sq_objtstring(const HSQOBJECT *o)
00294 {
00295     if(sq_type(*o) == OT_STRING) {
00296         return _stringval(*o);
00297     }
00298     return NULL;
00299 }
00300 SQInteger sq_objtinteger(const HSQOBJECT *o)
00301 {
00302     if(sq_isnumeric(*o)) {
00303         return tointeger(*o);
00304     }
00305     return 0;
00306 }
00307 SQFloat sq_objtfloat(const HSQOBJECT *o)
00308 {
00309     if(sq_isnumeric(*o)) {
00310         return tofloat(*o);
00311     }
00312     return 0;
00313 }
00314 SQBool sq_objtbool(const HSQOBJECT *o)

```

```

00402 {
00403     if(sq_isbool(*o)) {
00404         return _integer(*o);
00405     }
00406     return SQFalse;
00407 }
00408
00415 SQUserPointer sq_objtouserpointer(const HSQOBJECT *o)
00416 {
00417     if(sq_isuserpointer(*o)) {
00418         return _userpointer(*o);
00419     }
00420     return 0;
00421 }
00422
00429 void sq_pushnull(HSQIRRELM v)
00430 {
00431     v->PushNull();
00432 }
00433
00443 void sq_pushstring(HSQIRRELM v,const SQChar *s,SQInteger len)
00444 {
00445     if(s)
00446         v->Push(SQObjectPtr(SQString::Create(_ss(v), s, len)));
00447     else v->PushNull();
00448 }
00449
00456 void sq_pushinteger(HSQIRRELM v,SQInteger n)
00457 {
00458     v->Push(n);
00459 }
00460
00467 void sq_pushbool(HSQIRRELM v,SQBool b)
00468 {
00469     v->Push(b?true:false);
00470 }
00471
00478 void sq_pushfloat(HSQIRRELM v,SQFloat n)
00479 {
00480     v->Push(n);
00481 }
00482
00490 void sq_pushuserpointer(HSQIRRELM v,SQUserPointer p)
00491 {
00492     v->Push(p);
00493 }
00494
00502 void sq_pushthread(HSQIRRELM v, HSQIRRELM thread)
00503 {
00504     v->Push(thread);
00505 }
00506
00516 SQUserPointer sq_newuserdata(HSQIRRELM v,SQUnsignedInteger size)
00517 {
00518     SQUserData *ud = SQUserData::Create(_ss(v), size + SQ_ALIGNMENT);
00519     v->Push(ud);
00520     return (SQUserPointer)sq_aligning(ud + 1);
00521 }
00522
00529 void sq_newtable(HSQIRRELM v)
00530 {
00531     v->Push(SQTable::Create(_ss(v), 0));
00532 }
00533
00541 void sq_newtableex(HSQIRRELM v,SQInteger initialcapacity)
00542 {
00543     v->Push(SQTable::Create(_ss(v), initialcapacity));
00544 }
00545
00553 void sq_newarray(HSQIRRELM v,SQInteger size)
00554 {
00555     v->Push(SQArray::Create(_ss(v), size));
00556 }
00557
00565 SQRESULT sq_newclass(HSQIRRELM v,SQBool hasbase)
00566 {
00567     SQClass *baseclass = NULL;
00568     if(hasbase) {
00569         SQObjectPtr &base = stack_get(v,-1);
00570         if(sq_type(base) != OT_CLASS)
00571             return sq_throwerror(v,_SC("invalid base type"));
00572         baseclass = _class(base);
00573     }
00574     SQClass *newclass = SQClass::Create(_ss(v), baseclass);
00575     if(baseclass) v->Pop();
00576     v->Push(newclass);
00577     return SQ_OK;

```

```

00578 }
00579
00586 SQBool sq_instanceof(HSQIRRELM v)
00587 {
00588     SQObjectPtr &inst = stack_get(v,-1);
00589     SQObjectPtr &cl = stack_get(v,-2);
00590     if(sq_type(inst) != OT_INSTANCE || sq_type(cl) != OT_CLASS)
00591         return sq_throwerror(v,_SC("invalid param type"));
00592     return _instance(inst)->InstanceOf(_class(cl)) ? SQTrue : SQFalse;
00593 }
00594
00602 SQRESULT sq_arrayappend(HSQIRRELM v,SQInteger idx)
00603 {
00604     sq_aux_paramscheck(v,2);
00605     SQObjectPtr *arr;
00606     _GETSAFE_OBJ(v, idx, OT_ARRAY,arr);
00607     _array(*arr)->Append(v->GetUp(-1));
00608     v->Pop();
00609     return SQ_OK;
00610 }
00611
00620 SQRESULT sq_arraypop(HSQIRRELM v,SQInteger idx,SQBool pushval)
00621 {
00622     sq_aux_paramscheck(v, 1);
00623     SQObjectPtr *arr;
00624     _GETSAFE_OBJ(v, idx, OT_ARRAY,arr);
00625     if(_array(*arr)->Size() > 0) {
00626         if(pushval != 0){ v->Push(_array(*arr)->Top()); }
00627         _array(*arr)->Pop();
00628         return SQ_OK;
00629     }
00630     return sq_throwerror(v, _SC("empty array"));
00631 }
00632
00641 SQRESULT sq_arrayresize(HSQIRRELM v,SQInteger idx,SQInteger newsize)
00642 {
00643     sq_aux_paramscheck(v,1);
00644     SQObjectPtr *arr;
00645     _GETSAFE_OBJ(v, idx, OT_ARRAY,arr);
00646     if(newsize >= 0) {
00647         _array(*arr)->Resize(newsize);
00648         return SQ_OK;
00649     }
00650     return sq_throwerror(v,_SC("negative size"));
00651 }
00652
00653
00661 SQRESULT sq_arrayreverse(HSQIRRELM v,SQInteger idx)
00662 {
00663     sq_aux_paramscheck(v, 1);
00664     SQObjectPtr *o;
00665     _GETSAFE_OBJ(v, idx, OT_ARRAY,o);
00666     SQArray *arr = _array(*o);
00667     if(arr->Size() > 0) {
00668         SQObjectPtr t;
00669         SQInteger size = arr->Size();
00670         SQInteger n = size » 1; size -= 1;
00671         for(SQInteger i = 0; i < n; i++) {
00672             t = arr->_values[i];
00673             arr->_values[i] = arr->_values[size-i];
00674             arr->_values[size-i] = t;
00675         }
00676         return SQ_OK;
00677     }
00678     return SQ_OK;
00679 }
00680
00689 SQRESULT sq_arrayremove(HSQIRRELM v,SQInteger idx,SQInteger itemidx)
00690 {
00691     sq_aux_paramscheck(v, 1);
00692     SQObjectPtr *arr;
00693     _GETSAFE_OBJ(v, idx, OT_ARRAY,arr);
00694     return _array(*arr)->Remove(itemidx) ? SQ_OK : sq_throwerror(v,_SC("index out of range"));
00695 }
00696
00705 SQRESULT sq_arrayinsert(HSQIRRELM v,SQInteger idx,SQInteger destpos)
00706 {
00707     sq_aux_paramscheck(v, 1);
00708     SQObjectPtr *arr;
00709     _GETSAFE_OBJ(v, idx, OT_ARRAY,arr);
00710     SQRESULT ret = _array(*arr)->Insert(destpos, v->GetUp(-1)) ? SQ_OK : sq_throwerror(v,_SC("index
out of range"));
00711     v->Pop();
00712     return ret;
00713 }
00714
00722 void sq_newclosure(HSQIRRELM v,SQFUNCTION func,SQUnsignedInteger nfreevars)

```

```

00723 {
00724     SQNativeClosure *nc = SQNativeClosure::Create(_ss(v), func, nfreevars);
00725     nc->_nparamscheck = 0;
00726     for(SQUnsignedInteger i = 0; i < nfreevars; i++) {
00727         nc->_noutervalues[i] = v->Top();
00728         v->Pop();
00729     }
00730     v->Push(SQObjectPtr(nc));
00731 }
00732
00742 SQRESULT sq_getclosureinfo(HSQUIRRELV v, SQInteger idx, SQInteger *nparams, SQInteger *nfreevars)
00743 {
00744     SQObject o = stack_get(v, idx);
00745     if(sq_type(o) == OT_CLOSURE) {
00746         SQClosure *c = _closure(o);
00747         SQFunctionProto *proto = c->_function;
00748         *nparams = proto->_nparameters;
00749         *nfreevars = proto->_noutervalues;
00750         return SQ_OK;
00751     }
00752     else if(sq_type(o) == OT_NATIVECLOSURE)
00753     {
00754         SQNativeClosure *c = _nativeclosure(o);
00755         *nparams = c->_nparamscheck;
00756         *nfreevars = (SQInteger)c->_noutervalues;
00757         return SQ_OK;
00758     }
00759     return sq_throwerror(v, _SC("the object is not a closure"));
00760 }
00761
00770 SQRESULT sq_setnativeclosurename(HSQUIRRELV v, SQInteger idx, const SQChar *name)
00771 {
00772     SQObject o = stack_get(v, idx);
00773     if(sq_isnativeclosure(o)) {
00774         SQNativeClosure *nc = _nativeclosure(o);
00775         nc->_name = SQString::Create(_ss(v), name);
00776         return SQ_OK;
00777     }
00778     return sq_throwerror(v, _SC("the object is not a nativeclosure"));
00779 }
00780
00789 SQRESULT sq_setparamscheck(HSQUIRRELV v, SQInteger nparamscheck, const SQChar *typemask)
00790 {
00791     SQObject o = stack_get(v, -1);
00792     if(!sq_isnativeclosure(o))
00793         return sq_throwerror(v, _SC("native closure expected"));
00794     SQNativeClosure *nc = _nativeclosure(o);
00795     nc->_nparamscheck = nparamscheck;
00796     if(typemask) {
00797         SQIntVec res;
00798         if(!CompileTypemask(res, typemask))
00799             return sq_throwerror(v, _SC("invalid typemask"));
00800         nc->_typecheck.copy(res);
00801     }
00802     else {
00803         nc->_typecheck.resize(0);
00804     }
00805     if(nparamscheck == SQ_MATCHTYPEMASKSTRING) {
00806         nc->_nparamscheck = nc->_typecheck.size();
00807     }
00808     return SQ_OK;
00809 }
00810
00818 SQRESULT sq_bindenv(HSQUIRRELV v, SQInteger idx)
00819 {
00820     SQObjectPtr &o = stack_get(v, idx);
00821     if(!sq_isnativeclosure(o) &&
00822         !sq_isclosure(o))
00823         return sq_throwerror(v, _SC("the target is not a closure"));
00824     SQObjectPtr &env = stack_get(v, -1);
00825     if(!sq_istable(env) &&
00826         !sq_isarray(env) &&
00827         !sq_isclass(env) &&
00828         !sq_isinstance(env))
00829         return sq_throwerror(v, _SC("invalid environment"));
00830     SQWeakRef *w = _refcounted(env)->GetWeakRef(sq_type(env));
00831     SQObjectPtr ret;
00832     if(sq_isclosure(o)) {
00833         SQClosure *c = _closure(o)->Clone();
00834         __ObjRelease(c->_env);
00835         c->_env = w;
00836         __ObjAddRef(c->_env);
00837         if(_closure(o)->_base) {
00838             c->_base = _closure(o)->_base;
00839             __ObjAddRef(c->_base);
00840         }
00841         ret = c;

```

```

00842     }
00843     else { //then must be a native closure
00844         SQNativeClosure *c = _nativeclosure(o)->Clone();
00845         __ObjRelease(c->_env);
00846         c->_env = w;
00847         __ObjAddRef(c->_env);
00848         ret = c;
00849     }
00850     v->Pop();
00851     v->Push(ret);
00852     return SQ_OK;
00853 }
00854
00862 SQRRESULT sq_getclosurename(HSQIRRELV v, SQInteger idx)
00863 {
00864     SQObjectPtr &o = stack_get(v, idx);
00865     if(!sq_isnativeclosure(o) &&
00866         !sq_isclosure(o))
00867         return sq_throwerror(v, _SC("the target is not a closure"));
00868     if(sq_isnativeclosure(o))
00869     {
00870         v->Push(_nativeclosure(o)->_name);
00871     }
00872     else { //closure
00873         v->Push(_closure(o)->_function->_name);
00874     }
00875     return SQ_OK;
00876 }
00877
00885 SQRRESULT sq_setclosureroot(HSQIRRELV v, SQInteger idx)
00886 {
00887     SQObjectPtr &c = stack_get(v, idx);
00888     SQObject o = stack_get(v, -1);
00889     if(!sq_isclosure(c)) return sq_throwerror(v, _SC("closure expected"));
00890     if(sq_istable(o)) {
00891         _closure(c)->SetRoot(_table(o)->GetWeakRef(OT_TABLE));
00892         v->Pop();
00893         return SQ_OK;
00894     }
00895     return sq_throwerror(v, _SC("invalid type"));
00896 }
00897
00905 SQRRESULT sq_getclosureroot(HSQIRRELV v, SQInteger idx)
00906 {
00907     SQObjectPtr &c = stack_get(v, idx);
00908     if(!sq_isclosure(c)) return sq_throwerror(v, _SC("closure expected"));
00909     v->Push(_closure(c)->_root->_obj);
00910     return SQ_OK;
00911 }
00912
00920 SQRRESULT sq_clear(HSQIRRELV v, SQInteger idx)
00921 {
00922     SQObject &o=stack_get(v, idx);
00923     switch(sq_type(o)) {
00924         case OT_TABLE: _table(o)->Clear(); break;
00925         case OT_ARRAY: _array(o)->Resize(0); break;
00926         default:
00927             return sq_throwerror(v, _SC("clear only works on table and array"));
00928             break;
00929     }
00930 }
00931     return SQ_OK;
00932 }
00933
00939 void sq_pushroottable(HSQIRRELV v)
00940 {
00941     v->Push(v->_roottable);
00942 }
00943
00949 void sq_pushregistrytable(HSQIRRELV v)
00950 {
00951     v->Push(_ss(v)->_registry);
00952 }
00953
00959 void sq_pushconsttable(HSQIRRELV v)
00960 {
00961     v->Push(_ss(v)->_consts);
00962 }
00963
00970 SQRRESULT sq_setroottable(HSQIRRELV v)
00971 {
00972     SQObject o = stack_get(v, -1);
00973     if(sq_istable(o) || sq_isnull(o)) {
00974         v->_roottable = o;
00975         v->Pop();
00976         return SQ_OK;
00977     }

```



```

00978     return sq_throwerror(v, _SC("invalid type"));
00979 }
00980
00987 SQRRESULT sq_setconsttable(HSQIRRELVM v)
00988 {
00989     SQObject o = stack_get(v, -1);
00990     if(sq_istable(o)) {
00991         _ss(v)->_consts = o;
00992         v->Pop();
00993         return SQ_OK;
00994     }
00995     return sq_throwerror(v, _SC("invalid type, expected table"));
00996 }
00997
01004 void sq_setforeignptr(HSQIRRELVM v, SQUserPointer p)
01005 {
01006     v->_foreignptr = p;
01007 }
01008
01015 SQUserPointer sq_getforeignptr(HSQIRRELVM v)
01016 {
01017     return v->_foreignptr;
01018 }
01019
01026 void sq_setsharedforeignptr(HSQIRRELVM v, SQUserPointer p)
01027 {
01028     _ss(v)->_foreignptr = p;
01029 }
01030
01037 SQUserPointer sq_getsharedforeignptr(HSQIRRELVM v)
01038 {
01039     return _ss(v)->_foreignptr;
01040 }
01041
01048 void sq_setvmreleasehook(HSQIRRELVM v, SQRELEASEHOOK hook)
01049 {
01050     v->_releasehook = hook;
01051 }
01052
01059 SQRELEASEHOOK sq_getvmreleasehook(HSQIRRELVM v)
01060 {
01061     return v->_releasehook;
01062 }
01063
01070 void sq_setsharedreleasehook(HSQIRRELVM v, SQRELEASEHOOK hook)
01071 {
01072     _ss(v)->_releasehook = hook;
01073 }
01074
01081 SQRELEASEHOOK sq_getsharedreleasehook(HSQIRRELVM v)
01082 {
01083     return _ss(v)->_releasehook;
01084 }
01085
01086
01093 void sq_push(HSQIRRELVM v, SQInteger idx)
01094 {
01095     v->Push(stack_get(v, idx));
01096 }
01097
01105 SQObjectType sq_gettype(HSQIRRELVM v, SQInteger idx)
01106 {
01107     return sq_type(stack_get(v, idx));
01108 }
01109
01117 SQRRESULT sq_typeof(HSQIRRELVM v, SQInteger idx)
01118 {
01119     SQObjectPtr &o = stack_get(v, idx);
01120     SQObjectPtr res;
01121     if(!v->TypeOf(o, res)) {
01122         return SQ_ERROR;
01123     }
01124     v->Push(res);
01125     return SQ_OK;
01126 }
01127
01135 SQRRESULT sq_tostring(HSQIRRELVM v, SQInteger idx)
01136 {
01137     SQObjectPtr &o = stack_get(v, idx);
01138     SQObjectPtr res;
01139     if(!v->ToString(o, res)) {
01140         return SQ_ERROR;
01141     }
01142     v->Push(res);
01143     return SQ_OK;
01144 }
01145

```

```

01153 void sq_tobool(HSQUIRRELM v, SQInteger idx, SQBool *b)
01154 {
01155     SQObjectPtr &o = stack_get(v, idx);
01156     *b = SQVM::IsFalse(o)?SQFalse:SQTrue;
01157 }
01158
01167 SQRESULT sq_getinteger(HSQUIRRELM v, SQInteger idx, SQInteger *i)
01168 {
01169     SQObjectPtr &o = stack_get(v, idx);
01170     if(sq_isnumeric(o)) {
01171         *i = tointeger(o);
01172         return SQ_OK;
01173     }
01174     if(sq_isbool(o)) {
01175         *i = SQVM::IsFalse(o)?SQFalse:SQTrue;
01176         return SQ_OK;
01177     }
01178     return SQ_ERROR;
01179 }
01180
01189 SQRESULT sq_getfloat(HSQUIRRELM v, SQInteger idx, SQFloat *f)
01190 {
01191     SQObjectPtr &o = stack_get(v, idx);
01192     if(sq_isnumeric(o)) {
01193         *f = tofloat(o);
01194         return SQ_OK;
01195     }
01196     return SQ_ERROR;
01197 }
01198
01207 SQRESULT sq_getbool(HSQUIRRELM v, SQInteger idx, SQBool *b)
01208 {
01209     SQObjectPtr &o = stack_get(v, idx);
01210     if(sq_isbool(o)) {
01211         *b = _integer(o);
01212         return SQ_OK;
01213     }
01214     return SQ_ERROR;
01215 }
01216
01226 SQRESULT sq_getstringandsize(HSQUIRRELM v, SQInteger idx, const SQChar **c, SQInteger *size)
01227 {
01228     SQObjectPtr *o = NULL;
01229     _GETSAFE_OBJ(v, idx, OT_STRING, o);
01230     *c = _stringval(*o);
01231     *size = _string(*o)->_len;
01232     return SQ_OK;
01233 }
01234
01243 SQRESULT sq_getstring(HSQUIRRELM v, SQInteger idx, const SQChar **c)
01244 {
01245     SQObjectPtr *o = NULL;
01246     _GETSAFE_OBJ(v, idx, OT_STRING, o);
01247     *c = _stringval(*o);
01248     return SQ_OK;
01249 }
01250
01259 SQRESULT sq_getthread(HSQUIRRELM v, SQInteger idx, HSQUIRRELM *thread)
01260 {
01261     SQObjectPtr *o = NULL;
01262     _GETSAFE_OBJ(v, idx, OT_THREAD, o);
01263     *thread = _thread(*o);
01264     return SQ_OK;
01265 }
01266
01274 SQRESULT sq_clone(HSQUIRRELM v, SQInteger idx)
01275 {
01276     SQObjectPtr &o = stack_get(v, idx);
01277     v->PushNull();
01278     if(!v->Clone(o, stack_get(v, -1))){
01279         v->Pop();
01280         return SQ_ERROR;
01281     }
01282     return SQ_OK;
01283 }
01284
01298 SQInteger sq_getsize(HSQUIRRELM v, SQInteger idx)
01299 {
01300     SQObjectPtr &o = stack_get(v, idx);
01301     SQObjectType type = sq_type(o);
01302     switch(type) {
01303     case OT_STRING:      return _string(o)->_len;
01304     case OT_TABLE:      return _table(o)->CountUsed();
01305     case OT_ARRAY:      return _array(o)->Size();
01306     case OT_USERDATA:   return _userdata(o)->_size;
01307     case OT_INSTANCE:   return _instance(o)->_class->_udsize;
01308     case OT_CLASS:      return _class(o)->_udsize;

```

```

01309     default:
01310         return sq_aux_invalidtype(v, type);
01311     }
01312 }
01313
01321 SQHash sq_gethash(HSQIRRELV v, SQInteger idx)
01322 {
01323     SQObjectPtr &o = stack_get(v, idx);
01324     return HashObj(o);
01325 }
01326
01336 SQRESULT sq_getuserdata(HSQIRRELV v, SQInteger idx, SQUserPointer *p, SQUserPointer *typetag)
01337 {
01338     SQObjectPtr *o = NULL;
01339     _GETSAFE_OBJ(v, idx, OT_USERDATA, o);
01340     (*p) = _userdataval(*o);
01341     if (typetag) *typetag = _userdata(*o)->_typetag;
01342     return SQ_OK;
01343 }
01344
01353 SQRESULT sq_settypetag(HSQIRRELV v, SQInteger idx, SQUserPointer typetag)
01354 {
01355     SQObjectPtr &o = stack_get(v, idx);
01356     switch(sq_type(o)) {
01357         case OT_USERDATA: _userdata(o)->_typetag = typetag; break;
01358         case OT_CLASS: _class(o)->_typetag = typetag; break;
01359         default: return sq_throwerror(v, _SC("invalid object type"));
01360     }
01361     return SQ_OK;
01362 }
01363
01371 SQRESULT sq_getobjtypetag(const HSQOBJECT *o, SQUserPointer * typetag)
01372 {
01373     switch(sq_type(*o)) {
01374         case OT_INSTANCE: *typetag = _instance(*o)->_class->_typetag; break;
01375         case OT_USERDATA: *typetag = _userdata(*o)->_typetag; break;
01376         case OT_CLASS: *typetag = _class(*o)->_typetag; break;
01377         default: return SQ_ERROR;
01378     }
01379     return SQ_OK;
01380 }
01381
01390 SQRESULT sq_gettypetag(HSQIRRELV v, SQInteger idx, SQUserPointer *typetag)
01391 {
01392     SQObjectPtr &o = stack_get(v, idx);
01393     if (SQ_FAILED(sq_getobjtypetag(&o, typetag)))
01394         return SQ_ERROR; // this is not an error it should be a bool but would break backward
01395     return SQ_OK;
01396 }
01397
01406 SQRESULT sq_getuserpointer(HSQIRRELV v, SQInteger idx, SQUserPointer *p)
01407 {
01408     SQObjectPtr *o = NULL;
01409     _GETSAFE_OBJ(v, idx, OT_USERPOINTER, o);
01410     (*p) = _userpointer(*o);
01411     return SQ_OK;
01412 }
01413
01422 SQRESULT sq_setinstanceup(HSQIRRELV v, SQInteger idx, SQUserPointer p)
01423 {
01424     SQObjectPtr &o = stack_get(v, idx);
01425     if (sq_type(o) != OT_INSTANCE) return sq_throwerror(v, _SC("the object is not a class instance"));
01426     _instance(o)->_userpointer = p;
01427     return SQ_OK;
01428 }
01429
01438 SQRESULT sq_setclassudsize(HSQIRRELV v, SQInteger idx, SQInteger udszie)
01439 {
01440     SQObjectPtr &o = stack_get(v, idx);
01441     if (sq_type(o) != OT_CLASS) return sq_throwerror(v, _SC("the object is not a class"));
01442     if (_class(o)->_locked) return sq_throwerror(v, _SC("the class is locked"));
01443     _class(o)->_udsize = udszie;
01444     return SQ_OK;
01445 }
01446
01457 SQRESULT sq_getinstanceup(HSQIRRELV v, SQInteger idx, SQUserPointer *p, SQUserPointer typetag,
01458     SQBool throwerror)
01459 {
01460     SQObjectPtr &o = stack_get(v, idx);
01461     if (sq_type(o) != OT_INSTANCE) return throwerror ? sq_throwerror(v, _SC("the object is not a class
01462 instance")) : SQ_ERROR;
01463     (*p) = _instance(o)->_userpointer;
01464     if (typetag != 0) {
01465         SQClass *cl = _instance(o)->_class;
01466         do {
01467             if (cl->_typetag == typetag)

```

```

01466         return SQ_OK;
01467         cl = cl->_base;
01468     } while (cl != NULL);
01469     return throwerror ? sq_throwerror(v, _SC("invalid type tag")) : SQ_ERROR;
01470 }
01471 return SQ_OK;
01472 }
01473
01480 SQInteger sq_gettop(HSQIRRELVM v)
01481 {
01482     return (v->_top) - v->_stackbase;
01483 }
01484
01491 void sq_settop(HSQIRRELVM v, SQInteger newtop)
01492 {
01493     SQInteger top = sq_gettop(v);
01494     if (top > newtop)
01495         sq_pop(v, top - newtop);
01496     else
01497         while (top++ < newtop) sq_pushnull(v);
01498 }
01499
01506 void sq_pop(HSQIRRELVM v, SQInteger nelemstopop)
01507 {
01508     assert(v->_top >= nelemstopop);
01509     v->Pop(nelemstopop);
01510 }
01511
01517 void sq_poptop(HSQIRRELVM v)
01518 {
01519     assert(v->_top >= 1);
01520     v->Pop();
01521 }
01522
01523
01530 void sq_remove(HSQIRRELVM v, SQInteger idx)
01531 {
01532     v->Remove(idx);
01533 }
01534
01541 SQInteger sq_cmp(HSQIRRELVM v)
01542 {
01543     SQInteger res;
01544     v->ObjCmp(stack_get(v, -1), stack_get(v, -2), res);
01545     return res;
01546 }
01547
01556 SQRESULT sq_newslot(HSQIRRELVM v, SQInteger idx, SQBool bstatic)
01557 {
01558     sq_aux_paramscheck(v, 3);
01559     SQObjectPtr &self = stack_get(v, idx);
01560     if (sq_type(self) == OT_TABLE || sq_type(self) == OT_CLASS) {
01561         SQObjectPtr &key = v->GetUp(-2);
01562         if (sq_type(key) == OT_NULL) return sq_throwerror(v, _SC("null is not a valid key"));
01563         v->NewSlot(self, key, v->GetUp(-1), bstatic?true:false);
01564         v->Pop(2);
01565     }
01566     return SQ_OK;
01567 }
01568
01577 SQRESULT sq_deleteslot(HSQIRRELVM v, SQInteger idx, SQBool pushval)
01578 {
01579     sq_aux_paramscheck(v, 2);
01580     SQObjectPtr *self;
01581     _GETSAFE_OBJ(v, idx, OT_TABLE, self);
01582     SQObjectPtr &key = v->GetUp(-1);
01583     if (sq_type(key) == OT_NULL) return sq_throwerror(v, _SC("null is not a valid key"));
01584     SQObjectPtr res;
01585     if (!v->DeleteSlot(*self, key, res)) {
01586         v->Pop();
01587         return SQ_ERROR;
01588     }
01589     if (pushval) v->GetUp(-1) = res;
01590     else v->Pop();
01591     return SQ_OK;
01592 }
01593
01601 SQRESULT sq_set(HSQIRRELVM v, SQInteger idx)
01602 {
01603     SQObjectPtr &self = stack_get(v, idx);
01604     if (v->Set(self, v->GetUp(-2), v->GetUp(-1), DONT_FALL_BACK)) {
01605         v->Pop(2);
01606         return SQ_OK;
01607     }
01608     return SQ_ERROR;
01609 }
01610

```

```

01618 SQRESULT sq_rawset(HSQIRRELVM v, SQInteger idx)
01619 {
01620     SQObjectPtr &self = stack_get(v, idx);
01621     SQObjectPtr &key = v->GetUp(-2);
01622     if(sq_type(key) == OT_NULL) {
01623         v->Pop(2);
01624         return sq_throwerror(v, _SC("null key"));
01625     }
01626     switch(sq_type(self)) {
01627     case OT_TABLE:
01628         _table(self)->NewSlot(key, v->GetUp(-1));
01629         v->Pop(2);
01630         return SQ_OK;
01631     break;
01632     case OT_CLASS:
01633         _class(self)->NewSlot(_ss(v), key, v->GetUp(-1), false);
01634         v->Pop(2);
01635         return SQ_OK;
01636     break;
01637     case OT_INSTANCE:
01638         if(_instance(self)->Set(key, v->GetUp(-1))) {
01639             v->Pop(2);
01640             return SQ_OK;
01641         }
01642     break;
01643     case OT_ARRAY:
01644         if(v->Set(self, key, v->GetUp(-1), false)) {
01645             v->Pop(2);
01646             return SQ_OK;
01647         }
01648     break;
01649     default:
01650         v->Pop(2);
01651         return sq_throwerror(v, _SC("rawset works only on array/table/class and instance"));
01652     }
01653     v->Raise_IdxError(v->GetUp(-2)); return SQ_ERROR;
01654 }
01655
01664 SQRESULT sq_newmember(HSQIRRELVM v, SQInteger idx, SQBool bstatic)
01665 {
01666     SQObjectPtr &self = stack_get(v, idx);
01667     if(sq_type(self) != OT_CLASS) return sq_throwerror(v, _SC("new member only works with classes"));
01668     SQObjectPtr &key = v->GetUp(-3);
01669     if(sq_type(key) == OT_NULL) return sq_throwerror(v, _SC("null key"));
01670     if(!v->NewSlotA(self, key, v->GetUp(-2), v->GetUp(-1), bstatic?true:false, false)) {
01671         v->Pop(3);
01672         return SQ_ERROR;
01673     }
01674     v->Pop(3);
01675     return SQ_OK;
01676 }
01677
01686 SQRESULT sq_rawnewmember(HSQIRRELVM v, SQInteger idx, SQBool bstatic)
01687 {
01688     SQObjectPtr &self = stack_get(v, idx);
01689     if(sq_type(self) != OT_CLASS) return sq_throwerror(v, _SC("new member only works with classes"));
01690     SQObjectPtr &key = v->GetUp(-3);
01691     if(sq_type(key) == OT_NULL) return sq_throwerror(v, _SC("null key"));
01692     if(!v->NewSlotA(self, key, v->GetUp(-2), v->GetUp(-1), bstatic?true:false, true)) {
01693         v->Pop(3);
01694         return SQ_ERROR;
01695     }
01696     v->Pop(3);
01697     return SQ_OK;
01698 }
01699
01707 SQRESULT sq_setdelegate(HSQIRRELVM v, SQInteger idx)
01708 {
01709     SQObjectPtr &self = stack_get(v, idx);
01710     SQObjectPtr &mt = v->GetUp(-1);
01711     SQObjectType type = sq_type(self);
01712     switch(type) {
01713     case OT_TABLE:
01714         if(sq_type(mt) == OT_TABLE) {
01715             if(!_table(self)->SetDelegate(_table(mt))) {
01716                 return sq_throwerror(v, _SC("delegate cycle"));
01717             }
01718             v->Pop();
01719         }
01720         else if(sq_type(mt) == OT_NULL) {
01721             _table(self)->SetDelegate(NULL); v->Pop(); }
01722         else return sq_aux_invalidtype(v, type);
01723     break;
01724     case OT_USERDATA:
01725         if(sq_type(mt) == OT_TABLE) {
01726             _userdata(self)->SetDelegate(_table(mt)); v->Pop(); }
01727         else if(sq_type(mt) == OT_NULL) {

```

```

01728         _userdata(self)->SetDelegate(NULL); v->Pop(); }
01729     else return sq_aux_invalidtype(v, type);
01730     break;
01731 default:
01732     return sq_aux_invalidtype(v, type);
01733     break;
01734 }
01735 return SQ_OK;
01736 }
01737
01746 SQRESULT sq_rawdeleteslot(HSQIRRELVM v, SQInteger idx, SQBool pushval)
01747 {
01748     sq_aux_paramscheck(v, 2);
01749     SQObjectPtr *self;
01750     _GETSAFE_OBJ(v, idx, OT_TABLE, self);
01751     SQObjectPtr &key = v->GetUp(-1);
01752     SQObjectPtr t;
01753     if(_table(*self)->Get(key,t)) {
01754         _table(*self)->Remove(key);
01755     }
01756     if(pushval != 0)
01757         v->GetUp(-1) = t;
01758     else
01759         v->Pop();
01760     return SQ_OK;
01761 }
01762
01770 SQRESULT sq_getdelegate(HSQIRRELVM v, SQInteger idx)
01771 {
01772     SQObjectPtr &self=stack_get(v,idx);
01773     switch(sq_type(self)) {
01774     case OT_TABLE:
01775     case OT_USERDATA:
01776         if(!_delegable(self)->_delegate){
01777             v->PushNull();
01778             break;
01779         }
01780         v->Push(SQObjectPtr(_delegable(self)->_delegate));
01781         break;
01782     default: return sq_throwerror(v,_SC("wrong type")); break;
01783     }
01784     return SQ_OK;
01785 }
01786 }
01787
01795 SQRESULT sq_get(HSQIRRELVM v, SQInteger idx)
01796 {
01797     SQObjectPtr &self=stack_get(v,idx);
01798     SQObjectPtr &obj = v->GetUp(-1);
01799     if(v->Get(self,obj,obj, false, DONT_FALL_BACK))
01800         return SQ_OK;
01801     v->Pop();
01802     return SQ_ERROR;
01803 }
01804
01812 SQRESULT sq_rawget(HSQIRRELVM v, SQInteger idx)
01813 {
01814     SQObjectPtr &self=stack_get(v,idx);
01815     SQObjectPtr &obj = v->GetUp(-1);
01816     switch(sq_type(self)) {
01817     case OT_TABLE:
01818         if(_table(self)->Get(obj,obj))
01819             return SQ_OK;
01820         break;
01821     case OT_CLASS:
01822         if(_class(self)->Get(obj,obj))
01823             return SQ_OK;
01824         break;
01825     case OT_INSTANCE:
01826         if(_instance(self)->Get(obj,obj))
01827             return SQ_OK;
01828         break;
01829     case OT_ARRAY: {
01830         if(sq_isnumeric(obj)){
01831             if(_array(self)->Get(tointeger(obj),obj)) {
01832                 return SQ_OK;
01833             }
01834         }
01835         else {
01836             v->Pop();
01837             return sq_throwerror(v,_SC("invalid index type for an array"));
01838         }
01839     }
01840     break;
01841     default:
01842         v->Pop();
01843         return sq_throwerror(v,_SC("rawget works only on array/table/instance and class"));

```

```

01844     }
01845     v->Pop();
01846     return sq_throwerror(v, _SC("the index doesn't exist"));
01847 }
01848
01857 SQRRESULT sq_getstackobj(HSQIRRELV v, SQInteger idx, HSQOBJECT *po)
01858 {
01859     *po=stack_get(v, idx);
01860     return SQ_OK;
01861 }
01862
01871 const SQChar *sq_getlocal(HSQIRRELV v, SQUnsignedInteger level, SQUnsignedInteger idx)
01872 {
01873     SQUnsignedInteger cstksize=v->_callsstacksize;
01874     SQUnsignedInteger lvl=(cstksize-level)-1;
01875     SQInteger stackbase=v->_stackbase;
01876     if(lvl<cstksize){
01877         for(SQUnsignedInteger i=0;i<level;i++){
01878             SQVM::CallInfo &ci=v->_callsstack[(cstksize-i)-1];
01879             stackbase=ci._prevstkbase;
01880         }
01881         SQVM::CallInfo &ci=v->_callsstack[lvl];
01882         if(sq_type(ci._closure)!=OT_CLOSURE)
01883             return NULL;
01884         SQClosure *c=_closure(ci._closure);
01885         SQFunctionProto *func=c->_function;
01886         if(func->_noutervalues > (SQInteger)idx) {
01887             v->Push(*_outer(c->_outervalues[idx])->_valptr);
01888             return _stringval(func->_outervalues[idx]._name);
01889         }
01890         idx -= func->_noutervalues;
01891         return func->GetLocal(v, stackbase, idx, (SQInteger)(ci._ip-func->_instructions)-1);
01892     }
01893     return NULL;
01894 }
01895
01902 void sq_pushobject(HSQIRRELV v, HSQOBJECT obj)
01903 {
01904     v->Push(SQObjectPtr(obj));
01905 }
01906
01912 void sq_resetobject(HSQOBJECT *po)
01913 {
01914     po->_unVal.pUserPointer=NULL; po->_type=OT_NULL;
01915 }
01916
01924 SQRRESULT sq_throwerror(HSQIRRELV v, const SQChar *err)
01925 {
01926     v->_lasterror=SQString::Create(_ss(v), err);
01927     return SQ_ERROR;
01928 }
01929
01936 SQRRESULT sq_throwobject(HSQIRRELV v)
01937 {
01938     v->_lasterror = v->GetUp(-1);
01939     v->Pop();
01940     return SQ_ERROR;
01941 }
01942
01943
01949 void sq_reseterror(HSQIRRELV v)
01950 {
01951     v->_lasterror.Null();
01952 }
01953
01959 void sq_getlasterror(HSQIRRELV v)
01960 {
01961     v->Push(v->_lasterror);
01962 }
01963
01971 SQRRESULT sq_reservestack(HSQIRRELV v, SQInteger nsize)
01972 {
01973     if (((SQUnsignedInteger)v->_top + nsize) > v->_stack.size()) {
01974         if(v->_nmetamethodscall) {
01975             return sq_throwerror(v, _SC("cannot resize stack while in a metamethod"));
01976         }
01977         v->_stack.resize(v->_stack.size() + ((v->_top + nsize) - v->_stack.size()));
01978     }
01979     return SQ_OK;
01980 }
01981
01990 SQRRESULT sq_resume(HSQIRRELV v, SQBool retval, SQBool raiseerror)
01991 {
01992     if (sq_type(v->GetUp(-1)) == OT_GENERATOR)
01993     {
01994         v->PushNull(); //retval
01995         if (!v->Execute(v->GetUp(-2), 0, v->_top, v->GetUp(-1), raiseerror,

```

```

    SQVM::ET_RESUME_GENERATOR))
01996     {v->Raise_Error(v->_lasterror); return SQ_ERROR;}
01997     if(!retval)
01998         v->Pop();
01999     return SQ_OK;
02000 }
02001 return sq_throwerror(v, _SC("only generators can be resumed"));
02002 }
02003
02013 SQRESULT sq_call(HSQUIRRELV v, SQInteger params, SQBool retval, SQBool raiseerror)
02014 {
02015     SQObjectPtr res;
02016     if(!v->Call(v->GetUp(-(params+1)), params, v->_top-params, res, raiseerror?true:false)){
02017         v->Pop(params); //pop args
02018         return SQ_ERROR;
02019     }
02020     if(!v->_suspended)
02021         v->Pop(params); //pop args
02022     if(retval)
02023         v->Push(res); // push result
02024     return SQ_OK;
02025 }
02026
02034 SQRESULT sq_tailcall(HSQUIRRELV v, SQInteger nparams)
02035 {
02036     SQObjectPtr &res = v->GetUp(-(nparams + 1));
02037     if (sq_type(res) != OT_CLOSURE) {
02038         return sq_throwerror(v, _SC("only closure can be tail called"));
02039     }
02040     SQClosure *clo = _closure(res);
02041     if (clo->_function->_bgenerator)
02042     {
02043         return sq_throwerror(v, _SC("generators cannot be tail called"));
02044     }
02045
02046     SQInteger stackbase = (v->_top - nparams) - v->_stackbase;
02047     if (!v->TailCall(clo, stackbase, nparams)) {
02048         return SQ_ERROR;
02049     }
02050     return SQ_TAILCALL_FLAG;
02051 }
02052
02059 SQRESULT sq_suspendvm(HSQUIRRELV v)
02060 {
02061     return v->Suspend();
02062 }
02063
02074 SQRESULT sq_wakeupvm(HSQUIRRELV v, SQBool wakeupret, SQBool retval, SQBool raiseerror, SQBool throwerror)
02075 {
02076     SQObjectPtr ret;
02077     if(!v->_suspended)
02078         return sq_throwerror(v, _SC("cannot resume a vm that is not running any code"));
02079     SQInteger target = v->_suspended_target;
02080     if(wakeupret) {
02081         if(target != -1) {
02082             v->GetAt(v->_stackbase+v->_suspended_target)=v->GetUp(-1); //retval
02083         }
02084         v->Pop();
02085     } else if(target != -1) { v->GetAt(v->_stackbase+v->_suspended_target).Null(); }
02086     SQObjectPtr dummy;
02087     if(!v->Execute(dummy, -1, -1, ret, raiseerror, throwerror?SQVM::ET_RESUME_THROW_VM :
    SQVM::ET_RESUME_VM)) {
02088         return SQ_ERROR;
02089     }
02090     if(retval)
02091         v->Push(ret);
02092     return SQ_OK;
02093 }
02094
02102 void sq_setreleasehook(HSQUIRRELV v, SQInteger idx, SQRELEASEHOOK hook)
02103 {
02104     SQObjectPtr &ud=stack_get(v, idx);
02105     switch(sq_type(ud) ) {
02106     case OT_USERDATA: _userdata(ud)->_hook = hook; break;
02107     case OT_INSTANCE: _instance(ud)->_hook = hook; break;
02108     case OT_CLASS: _class(ud)->_hook = hook; break;
02109     default: return;
02110     }
02111 }
02112
02120 SQRELEASEHOOK sq_getreleasehook(HSQUIRRELV v, SQInteger idx)
02121 {
02122     SQObjectPtr &ud=stack_get(v, idx);
02123     switch(sq_type(ud) ) {
02124     case OT_USERDATA: return _userdata(ud)->_hook; break;
02125     case OT_INSTANCE: return _instance(ud)->_hook; break;
02126     case OT_CLASS: return _class(ud)->_hook; break;

```



```

02127     default: return NULL;
02128 }
02129 }
02130
02137 void sq_setcompilererrorhandler(HSQIRRELVLM v,SQCOMPILERERROR f)
02138 {
02139     _ss(v)->_compilererrorhandler = f;
02140 }
02141
02150 SQRESULT sq_writeclosure(HSQIRRELVLM v,SQWRITEFUNC w,SQUserPointer up)
02151 {
02152     SQObjectPtr *o = NULL;
02153     _GETSAFE_OBJ(v, -1, OT_CLOSURE,o);
02154     unsigned short tag = SQ_BYTECODE_STREAM_TAG;
02155     if(_closure(*o)->_function->_noutervalues)
02156         return sq_throwerror(v,_SC("a closure with free variables bound cannot be serialized"));
02157     if(w(up,&tag,2) != 2)
02158         return sq_throwerror(v,_SC("io error"));
02159     if(!_closure(*o)->Save(v,up,w))
02160         return SQ_ERROR;
02161     return SQ_OK;
02162 }
02163
02172 SQRESULT sq_readclosure(HSQIRRELVLM v,SQREADFUNC r,SQUserPointer up)
02173 {
02174     SQObjectPtr closure;
02175
02176     unsigned short tag;
02177     if(r(up,&tag,2) != 2)
02178         return sq_throwerror(v,_SC("io error"));
02179     if(tag != SQ_BYTECODE_STREAM_TAG)
02180         return sq_throwerror(v,_SC("invalid stream"));
02181     if(!SQClosure::Load(v,up,r,closure))
02182         return SQ_ERROR;
02183     v->Push(closure);
02184     return SQ_OK;
02185 }
02186
02194 SQChar *sq_getscratchpad(HSQIRRELVLM v,SQInteger minsize)
02195 {
02196     return _ss(v)->GetScratchPad(minsize);
02197 }
02198
02205 SQRESULT sq_resurrectunreachable(HSQIRRELVLM v)
02206 {
02207     #ifndef NO_GARBAGE_COLLECTOR
02208         _ss(v)->ResurrectUnreachable(v);
02209         return SQ_OK;
02210     #else
02211         return sq_throwerror(v,_SC("sq_resurrectunreachable requires a garbage collector build"));
02212     #endif
02213 }
02214
02221 SQInteger sq_collectgarbage(HSQIRRELVLM v)
02222 {
02223     #ifndef NO_GARBAGE_COLLECTOR
02224         return _ss(v)->CollectGarbage(v);
02225     #else
02226         return -1;
02227     #endif
02228 }
02229
02236 SQRESULT sq_getcallee(HSQIRRELVLM v)
02237 {
02238     if(v->_callsstacksize > 1)
02239     {
02240         v->Push(v->_callsstack[v->_callsstacksize - 2]._closure);
02241         return SQ_OK;
02242     }
02243     return sq_throwerror(v,_SC("no closure in the calls stack"));
02244 }
02245
02254 const SQChar *sq_getfreevariable(HSQIRRELVLM v,SQInteger idx,SQUnsignedInteger nval)
02255 {
02256     SQObjectPtr &self=stack_get(v,idx);
02257     const SQChar *name = NULL;
02258     switch(sq_type(self))
02259     {
02260     case OT_CLOSURE:{
02261         SQClosure *clo = _closure(self);
02262         SQFunctionProto *fp = clo->_function;
02263         if(((SQUnsignedInteger)fp->_noutervalues) > nval) {
02264             v->Push(*(_outer(clo->_outervalues[nval])->_valptr));
02265             SQOuterVar &ov = fp->_outervalues[nval];
02266             name = _stringval(ov._name);
02267         }
02268     }

```

```

02269         break;
02270     case OT_NATIVECLOSURE:{
02271         SQNativeClosure *clo = _nativeclosure(self);
02272         if(clo->noutervalues > nval) {
02273             v->Push(clo->outervalues[nval]);
02274             name = _SC("@NATIVE");
02275         }
02276     }
02277     break;
02278     default: break; //shutup compiler
02279 }
02280 return name;
02281 }
02282
02291 SQRESULT sq_setfreevariable(HSQIRRELVM v,SQInteger idx,SQUnsignedInteger nval)
02292 {
02293     SQObjectPtr &self=stack_get(v,idx);
02294     switch(sq_type(self))
02295     {
02296     case OT_CLOSURE:{
02297         SQFunctionProto *fp = _closure(self)->_function;
02298         if(((SQUnsignedInteger)fp->noutervalues) > nval){
02299             *(_outer(_closure(self)->outervalues[nval])->_valptr) = stack_get(v,-1);
02300         }
02301         else return sq_throwerror(v,_SC("invalid free var index"));
02302     }
02303     break;
02304     case OT_NATIVECLOSURE:
02305         if(_nativeclosure(self)->noutervalues > nval){
02306             _nativeclosure(self)->outervalues[nval] = stack_get(v,-1);
02307         }
02308         else return sq_throwerror(v,_SC("invalid free var index"));
02309         break;
02310     default:
02311         return sq_aux_invalidtype(v, sq_type(self));
02312     }
02313     v->Pop();
02314     return SQ_OK;
02315 }
02316
02324 SQRESULT sq_setattributes(HSQIRRELVM v,SQInteger idx)
02325 {
02326     SQObjectPtr *o = NULL;
02327     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
02328     SQObjectPtr &key = stack_get(v,-2);
02329     SQObjectPtr &val = stack_get(v,-1);
02330     SQObjectPtr attrs;
02331     if(sq_type(key) == OT_NULL) {
02332         attrs = _class(*o)->_attributes;
02333         _class(*o)->_attributes = val;
02334         v->Pop(2);
02335         v->Push(attrs);
02336         return SQ_OK;
02337     }else if(_class(*o)->GetAttributes(key,attrs)) {
02338         _class(*o)->SetAttributes(key,val);
02339         v->Pop(2);
02340         v->Push(attrs);
02341         return SQ_OK;
02342     }
02343     return sq_throwerror(v,_SC("wrong index"));
02344 }
02345
02353 SQRESULT sq_getattributes(HSQIRRELVM v,SQInteger idx)
02354 {
02355     SQObjectPtr *o = NULL;
02356     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
02357     SQObjectPtr &key = stack_get(v,-1);
02358     SQObjectPtr attrs;
02359     if(sq_type(key) == OT_NULL) {
02360         attrs = _class(*o)->_attributes;
02361         v->Pop();
02362         v->Push(attrs);
02363         return SQ_OK;
02364     }
02365     else if(_class(*o)->GetAttributes(key,attrs)) {
02366         v->Pop();
02367         v->Push(attrs);
02368         return SQ_OK;
02369     }
02370     return sq_throwerror(v,_SC("wrong index"));
02371 }
02372
02381 SQRESULT sq_getmemberhandle(HSQIRRELVM v,SQInteger idx,HSQMEMBERHANDLE *handle)
02382 {
02383     SQObjectPtr *o = NULL;
02384     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
02385     SQObjectPtr &key = stack_get(v,-1);

```

```

02386     SQTable *m = _class(*o)->_members;
02387     SQObjectPtr val;
02388     if(m->Get(key,val)) {
02389         handle->_static = _isfield(val) ? SQFalse : SQTrue;
02390         handle->_index = _member_idx(val);
02391         v->Pop();
02392         return SQ_OK;
02393     }
02394     return sq_throwerror(v,_SC("wrong index"));
02395 }
02396
02406 SQRRESULT _getmemberbyhandle(HSQIRRELVM v,SQObjectPtr &self,const HSQMEMBERHANDLE *handle,SQObjectPtr
*&val)
02407 {
02408     switch(sq_type(self)) {
02409         case OT_INSTANCE: {
02410             SQInstance *i = _instance(self);
02411             if(handle->_static) {
02412                 SQClass *c = i->_class;
02413                 val = &c->_methods[handle->_index].val;
02414             }
02415             else {
02416                 val = &i->_values[handle->_index];
02417             }
02418         }
02419         break;
02420         case OT_CLASS: {
02421             SQClass *c = _class(self);
02422             if(handle->_static) {
02423                 val = &c->_methods[handle->_index].val;
02424             }
02425             else {
02426                 val = &c->_defaultvalues[handle->_index].val;
02427             }
02428         }
02429         break;
02430         default:
02431             return sq_throwerror(v,_SC("wrong type(expected class or instance)"));
02432     }
02433     return SQ_OK;
02434 }
02435
02445 SQRRESULT sq_getbyhandle(HSQIRRELVM v,SQInteger idx,const HSQMEMBERHANDLE *handle)
02446 {
02447     SQObjectPtr &self = stack_get(v,idx);
02448     SQObjectPtr *val = NULL;
02449     if(SQ_FAILED(_getmemberbyhandle(v,self,handle,val))) {
02450         return SQ_ERROR;
02451     }
02452     v->Push(_realval(*val));
02453     return SQ_OK;
02454 }
02455
02464 SQRRESULT sq_setbyhandle(HSQIRRELVM v,SQInteger idx,const HSQMEMBERHANDLE *handle)
02465 {
02466     SQObjectPtr &self = stack_get(v,idx);
02467     SQObjectPtr &newval = stack_get(v,-1);
02468     SQObjectPtr *val = NULL;
02469     if(SQ_FAILED(_getmemberbyhandle(v,self,handle,val))) {
02470         return SQ_ERROR;
02471     }
02472     *val = newval;
02473     v->Pop();
02474     return SQ_OK;
02475 }
02476
02484 SQRRESULT sq_getbase(HSQIRRELVM v,SQInteger idx)
02485 {
02486     SQObjectPtr *o = NULL;
02487     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
02488     if(_class(*o)->_base)
02489         v->Push(SQObjectPtr(_class(*o)->_base));
02490     else
02491         v->PushNull();
02492     return SQ_OK;
02493 }
02494
02502 SQRRESULT sq_getclass(HSQIRRELVM v,SQInteger idx)
02503 {
02504     SQObjectPtr *o = NULL;
02505     _GETSAFE_OBJ(v, idx, OT_INSTANCE,o);
02506     v->Push(SQObjectPtr(_instance(*o)->_class));
02507     return SQ_OK;
02508 }
02509
02517 SQRRESULT sq_createinstance(HSQIRRELVM v,SQInteger idx)

```

```

02518 {
02519     SQObjectPtr *o = NULL;
02520     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
02521     v->Push(_class(*o)->CreateInstance());
02522     return SQ_OK;
02523 }
02524
02531 void sq_weakref(HSQIRRELVM v,SQInteger idx)
02532 {
02533     SQObject &o=stack_get(v,idx);
02534     if(ISREFCOUNTED(sq_type(o))) {
02535         v->Push(_refcounted(o)->GetWeakRef(sq_type(o)));
02536         return;
02537     }
02538     v->Push(o);
02539 }
02540
02548 SQRESULT sq_getweakrefval(HSQIRRELVM v,SQInteger idx)
02549 {
02550     SQObjectPtr &o = stack_get(v,idx);
02551     if(sq_type(o) != OT_WEAKREF) {
02552         return sq_throwerror(v,_SC("the object must be a weakref"));
02553     }
02554     v->Push(_weakref(o)->_obj);
02555     return SQ_OK;
02556 }
02557
02565 SQRESULT sq_getdefaultdelegate(HSQIRRELVM v,SQObjectType t)
02566 {
02567     SQSharedState *ss = _ss(v);
02568     switch(t) {
02569     case OT_TABLE: v->Push(ss->_table_default_delegate); break;
02570     case OT_ARRAY: v->Push(ss->_array_default_delegate); break;
02571     case OT_STRING: v->Push(ss->_string_default_delegate); break;
02572     case OT_INTEGER: case OT_FLOAT: v->Push(ss->_number_default_delegate); break;
02573     case OT_GENERATOR: v->Push(ss->_generator_default_delegate); break;
02574     case OT_CLOSURE: case OT_NATIVECLOSURE: v->Push(ss->_closure_default_delegate); break;
02575     case OT_THREAD: v->Push(ss->_thread_default_delegate); break;
02576     case OT_CLASS: v->Push(ss->_class_default_delegate); break;
02577     case OT_INSTANCE: v->Push(ss->_instance_default_delegate); break;
02578     case OT_WEAKREF: v->Push(ss->_weakref_default_delegate); break;
02579     default: return sq_throwerror(v,_SC("the type doesn't have a default delegate"));
02580     }
02581     return SQ_OK;
02582 }
02583
02591 SQRESULT sq_next(HSQIRRELVM v,SQInteger idx)
02592 {
02593     SQObjectPtr o=stack_get(v,idx),&refpos = stack_get(v,-1),realkey,val;
02594     if(sq_type(o) == OT_GENERATOR) {
02595         return sq_throwerror(v,_SC("cannot iterate a generator"));
02596     }
02597     int faketojump;
02598     if(!v->FOREACH_OP(o,realkey,val,refpos,0,666,faketojump))
02599         return SQ_ERROR;
02600     if(faketojump != 666) {
02601         v->Push(realkey);
02602         v->Push(val);
02603         return SQ_OK;
02604     }
02605     return SQ_ERROR;
02606 }
02607
02614 struct BufState{
02615     const SQChar *buf;
02616     SQInteger ptr;
02617     SQInteger size;
02618 };
02619
02626 SQInteger buf_lexfeed(SQUserPointer file)
02627 {
02628     BufState *buf=(BufState*) file;
02629     if(buf->size<(buf->ptr+1))
02630         return 0;
02631     return buf->buf[buf->ptr++];
02632 }
02633
02644 SQRESULT sq_compilebuffer(HSQIRRELVM v,const SQChar *s,SQInteger size,const SQChar *sourcename,SQBool
raiseerror) {
02645     BufState buf;
02646     buf.buf = s;
02647     buf.size = size;
02648     buf.ptr = 0;
02649     return sq_compile(v, buf_lexfeed, &buf, sourcename, raiseerror);
02650 }
02651
02659 void sq_move(HSQIRRELVM dest,HSQIRRELVM src,SQInteger idx)

```

```
02660 {
02661     dest->Push(stack_get(src,idx));
02662 }
02663
02671 void sq_setprintfunc(HSQUIRRELM v, SQPRINTFUNCTION printfunc,SQPRINTFUNCTION errfunc)
02672 {
02673     _ss(v)->_printfunc = printfunc;
02674     _ss(v)->_errorfunc = errfunc;
02675 }
02676
02683 SQPRINTFUNCTION sq_getprintfunc(HSQUIRRELM v)
02684 {
02685     return _ss(v)->_printfunc;
02686 }
02687
02694 SQPRINTFUNCTION sq_geterrorfunc(HSQUIRRELM v)
02695 {
02696     return _ss(v)->_errorfunc;
02697 }
02698
02705 void *sq_malloc(SQUnsignedInteger size)
02706 {
02707     return SQ_MALLOC(size);
02708 }
02709
02718 void *sq_realloc(void* p,SQUnsignedInteger oldsize,SQUnsignedInteger newsize)
02719 {
02720     return SQ_REALLOC(p,oldsize,newsize);
02721 }
02722
02729 void sq_free(void *p,SQUnsignedInteger size)
02730 {
02731     SQ_FREE(p,size);
02732 }
```

